
Probabilistic Analysis of Algorithms and Randomized Algorithms

IE 3080, Spring 2026

University of Pittsburgh

Instructor: Prof. Amin Rahimian | Scribe: Donghwan Kim

Course Rationale

Randomization is a powerful tool in algorithm design when deterministic solutions are inefficient or hard to come by. Probabilistic analysis provides performance guarantees that reflect input properties and problem uncertainty. This advanced topics course introduces fundamental tools (concentration bounds, coupling, random graphs, probabilistic method, martingales) and explores seminal areas (optimal stopping, prophet and secretary inequalities, Pandora's box, multi-armed bandits, and inference/learning on trees and networks). The course emphasizes both technical mastery and critical engagement with research papers.

Learning Objectives

- Apply concentration inequalities, coupling, and random graph tools to analyze algorithm performance.
- Derive and interpret PAC learning bounds, VC-dimension, and Rademacher complexity.
- Understand and critique seminal results on prophet inequalities, secretary/Pandora's box, and multi-armed bandits.
- Analyze algorithms for inference and learning on trees and networks.
- Develop skills in presenting technical research and engaging in scholarly discussion.

Contents

1	Binomial Tails and the Chernoff Bound	6
1.1	Markov's Inequality	6
1.1.1	Application: Binomial Distribution	7
1.2	Chebyshev Bound	7
1.2.1	Example	8
1.2.2	Extension	8
1.2.3	Application: Binomial Distribution	8
1.3	Chernoff Bound	8
1.3.1	Derivation of Chernoff Bound for Binomial Distribution	9
1.3.2	Application: Binomial Distribution	12
1.4	Normal Approximation Bound	13
1.5	Comparison with Bounds	13
2	The Triangle of Polling and Differential Privacy	14
2.1	Triangle of Polling	14
2.1.1	Main Objective	15
2.1.2	Approach 1: Chebyshev's Inequality	15
2.1.3	Approach 2: Chernoff Bound	15
2.1.4	Approach 3: CLT and Mills' Inequality	16
2.2	Almost sure convergence in terms of Sets	17
2.2.1	Convergence in Probability vs. Almost Sure Convergence	17
2.2.2	Set-Theoretic Representation	17
2.2.3	Intersection of Probability 1 Events	17
2.2.4	Infinitely Often (i.o.)	18
2.2.5	The First Borel-Cantelli Lemma	18
2.2.6	Application: Convergence of Sample Means	18
2.3	Differential Privacy	19
2.3.1	Mathematical Definition	20
2.3.2	Example 1: A Naive Approach (Does not satisfy DP)	20
2.3.3	Example 2: Randomized Response (Satisfies DP)	21
2.3.4	Cost of Privacy (Sample Complexity)	21
3	Local and Central Differential Privacy	22
3.1	Cost of Privacy(ϵ -LDP)	23
3.1.1	Baseline: Without Privacy	23
3.1.2	With Privacy (ϵ -LDP)	23
3.1.3	Effective Sample Size	24
3.2	Local DP vs. Central DP	24
3.3	ϵ -Central Differential Privacy	25
3.3.1	The Optimization Problem	25
3.3.2	The Laplace Mechanism	25
3.3.3	Relation between b , n , and ϵ	26
3.4	Sample Complexity for ϵ -CDP	28
3.4.1	Error Decomposition	28
3.4.2	Solving for n	29
3.4.3	Optimization of Parameters	29

3.4.4	Comparison: CDP vs. LDP	30
4	Balls, Bins, and the Birthday Paradox	32
4.1	Balls and Bins	32
4.2	The Birthday Paradox	33
4.2.1	Exact Probability of No Collision	33
4.2.2	Approximation using Exponential Inequality	33
4.2.3	Approximation using Union Bound (Deferred Decisions)	34
4.2.4	Intuition: Exponential Decay over $\sqrt{n}$ Intervals	35
4.3	Coupon Collector's Problem	36
4.3.1	Derivation of the Expected Number of Balls	36
4.3.2	Bounding the Harmonic Series	37
4.3.3	Analysis of the Scaling Factor c	38
4.4	Alternative Analysis: Union Bound on Empty Bins	40
4.4.1	Calculating the Probability	40
4.4.2	Remark: Is this bound tight?	41
4.5	Poisson Approximation and Limiting Distribution	41
4.5.1	Conclusion	42
5	Coupon Collector and Load Balancing	43
5.1	Poisson Approximation with Independent Bins	43
5.1.1	Equivalence of the Models	44
5.2	Load Balancing	47
5.2.1	Bounding the Probability	48
5.2.2	Applying the Taylor Bound	48
5.2.3	Determining the Threshold K	48
6	Sorting, Quicksort, and Monte Carlo Min-Cut	52
6.1	Balls into Bins	52
6.2	Sorting Algorithms and Lower Bounds	53
6.2.1	The $\Omega(n \log n)$ Lower Bound	53
6.2.2	Bucket Sort	53
6.2.3	Expected Total Number of Comparisons in Bucket Sort	53
6.3	Comparison-Based Sorting Algorithms	54
6.3.1	Merge Sort	54
6.3.2	Quicksort	54
6.4	Monte Carlo Algorithms	57
6.4.1	Min-Cut	58
7	Randomized Selection and Polynomial Identity Testing	60
7.1	The k -Select Problem	60
7.2	Randomized Median-Select	62
7.2.1	Expected Complexity	62
7.2.2	Proof that $\mathbb{E}[C(n)] \leq 4n$	62
7.3	Approximate Median via Sampling	63
7.3.1	The Sampling Algorithm	64
7.3.2	Failure Events	64
7.3.3	Bounding the Failure Probability	65

7.3.4	Total Failure Probability	66
7.3.5	Computational Complexity	67
7.4	Computational Complexity: P, NP, and NP-Hard	67
7.4.1	Min-Cut vs. Max-Cut	67
7.4.2	NP and Certificates	68
7.5	Randomized Polynomial Identity Testing	68
7.5.1	The Problem	68
7.5.2	Deterministic Approach: $O(d^2)$	69
7.5.3	Verification at a Single Point: $O(d)$	69
7.5.4	A Randomized Checker	69
8	Min-Cut Amplification, Max-Cut, and Submodular Greedy	70
8.1	Min-Cut: Amplifying the Contraction Algorithm	71
8.1.1	Boosting Success by Independent Repetitions	71
8.1.2	A Smarter Amplification Strategy	71
8.2	Maximum Cut	72
8.2.1	Problem Definition	73
8.2.2	A Randomized Approximation: Coin-Flipping Algorithm	73
8.2.3	Concentration and the Probability of a Good Cut	74
8.2.4	Approximation Guarantee	75
8.3	Greedy Max-Cut	75
8.3.1	The Algorithm	75
8.3.2	Local Search Improvement	76
8.4	Greedy Algorithm for Monotone Submodular Maximization	76
8.4.1	Maximum Coverage	76
8.4.2	Monotonicity and Submodularity	77
8.5	The Greedy Algorithm and Its Guarantee	78
8.5.1	The Greedy Strategy	78
8.5.2	Warm-Up: Greedy Achieves $\frac{1}{2}$ OPT	78
8.5.3	The Tight Bound: Greedy Achieves $(1 - 1/e)$ OPT	79
8.6	Beyond Monotone Submodularity	80
9	Independent Sets, Satisfiability, and Random Walks	81
9.1	Independent Sets	81
9.2	A Randomized Algorithm for Independent Sets	82
9.3	Minimum Vertex Cover	82
9.4	Satisfiability	83
9.4.1	Historical Context	83
9.4.2	Conjunctive Normal Form and SAT	83
9.4.3	The Phase Transition	84
9.5	Maximum Satisfiability (MAX-SAT)	84
9.6	k -SAT and the Lovász Local Lemma	85
9.6.1	The Lovász Local Lemma	85
9.6.2	Application to k -SAT	86
9.7	2-SAT	86
9.7.1	The Implication Graph	86
9.7.2	A Randomized Algorithm for 2-SAT	87
9.7.3	Analysis via Random Walk	87

9.7.4	Hitting Time Analysis	87
9.7.5	Bounding the Failure Probability	88
9.8	3-SAT	88
9.8.1	The Random Walk for 3-SAT	88
9.8.2	Hitting Time Analysis	89
10	Improved 3-SAT and the Monte Carlo Method	90
10.1	An Improved Algorithm for 3-SAT	90
10.1.1	Random Initialization and Restarts	90
10.1.2	Analysis of the Success Probability	91
10.1.3	Applying Stirling's Approximation	92
10.1.4	Overall Success Probability	92
10.1.5	Expected Runtime	93
10.2	Amplifying the Success Probability	93
10.3	The Monte Carlo Method	94
10.3.1	Counting Problems and $\#P$	94
10.3.2	(ϵ, δ) -Approximation and FPRAS	94
10.4	The DNF Counting Problem	95
10.4.1	What is DNF?	96
10.4.2	Why is DNF Counting $\#P$ -Complete?	96
10.4.3	The Naïve Sampling Approach	96
10.5	An FPRAS for DNF Counting	97
10.5.1	Constructing the Universe U	97
10.5.2	Defining the Canonical Set S	97
10.5.3	Estimating $c(F)$ via Sampling from U	97
10.5.4	How to Sample Uniformly from U	98
11	Approximate Counting, MCMC, and Coupling	98
11.1	From Approximate Sampling to Approximate Counting	99
11.1.1	FPRAS and FPAUS	99
11.1.2	Self-Reducible Problems: Independent Sets	100
11.1.3	The Telescoping Product	100
11.1.4	The Self-Reducibility Lemma	100
11.1.5	Estimating r_i via Sampling	101
11.1.6	Sample Size	103
11.2	Markov Chain Monte Carlo (MCMC)	104
11.2.1	Time Reversibility and Detailed Balance	104
11.3	Total Variation Distance and Mixing Time	105
11.3.1	Total Variation Distance	105
11.3.2	Mixing Time	105
11.4	Coupling of Markov Chains	106
11.4.1	The Coupling Idea	106
11.4.2	The Coupling Lemma	106

Textbook reference conventions. Throughout these notes, inline tags such as [Mor: Thm. 18.1] pinpoint the corresponding result in one of the three course textbooks. The abbreviations are:

- [Mor] — Mor Harchol-Balter, *Introduction to Probability for Computing*, Cambridge University Press (2023).
- [MU] — Michael Mitzenmacher & Eli Upfal, *Probability and Computing*, 2nd ed., Cambridge University Press (2017).
- [AS] — Noga Alon & Joel Spencer, *The Probabilistic Method*, 4th ed., Wiley (2016).

1 Binomial Tails and the Chernoff Bound

Textbook Roadmap

For a parallel treatment in the course textbooks, see:

- *Markov & Chebyshev*: [Mor: Ch. 18, §§18.1–18.2]; [MU: Ch. 3, §§3.1, 3.3].
- *Chernoff bound (general & Binomial) and Hoeffding's MGF lemma*: [Mor: Ch. 18, §§18.3, 18.5, 18.10]; [MU: Ch. 4, §§4.1–4.2, 4.5].
- *Normal approximation & comparison of bounds*: [Mor: Ch. 18, §18.6]; [MU: Ch. 9, §9.3] (CLT background).

Pinpoint cites in the headers below tag each specific theorem/lemma to its textbook counterpart.

A central question in the probabilistic analysis of algorithms is how sharply a sum of independent random variables concentrates around its mean. This chapter develops a sequence of tail bounds of progressively increasing strength, using the binomial distribution as a running example. We start with the weakest universal bound that requires only the mean, then add information about the variance, and finally exploit the full distribution through the moment generating function.

1.1 Markov's Inequality

Theorem 1 (Markov's Inequality [Mor: Thm. 18.1] [MU: Thm. 3.1]). *For a random variable $X \geq 0$ and $a > 0$,*

$$\mathbb{P}[X \geq a] \leq \frac{\mathbb{E}[X]}{a}$$

Proof 1 (Continuous Case).

$$\begin{aligned} \mathbb{E}[X] &= \int_0^\infty x f(x) dx \\ &= \int_0^a x f(x) dx + \int_a^\infty x f(x) dx \end{aligned}$$

Since $X \geq 0$ and $f(x) \geq 0$ (as f is a pdf), the integrand $x f(x)$ is non-negative, so $\int_0^a x f(x) dx \geq 0$. Therefore:

$$\mathbb{E}[X] \geq \int_a^\infty x f(x) dx$$

Now, we bound the remaining integral. For $x \in [a, \infty)$ we have $x \geq a$, and since $f(x) \geq 0$, we get $x f(x) \geq a f(x)$:

$$\int_a^\infty x f(x) dx \geq \int_a^\infty a f(x) dx = a \cdot \mathbb{P}[X \geq a]$$

Rearranging terms yields the Markov inequality. □

Proof 2 (General Case). There is another proof because the previous one requires the continuity of $f(x)$. We use the indicator function $\mathbb{1}(X \geq a)$:

$$\mathbb{1}(X \geq a) = \begin{cases} 1 & \text{if } X \geq a \\ 0 & \text{if } X < a \end{cases}$$

Consider the inequality:

$$a \cdot \mathbb{1}(X \geq a) \leq X$$

This statement is **always true** because:

- If $X < a$: LHS is 0. Since $X \geq 0$, $0 \leq X$ holds.
- If $X \geq a$: LHS is a . Obviously, $a \leq X$ holds.

Apply expectation to both sides (inequality is preserved):

$$\begin{aligned} \mathbb{E}[a \cdot \mathbb{1}(X \geq a)] &\leq \mathbb{E}[X] \\ a \cdot \mathbb{E}[\mathbb{1}(X \geq a)] &\leq \mathbb{E}[X] \end{aligned}$$

Since $\mathbb{E}[\mathbb{1}(X \geq a)] = \mathbb{P}[X \geq a]$, we conclude:

$$\therefore \mathbb{P}[X \geq a] \leq \frac{\mathbb{E}[X]}{a}$$

□

1.1.1 Application: Binomial Distribution

Applying Markov bound to Binomial distribution:

$$X \sim \text{Binomial}(n, p) \quad \text{with} \quad p = 1/2$$

We want to bound the probability:

$$\mathbb{P}\left[X > \frac{3n}{4}\right] \leq \frac{\mathbb{E}[X]}{3n/4} = \frac{n/2}{3n/4} = \frac{2}{3}$$

Remark 2. The problem with the Markov bound is that it does not depend on n at all. Even as $n \rightarrow \infty$, the bound stays fixed at $2/3$, making it a **very weak and undesirable** bound in practice. It only uses the mean $\mathbb{E}[X]$, discarding all other information about the distribution.

1.2 Chebyshev Bound

Theorem 3 (Chebyshev's Inequality [Mor: Thm. 18.2] [MU: Thm. 3.6]).

$$\mathbb{P}[|X - \mu| > \varepsilon] \leq \frac{\text{Var}(X)}{\varepsilon^2}$$

This bounds the probability that a certain random variable is far from the mean.

Proof. Proving this is quite simple:

$$\begin{aligned} \mathbb{P}[|X - \mu| > \varepsilon] &= \mathbb{P}[(X - \mu)^2 > \varepsilon^2] \\ &\leq \frac{\mathbb{E}[(X - \mu)^2]}{\varepsilon^2} \quad (\text{by Markov's inequality, since } (X - \mu)^2 \geq 0) \end{aligned}$$

Since $\mathbb{E}[(X - \mu)^2] = \text{Var}(X)$, we obtain the Chebyshev bound.

□

1.2.1 Example

Set $S_n = \sum_{i=1}^n X_i$ where $X_i \sim \text{Bernoulli}(p)$ are i.i.d.

$$\mathbb{P} \left[\left| \frac{S_n}{n} - p \right| > \varepsilon \right] \leq \frac{\text{Var}(\frac{S_n}{n})}{\varepsilon^2} = \frac{p(1-p)}{n\varepsilon^2}$$

As $n \rightarrow \infty$, this probability goes to 0.

Remark 4. This highlights a big difference between the Chebyshev bound and Markov's inequality. It implies that as n increases, the observed value converges to the theoretical probability.

1.2.2 Extension

Additionally, using the same mechanism of the proof, we can derive higher-order bounds:

$$\mathbb{P}[|X - \mu| > \varepsilon] \leq \frac{\mathbb{E}[(X - \mu)^4]}{\varepsilon^4}$$

1.2.3 Application: Binomial Distribution

Let $X \sim \text{Binomial}(n, 1/2)$. We have:

$$\mu = \mathbb{E}[X] = \frac{n}{2}, \quad \text{Var}(X) = np(1-p) = \frac{n}{4}$$

We want to bound the probability $\mathbb{P}[X > 3n/4]$. We can relate this one-sided tail to the two-sided Chebyshev bound:

$$\mathbb{P} \left[X > \frac{3n}{4} \right] \leq \mathbb{P} \left[\left| X - \frac{n}{2} \right| > \frac{n}{4} \right]$$

Using Chebyshev's inequality with $\varepsilon = n/4$:

$$\mathbb{P} \left[\left| X - \frac{n}{2} \right| > \frac{n}{4} \right] \leq \frac{\text{Var}(X)}{(n/4)^2} = \frac{n/4}{n^2/16} = \frac{4}{n}$$

Comparison: Unlike the Markov bound (which was constant $2/3$), the Chebyshev bound $\frac{4}{n}$ decreases as n grows.

1.3 Chernoff Bound

This is the strongest bound as it uses the exponential form of the bound. Crucially, $\mathbb{E}[e^{tX}]$ is the *moment generating function* (MGF) of X ; since the MGF encodes *all* moments of X , the Chernoff bound exploits the full distributional information of the random variable, making it strictly tighter than bounds that rely only on the mean (Markov) or variance (Chebyshev). A key structural insight is that when $X = \sum_i X_i$ is a sum of *independent* random variables, the MGF factorizes as $\mathbb{E}[e^{tX}] = \prod_i \mathbb{E}[e^{tX_i}]$, converting a hard sum into a tractable product, and this is exactly what makes Step 2 of the derivation below work.

Applicability. The Chernoff bound is most powerful when (1) X is a sum of independent (or weakly dependent) bounded random variables, and (2) the MGF $\mathbb{E}[e^{tX}]$ exists and can be computed or bounded. For heavy-tailed distributions (e.g., Pareto, Cauchy) the MGF may not exist at all, in which case Chebyshev or other moment-based bounds are more appropriate.

Theorem 5 (Chernoff Bound [Mor: Thm. 18.3] [MU: §§4.1–4.2]). *For any random variable X and $a \in \mathbb{R}$,*

$$\mathbb{P}[X > a] \leq \inf_{t>0} \frac{\mathbb{E}[e^{tX}]}{e^{ta}}.$$

Proof. For any $t > 0$, the map $x \mapsto e^{tx}$ is strictly increasing, so

$$\mathbb{P}[X > a] = \mathbb{P}[tX > ta] = \mathbb{P}[e^{tX} > e^{ta}].$$

Since $e^{tX} \geq 0$, applying Markov's inequality to e^{tX} at threshold e^{ta} yields

$$\mathbb{P}[e^{tX} > e^{ta}] \leq \frac{\mathbb{E}[e^{tX}]}{e^{ta}}.$$

This bound holds for every $t > 0$, so we may optimize over t to get the infimum form. $\square$

Remark 6 (Lower tail). The same technique handles lower-tail bounds. For $t < 0$, multiplying by t reverses the inequality: $\mathbb{P}[X < a] = \mathbb{P}[tX > ta]$, so $\mathbb{P}[X < a] \leq \inf_{t<0} \frac{\mathbb{E}[e^{tX}]}{e^{ta}}$.

1.3.1 Derivation of Chernoff Bound for Binomial Distribution

Setup: Let $X = X_1 + X_2 + \cdots + X_n$, where $X_i \sim \text{Bernoulli}(p)$ are independent.

$$\mu = \mathbb{E}[X] = np$$

Goal: Bound $\mathbb{P}\{X - np \geq \delta\}$

Step 1: Exponentiation and Markov's Inequality. For any $t > 0$:

$$\begin{aligned} \mathbb{P}\{X - np \geq \delta\} &= \mathbb{P}\{t(X - np) \geq t\delta\} \\ &= \mathbb{P}\{e^{t(X-np)} \geq e^{t\delta}\} \\ &\leq \frac{\mathbb{E}[e^{t(X-np)}]}{e^{t\delta}} \quad (\text{Markov's inequality}) \end{aligned}$$

Step 2: Decomposing the Expectation.

$$\begin{aligned} \mathbb{E}[e^{t(X-np)}] &= \mathbb{E}\left[e^{t(-np + \sum_{i=1}^n X_i)}\right] \\ &= \mathbb{E}\left[e^{t \sum_{i=1}^n (X_i - p)}\right] \quad (\text{since } np = \sum_{i=1}^n p, \text{ distribute } -np \text{ into the sum}) \\ &= \mathbb{E}\left[\prod_{i=1}^n e^{t(X_i - p)}\right] \\ &= \prod_{i=1}^n \mathbb{E}[e^{t(X_i - p)}] \quad (\text{since } X_i\text{'s are independent}) \end{aligned}$$

Step 3: Computing Each Term. Since $X_i \sim \text{Bernoulli}(p)$:

$$\begin{aligned}\mathbb{E}[e^{t(X_i-p)}] &= p \cdot e^{t(1-p)} + (1-p) \cdot e^{t(0-p)} \\ &= p \cdot e^{tq} + q \cdot e^{-tp} \quad (\text{where } q = 1-p)\end{aligned}$$

Lemma 7 (Hoeffding's MGF Bound [Mor: Lem. 18.5 (proof in §18.10)] [MU: Lem. 4.13]). *For all $t > 0$, $0 < p < 1$, and $q = 1 - p$,*

$$pe^{tq} + qe^{-tp} \leq e^{t^2/8}.$$

Therefore:

$$\mathbb{E}[e^{t(X_i-p)}] \leq e^{t^2/8}$$

Step 4: Combining the Bound.

$$\begin{aligned}\prod_{i=1}^n \mathbb{E}[e^{t(X_i-p)}] &\leq \prod_{i=1}^n e^{t^2/8} \\ &= e^{nt^2/8}\end{aligned}$$

Thus:

$$\begin{aligned}\mathbb{P}\{X - np \geq \delta\} &\leq \frac{e^{nt^2/8}}{e^{t\delta}} \\ &= e^{nt^2/8 - t\delta} \\ &= e^{-(t\delta - nt^2/8)}\end{aligned}$$

Step 5: Finding the Optimal t . To minimize the exponent, we maximize $f(t) = t\delta - \frac{nt^2}{8}$:

$$f'(t) = \delta - \frac{2nt}{8} = \delta - \frac{nt}{4} = 0 \implies t^* = \frac{4\delta}{n}$$

Substituting the optimal value t^* :

$$\begin{aligned}f(t^*) &= \frac{4\delta}{n} \cdot \delta - n \cdot \left(\frac{4\delta}{n}\right)^2 \cdot \frac{1}{8} \\ &= \frac{4\delta^2}{n} - n \cdot \frac{16\delta^2}{n^2} \cdot \frac{1}{8} \\ &= \frac{4\delta^2}{n} - \frac{2\delta^2}{n} \\ &= \frac{2\delta^2}{n}\end{aligned}$$

Final Result — Additive Chernoff for Binomial [Mor: Thm. 18.4] [MU: Cor. 4.9–4.10]

$$\mathbb{P}\{X - np \geq \delta\} \leq \min_{t>0} e^{-(t\delta - nt^2/8)} = e^{-2\delta^2/n}$$

Proof of Lemma 7. To prove the inequality $pe^{t(1-p)} + (1-p)e^{-tp} \leq e^{t^2/8}$, we first define a centered random variable Y_i :

$$Y_i = \begin{cases} 1-p & \text{with probability } p \\ -p & \text{with probability } 1-p \end{cases}$$

Observe that $\mathbb{E}[Y_i] = 0$. The term we want to bound involves the Moment Generating Function (MGF) of Y_i :

$$\mathbb{E}[e^{tY_i}] = pe^{t(1-p)} + (1-p)e^{-tp}$$

Let $F(t) = \log \mathbb{E}[e^{tY_i}]$. We want to show $F(t) \leq \frac{t^2}{8}$. Define an auxiliary function:

$$G(t) = F(t) - \frac{t^2}{8}$$

We aim to show $G(t) \leq 0$ for all $t \geq 0$.

1. Checking initial conditions: Note that $G(0) = \log(1) - 0 = 0$. Also, the first derivative is:

$$G'(t) = F'(t) - \frac{t}{4}, \quad \text{where } F'(t) = \frac{\mathbb{E}[Y_i e^{tY_i}]}{\mathbb{E}[e^{tY_i}]}$$

At $t = 0$, we have $F'(0) = \mathbb{E}[Y_i] = 0$, so $G'(0) = 0$.

2. Analyzing the second derivative: By differentiating $F'(t)$ again using the quotient rule, we obtain:

$$F''(t) = \frac{\mathbb{E}[Y_i^2 e^{tY_i}] \mathbb{E}[e^{tY_i}] - (\mathbb{E}[Y_i e^{tY_i}])^2}{(\mathbb{E}[e^{tY_i}])^2} = \frac{\mathbb{E}[Y_i^2 e^{tY_i}]}{\mathbb{E}[e^{tY_i}]} - \left(\frac{\mathbb{E}[Y_i e^{tY_i}]}{\mathbb{E}[e^{tY_i}]} \right)^2$$

To analyze this expression, we introduce a new random variable Z_i whose distribution is a “tilted” version of Y_i . Specifically, we define the probability mass function of Z_i as:

$$\mathbb{P}(Z_i = y) = \frac{e^{ty}}{\mathbb{E}[e^{tY_i}]} \mathbb{P}(Y_i = y)$$

Under this definition, the moments of Z_i correspond to the terms in $F''(t)$:

$$\mathbb{E}[Z_i] = \sum_y y \mathbb{P}(Z_i = y) = \frac{\mathbb{E}[Y_i e^{tY_i}]}{\mathbb{E}[e^{tY_i}]}, \quad \mathbb{E}[Z_i^2] = \frac{\mathbb{E}[Y_i^2 e^{tY_i}]}{\mathbb{E}[e^{tY_i}]}$$

Substituting these back into the expression for $F''(t)$, we get:

$$F''(t) = \mathbb{E}[Z_i^2] - (\mathbb{E}[Z_i])^2 = \text{Var}(Z_i)$$

Thus, the second derivative of our goal function is:

$$G''(t) = \text{Var}(Z_i) - \frac{1}{4}$$

3. Bounding the Variance: The random variable Z_i takes values in the set $\{-p, 1-p\}$. Note that the distance between these two values is $(1-p) - (-p) = 1$. To see why the variance is bounded by $1/4$, consider a general random variable Y taking value a with probability q and b with

probability $1 - q$, where the span is $a - b = 1$. We can explicitly derive the variance as follows:

$$\begin{aligned}
\text{Var}(Y) &= \mathbb{E}[Y^2] - (\mathbb{E}[Y])^2 \\
&= a^2q + b^2(1 - q) - (aq + b(1 - q))^2 \\
&= a^2q + b^2(1 - q) - ((1 + b)q + b(1 - q))^2 \quad (\text{using } a = 1 + b) \\
&= a^2q + b^2 - b^2q - (q + bq + b - bq)^2 \\
&= (1 + b)^2q + b^2 - b^2q - (q + b)^2 \\
&= (1 + 2b + b^2)q + b^2 - b^2q - (q^2 + 2bq + b^2) \\
&= q + 2bq + b^2q + b^2 - b^2q - q^2 - 2bq - b^2 \\
&= q - q^2 \\
&= q(1 - q)
\end{aligned}$$

The function $f(q) = q(1 - q)$ is a parabola opening downward, maximized at $q = 1/2$, giving a maximum value of $1/4$. Since Z_i is a random variable of this form (supported on two points with distance 1), its variance $\text{Var}(Z_i)$ corresponds to $q_t(1 - q_t)$ for some probability q_t . Therefore:

$$\text{Var}(Z_i) \leq \frac{1}{4}$$

Consequently, $G''(t) = \text{Var}(Z_i) - 1/4 \leq 0$ for all t .

Conclusion: Since $G(0) = 0$, $G'(0) = 0$, and $G''(t) \leq 0$, the function $G(t)$ is concave and maximized at $t = 0$. Thus $G(t) \leq 0$ for all t , which implies:

$$\log \mathbb{E}[e^{tY_i}] \leq \frac{t^2}{8} \implies \mathbb{E}[e^{tY_i}] \leq e^{t^2/8}$$

□

1.3.2 Application: Binomial Distribution

Let $X \sim \text{Binomial}(n, 1/2)$. We apply the additive Chernoff bound derived above:

$$\mathbb{P}[X - np \geq \delta] \leq e^{-2\delta^2/n}$$

We want to bound $\mathbb{P}[X > 3n/4]$. First, we determine the deviation δ :

$$X > \frac{3n}{4} \iff X - \frac{n}{2} > \frac{3n}{4} - \frac{n}{2} = \frac{n}{4}$$

Thus, we set $\delta = n/4$. Substituting this into the bound:

$$\begin{aligned}
\mathbb{P}\left[X > \frac{3n}{4}\right] &\leq \exp\left(-\frac{2(n/4)^2}{n}\right) \\
&= \exp\left(-\frac{2(n^2/16)}{n}\right) \\
&= e^{-n/8}
\end{aligned}$$

Comparison: The Chernoff bound provides an *exponentially* decaying bound ($e^{-n/8}$), which is significantly stronger than the polynomial decay of Chebyshev ($4/n$) or the constant bound of Markov ($2/3$).

1.4 Normal Approximation Bound

Let $X \sim \text{Binomial}(n, 1/2)$. By the Central Limit Theorem (CLT), the standardized sum $\frac{X-np}{\sqrt{np(1-p)}}$ converges in distribution to $N(0, 1)$ as $n \rightarrow \infty$. We use this to approximate the tail probability. Here Φ and $\bar{\Phi}$ denote, respectively, the CDF and tail probability of the standard normal:

$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-u^2/2} du, \quad \bar{\Phi}(x) = 1 - \Phi(x).$$

$$\begin{aligned} \mathbb{P}\left[X \geq \frac{3n}{4}\right] &= \mathbb{P}\left[\frac{X - n/2}{\sqrt{n/4}} \geq \frac{3n/4 - n/2}{\sqrt{n/4}}\right] \\ &\approx \bar{\Phi}\left(\frac{\sqrt{n}}{2}\right) \quad (\text{by CLT, convergence in distribution}) \\ &= 1 - \Phi\left(\frac{\sqrt{n}}{2}\right) \\ &\leq \frac{\phi(\sqrt{n}/2)}{\sqrt{n}/2} \quad (\text{by Mills' inequality}) \\ &= \frac{2}{\sqrt{n}} \cdot \frac{1}{\sqrt{2\pi}} e^{-n/8} \end{aligned}$$

Normal Approximation Bound [Mor: §18.6]

$$\mathbb{P}\left[X \geq \frac{3n}{4}\right] \lesssim \frac{2}{\sqrt{2\pi n}} e^{-n/8}$$

Remark 8 (Mills' Inequality [Mor: §18.6]). Mills' inequality states that $\bar{\Phi}(x) \leq \frac{\phi(x)}{x}$, where $\phi(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$ is the standard normal PDF.

Remark 9 (Applicability and strength). This bound is *not* exact; it is an asymptotic approximation valid for large n . When applicable, it is the **tightest** of all four bounds: it decays as $\frac{C}{\sqrt{n}} e^{-n/8}$, matching the $e^{-n/8}$ exponential rate of the Chernoff bound but with an additional $1/\sqrt{n}$ factor that brings it closer to the true tail probability. The Chernoff bound ($e^{-n/8}$) is an upper bound; the normal approximation reveals that the true decay rate is $\Theta\left(\frac{1}{\sqrt{n}} e^{-n/8}\right)$, confirming that Chernoff is tight up to a polynomial.

1.5 Comparison with Bounds

The **Chernoff bound** provides a **tight** upper bound that decays exponentially with n , closely tracking the exact tail probability. In contrast, the **Chebyshev bound** is significantly **looser**, offering a much weaker guarantee. The **Markov bound** is so ineffective in this context that it yields values far exceeding the plotted range, effectively rendering it invisible on the graph.

To see precisely *how* tight the Chernoff bound is, compare the exponents derived in the previous two subsections:

$$\text{Normal approx.: } \mathbb{P}\left[X \geq \frac{3n}{4}\right] \lesssim \frac{2}{\sqrt{2\pi n}} e^{-n/8} \quad \text{vs.} \quad \text{Chernoff: } \mathbb{P}\left[X \geq \frac{3n}{4}\right] \leq e^{-n/8}.$$

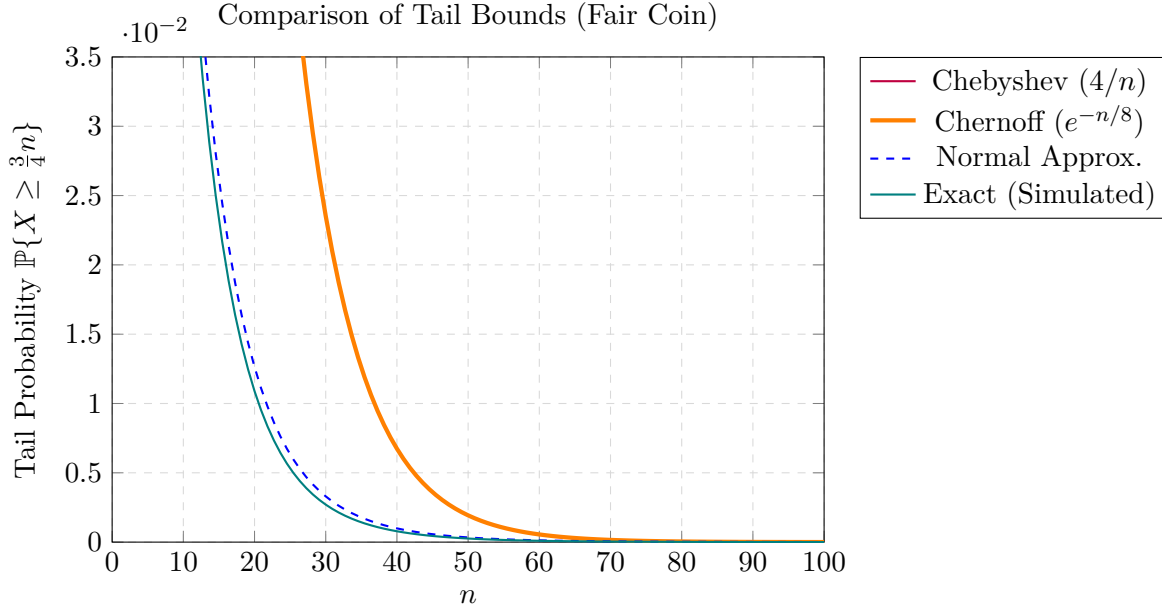


Figure 1: Comparison of Chebyshev, Chernoff, and Normal approximation for Binomial tails. After [Mor: Fig. in §18.6].

Both bounds share the *same* exponential decay rate $e^{-n/8}$. The Chernoff bound is therefore tight up to a polynomial prefactor of $O(\sqrt{n})$, confirming that it correctly captures the true rate of exponential decay.

2 The Triangle of Polling and Differential Privacy

Textbook Roadmap

For a parallel treatment in the course textbooks, see:

- *Triangle of polling (sample complexity via Chebyshev, Chernoff, CLT+Mills')*: [Mor: Ch. 19, §§19.2–19.3]; [MU: Ch. 4, §4.2.3].
- *Almost-sure convergence & First Borel–Cantelli*: not given a dedicated section in any of the three textbooks; the Strong LLN is implicit in [Mor: Ch. 25–26]. Standard measure-theoretic probability background.
- *Differential Privacy (ϵ -DP, randomized response, cost of privacy)*: not in the course textbooks. DP postdates all three books (Dwork, 2006); the supporting tools (unbiased estimation, Chernoff bounds) are from [Mor: Ch. 18–19] and [MU: Ch. 4].

2.1 Triangle of Polling

This section explores the relationships between three fundamental parameters in polling:

- n : The number of samples (sample size).
- ϵ : The precision (margin of error).
- δ : The confidence parameter (where $1 - \delta$ is the confidence level).

Let $X_1, X_2, \dots, X_n$ be i.i.d. Bernoulli(p) random variables, and let $S_n = \sum_{i=1}^n X_i$ denote their sum, so that $S_n \sim \text{Binomial}(n, p)$. We wish to estimate the true proportion p using the empirical mean $\frac{S_n}{n}$.

2.1.1 Main Objective

We seek to bound the probability that our estimate deviates from the true parameter by more than ε .

$$\text{Success condition: } \mathbb{P}\left(\left|\frac{S_n}{n} - p\right| < \varepsilon\right) \geq 1 - \delta$$

$$\text{Failure condition: } \mathbb{P}\left(\left|\frac{S_n}{n} - p\right| \geq \varepsilon\right) \leq \delta$$

2.1.2 Approach 1: Chebyshev's Inequality

Using Chebyshev's inequality, we bound the deviation using the variance.

$$\mathbb{P}\left(\left|\frac{S_n}{n} - p\right| \geq \varepsilon\right) \leq \frac{\text{Var}(S_n/n)}{\varepsilon^2}$$

Recall that $\text{Var}(S_n/n) = \frac{1}{n^2} \text{Var}(S_n) = \frac{np(1-p)}{n^2} = \frac{p(1-p)}{n}$. To eliminate the unknown parameter p , we use the fact that $p(1-p)$ is maximized at $p = 0.5$, giving $p(1-p) \leq \frac{1}{4}$.

$$\frac{p(1-p)}{n\varepsilon^2} \leq \frac{1}{4n\varepsilon^2} = \delta$$

Solving for n :

Sample Complexity (Chebyshev) [Mor: §19.2.2]

$$n \geq \frac{1}{4\varepsilon^2\delta}$$

Example 1: If $\delta = 0.05$ and $\varepsilon = 0.01$:

$$n = \frac{1}{4(0.01)^2(0.05)} = 50,000$$

2.1.3 Approach 2: Chernoff Bound

The Chernoff bound provides a tighter exponential decay for sums of independent random variables.

$$\mathbb{P}(|S_n - np| \geq n\varepsilon) \leq 2e^{-2n\varepsilon^2}$$

Setting this bound equal to δ :

$$2e^{-2n\varepsilon^2} = \delta \implies e^{-2n\varepsilon^2} = \frac{\delta}{2}$$

Taking the natural log of both sides:

Sample Complexity (Chernoff) [Mor: §19.2.1] [MU: §4.2.3]

$$n = \frac{\ln(2/\delta)}{2\varepsilon^2}$$

Example 2: If $\delta = 0.05$ and $\varepsilon = 0.01$:

$$n = \frac{\ln(40)}{2(0.0001)} \approx 18,444$$

This yields a significantly smaller required sample size compared to Chebyshev.

2.1.4 Approach 3: CLT and Mills' Inequality

We can approximate the Binomial distribution with a Normal distribution $N(np, np(1-p))$ via the Central Limit Theorem (CLT).

$$Z = \frac{S_n - np}{\sqrt{np(1-p)}} \sim N(0, 1)$$

The probability of failure is:

$$\mathbb{P}\left(\left|\frac{S_n - np}{\sqrt{np(1-p)}}\right| \geq \frac{n\varepsilon}{\sqrt{np(1-p)}}\right)$$

Using the worst-case variance where $p(1-p) = 1/4$, the threshold becomes $2\varepsilon\sqrt{n}$. Thus:

$$\mathbb{P}(|Z| \geq 2\varepsilon\sqrt{n}) \approx 2(1 - \Phi(2\varepsilon\sqrt{n}))$$

where Φ is the CDF of the standard normal distribution.

Applying Mills' Inequality: For $x > 0$, Mills' inequality states that $1 - \Phi(x) < \frac{1}{x\sqrt{2\pi}}e^{-x^2/2}$. Substituting $x = 2\varepsilon\sqrt{n}$:

$$2(1 - \Phi(x)) < \frac{2}{2\varepsilon\sqrt{n}\sqrt{2\pi}}e^{-(2\varepsilon\sqrt{n})^2/2} = \frac{e^{-2n\varepsilon^2}}{\varepsilon\sqrt{2\pi n}}$$

Setting this approximately equal to δ allows us to solve for n^* (often done numerically or iteratively). A simplified closed-form approximation often derived from this perspective is:

Sample Complexity (CLT + Mills') [Mor: §19.3]

$$n^* \approx \frac{-\ln(\delta\sqrt{2\pi})}{2\varepsilon^2}$$

Here we have absorbed the $\ln(\varepsilon\sqrt{n})$ contribution into a lower-order correction; the dominant exponential term is $-2n\varepsilon^2$, and the formula above is exact only up to that approximation. **Example 3:** If $\delta = 0.05$ and $\varepsilon = 0.01$, the calculated n^* is approximately 10,400.

- Note: We require large enough n (e.g., $n > 10,000$) and $\varepsilon\sqrt{n} > 1$ for the approximation error of Mills' inequality to be negligible.

2.2 Almost sure convergence in terms of Sets

In this section, we explain almost sure convergence using set notation. This helps us calculate the probability of convergence and allows us to use the First Borel-Cantelli Lemma.

2.2.1 Convergence in Probability vs. Almost Sure Convergence

First, we look at the difference between the two types of convergence.

1. Convergence in Probability: The sequence of sample means $\frac{S_n}{n}$ converges to p in probability if the probability of a large error goes to zero as $n \rightarrow \infty$:

$$\lim_{n \rightarrow \infty} \mathbb{P} \left(\left| \frac{S_n}{n} - p \right| > \varepsilon \right) = 0. \quad (1)$$

Note that this is a statement about a *sequence of real numbers* (i.e., the probabilities $\mathbb{P}(|\frac{S_n}{n} - p| > \varepsilon)$) converging to zero. The limit is *outside* the probability.

2. Almost Sure Convergence: The sequence converges almost surely (a.s.) if the probability of the event “the sequence converges to p ” is exactly 1:

$$\mathbb{P} \left(\left\{ \omega : \lim_{n \rightarrow \infty} \frac{S_n}{n}(\omega) = p \right\} \right) = 1. \quad (2)$$

In contrast to convergence in probability, this is a statement about the *sample paths* of the stochastic process: for (almost) every outcome ω , the entire trajectory $\frac{S_1}{1}(\omega), \frac{S_2}{2}(\omega), \dots$ eventually settles at p . The limit is *inside* the probability. Using the limit definition, the condition inside is:

$$\forall \varepsilon > 0, \exists N_\varepsilon \text{ s.t. } \forall n > N_\varepsilon, \left| \frac{S_n}{n} - p \right| < \varepsilon.$$

Remark 10. Almost sure convergence is strictly stronger than convergence in probability: $\frac{S_n}{n} \xrightarrow{a.s.} p$ implies $\frac{S_n}{n} \xrightarrow{P} p$, but the converse does not hold in general.

2.2.2 Set-Theoretic Representation

To calculate probability, we rewrite the logic symbols using set symbols. We replace ε with $1/k$ for integer $k \geq 1$: this is valid because for any $\varepsilon > 0$, we can find k large enough so that $1/k \leq \varepsilon$, and conversely every $1/k$ is a valid choice of ε . The event $\frac{S_n}{n} \rightarrow p$ is then the same as:

$$\left\{ \frac{S_n}{n} \rightarrow p \right\} \iff \bigcap_{k=1}^{\infty} \bigcup_{N=1}^{\infty} \bigcap_{n=N}^{\infty} \left\{ \left| \frac{S_n}{n} - p \right| < \frac{1}{k} \right\}. \quad (3)$$

- $\bigcap_{n=N}^{\infty}$: The error stays small forever after time N .
- $\bigcup_{N=1}^{\infty}$: There is a starting time N when this happens (“Eventually”).
- $\bigcap_{k=1}^{\infty}$: This must be true for every error size $1/k$.

2.2.3 Intersection of Probability 1 Events

We show that if many events each have probability 1, their intersection also has probability 1.

Lemma 11. *If $\mathbb{P}(E_i) = 1$ for all i , then $\mathbb{P}(\bigcap_{i=1}^{\infty} E_i) = 1$.*

Proof. We use properties of complements and sums. Since $\mathbb{P}(E_i) = 1$, the probability of the complement is $\mathbb{P}(E_i^c) = 0$.

$$1 - \mathbb{P}\left(\bigcap_{i=1}^{\infty} E_i\right) = \mathbb{P}\left(\bigcup_{i=1}^{\infty} E_i^c\right) \leq \sum_{i=1}^{\infty} \mathbb{P}(E_i^c) = 0. \quad (4)$$

Therefore, the intersection has probability 1. $\square$

2.2.4 Infinitely Often (i.o.)

We define the opposite concept: when the error does not stop.

Definition 12 (Infinitely Often). An event sequence A_n happens “infinitely often” if:

$$\{A_n \text{ i.o.}\} = \bigcap_{N=1}^{\infty} \bigcup_{n=N}^{\infty} A_n. \quad (5)$$

If we let $A_n = \{|\frac{S_n}{n} - p| \geq \varepsilon\}$ be the “bad” event (large error), then convergence fails if A_n happens infinitely often.

2.2.5 The First Borel-Cantelli Lemma

This lemma gives us a way to prove that bad events eventually stop.

Theorem 13 (First Borel–Cantelli Lemma [standard measure-theoretic background; not in course textbooks]). *If the sum of probabilities is finite ($\sum \mathbb{P}(A_n) < \infty$), then the probability of A_n happening infinitely often is 0.*

Proof.

$$\mathbb{P}(A_n \text{ i.o.}) \leq \mathbb{P}\left(\bigcup_{k=N}^{\infty} A_k\right) \leq \sum_{k=N}^{\infty} \mathbb{P}(A_k). \quad (6)$$

Because the total sum is finite, the tail sum (from N to ∞) goes to 0 as $N \rightarrow \infty$. $\square$

2.2.6 Application: Convergence of Sample Means

We apply the lemma to prove almost sure convergence. We need to show that the sum of error probabilities is finite.

1. Using Chebyshev’s Inequality (Fails):

$$\sum \mathbb{P}\left(\left|\frac{S_n}{n} - p\right| > \varepsilon\right) \leq \sum \frac{p(1-p)}{n\varepsilon^2} = \infty.$$

The sum diverges (goes to infinity), so we cannot use the lemma here.

2. Using Exponential Bounds (Succeeds):

$$\mathbb{P}\left(\left|\frac{S_n}{n} - p\right| > \varepsilon\right) \leq 2e^{-2n\varepsilon^2}. \quad (7)$$

Now, the sum is a geometric series, which is finite:

$$\sum_{n=1}^{\infty} 2e^{-2n\varepsilon^2} < \infty. \quad (8)$$

Since the sum is finite, the First Borel-Cantelli Lemma tells us the error happens only finitely often. Thus, $\frac{S_n}{n}$ converges to p almost surely.

An alternative proof of this almost-sure convergence using only Chebyshev's inequality, via a subsequence argument, is the focus of the following exercise.

Problem 1: Alternative Proof using Chebyshev's Inequality

Adapted from [Mor: Ex. 18.13] / [MU: Ex. 3.26] (WLLN via Chebyshev); the homework strengthens this to almost-sure convergence via the subsequence $n_k = k^2$.

In the section above, we saw that applying Chebyshev's inequality directly to all n failed. However, it is possible to prove almost sure convergence using Chebyshev's inequality if we use a specific subsequence.

Problem Statement: Prove that $\frac{S_n}{n} \rightarrow p$ almost surely using Chebyshev's inequality.

Hint:

1. Instead of checking every n , consider only the subsequence $n_k = k^2$. Apply Chebyshev's inequality to this subsequence and use the fact that $\sum \frac{1}{k^2} < \infty$.
2. Use the "Sandwich Theorem" (Squeeze Theorem). For any general n , find a k such that $k^2 \leq n < (k+1)^2$.
3. Use the property that S_n is non-decreasing to bound the term $\frac{S_n}{n}$:

$$S_{k^2} \leq S_n \leq S_{(k+1)^2}.$$

2.3 Differential Privacy

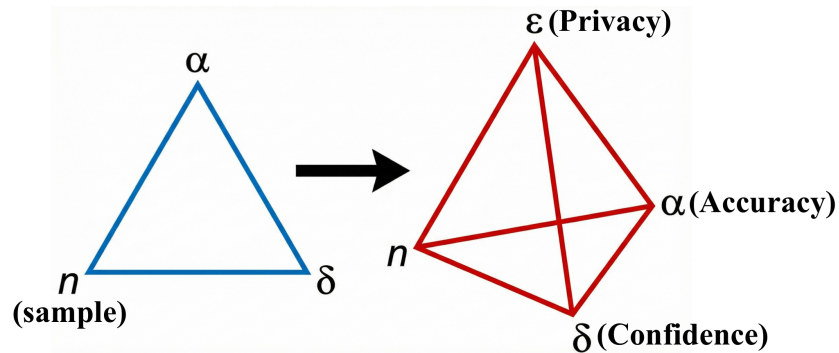


Figure 2: Privacy-Aware Triangular Pyramid Model.

In the absence of privacy considerations, the sample size n is determined solely by the desired **accuracy** (α) and **confidence** (δ). By introducing **privacy** (ϵ) into this trade-off, we investigate how the sample requirements scale. Specifically, we focus on the surplus of data needed to ensure

privacy without compromising the established precision of our estimates.

2.3.1 Mathematical Definition

Definition 14 (ϵ -Differential Privacy [Dwork, 2006; not in course textbooks]). A randomized algorithm $M : \mathcal{S} \rightarrow \mathcal{R}$ is ϵ -differentially private if for all pairs of input datasets S and S' differing in the contribution of a single individual, and for all subsets $r \subseteq \mathcal{R}$, the following inequality holds:

$$\mathbb{P}(M(S) \in r) \leq e^\epsilon \mathbb{P}(M(S') \in r) \quad (9)$$

Here, $\epsilon \geq 0$ is the privacy budget.

Intuition: The definition requires that the output distribution of the mechanism changes very little when any single individual's data is added or removed. In other words, no individual's contribution can significantly influence the final result, so an adversary observing the output learns almost nothing about any specific person's data.

- If ϵ is very large, the privacy guarantee is weak (the output reveals nearly exact information about the input).
- For meaningful privacy, ϵ should be small but positive (e.g., $0.1 \leq \epsilon \leq 1$).

Remark 15. The notion of “neighboring datasets” (S and S' differing in one individual) is intentionally flexible. It can mean either *substitution* (replacing one entry, e.g., $S = \{0, 1, 0\}$ and $S' = \{0, 1, 1\}$) or *addition/removal* (adding or removing one entry, e.g., $S = \{0, 1, 0\}$ and $S' = \{0, 1\}$). Both conventions appear in the literature. See Dwork et al. [5] for the foundational discussion underlying these conventions.

2.3.2 Example 1: A Naive Approach (Does not satisfy DP)

Let X_i be the real value (the sensitive attribute) and Y_i be the randomized private response.

- $X_i = 1$ with probability p .
- $X_i = 0$ with probability $1 - p$.

Consider a mechanism where we act as follows:

1. If $X_i = 1$, we always output $Y_i = 1$.
2. If $X_i = 0$, we output 0 or 1 uniformly (probability 0.5).

This can be written as conditional probabilities:

$$\begin{aligned} \mathbb{P}(Y_i = 1 | X_i = 1) &= 1 \\ \mathbb{P}(Y_i = 0 | X_i = 1) &= 0 \\ \mathbb{P}(Y_i = 1 | X_i = 0) &= 0.5 \\ \mathbb{P}(Y_i = 0 | X_i = 0) &= 0.5 \end{aligned}$$

Let's check if this fits the definition of differential privacy by comparing the ratio of probabilities:

$$\frac{\mathbb{P}(Y_i = 1 | X_i = 1)}{\mathbb{P}(Y_i = 1 | X_i = 0)} = \frac{1}{0.5} = 2 \quad (\leq e^{\ln 2}) \quad (10)$$

However, looking at the case for output 0:

$$\frac{\mathbb{P}(Y_i = 0|X_i = 0)}{\mathbb{P}(Y_i = 0|X_i = 1)} = \frac{0.5}{0} = \infty \quad (11)$$

Since the ratio is infinite, we cannot find any finite satisfying value for ε . Thus, this mechanism **does not** satisfy differential privacy. (This happens because the supports of the distributions do not match; an output of 0 perfectly reveals that $X_i \neq 1$).

2.3.3 Example 2: Randomized Response (Satisfies DP)

To fix the issue above, we ensure that every output is possible for every input. We define a specific “flip probability” designed to satisfy ε .

Let the flip probability be $\frac{1}{1+e^\varepsilon}$. We define the mechanism Y_i as follows:

- Output the **true** value (X_i) with probability $\frac{e^\varepsilon}{1+e^\varepsilon}$.
- Output the **false** value ($1 - X_i$) with probability $\frac{1}{1+e^\varepsilon}$.

The marginal distribution of Y_i (using Total Probability) is:

$$Y_i = \begin{cases} 0 & \text{with prob } \frac{e^\varepsilon}{1+e^\varepsilon}(1-p) + \frac{1}{1+e^\varepsilon}p \\ 1 & \text{with prob } \frac{1}{1+e^\varepsilon}(1-p) + \frac{e^\varepsilon}{1+e^\varepsilon}p \end{cases} \quad (12)$$

Now, let's verify the privacy definition using the conditional probabilities. The ratio for outputting 1:

$$\frac{\mathbb{P}(Y_i = 1|X_i = 1)}{\mathbb{P}(Y_i = 1|X_i = 0)} = \frac{\frac{e^\varepsilon}{1+e^\varepsilon}}{\frac{1}{1+e^\varepsilon}} = e^\varepsilon \quad (13)$$

The ratio for outputting 0:

$$\frac{\mathbb{P}(Y_i = 0|X_i = 1)}{\mathbb{P}(Y_i = 0|X_i = 0)} = \frac{\frac{1}{1+e^\varepsilon}}{\frac{e^\varepsilon}{1+e^\varepsilon}} = \frac{1}{e^\varepsilon} = e^{-\varepsilon} \quad (14)$$

Since $e^\varepsilon \leq e^\varepsilon$ and $e^{-\varepsilon} \leq e^\varepsilon$ (for $\varepsilon > 0$), the ratio is always bounded by e^ε . Thus, this mechanism **satisfies** ε -differential privacy.

2.3.4 Cost of Privacy (Sample Complexity)

To achieve an equal level of accuracy as a non-private algorithm, how many more samples do we need?

Let Y_i be the private response and $S_n = \sum_{i=1}^n Y_i$. The expected value of the raw average is:

$$\mathbb{E}\left[\frac{S_n}{n}\right] = p \left(\frac{e^\varepsilon - 1}{e^\varepsilon + 1} \right) + \frac{1}{e^\varepsilon + 1} \quad (15)$$

Bias: This estimator is *biased* because $\mathbb{E}[S_n/n] \neq p$. It converges to a value distorted by the noise parameter ε .

To correct this, we construct an *unbiased* estimator T_n :

$$T_n = \left(\frac{S_n}{n} - \frac{1}{1+e^\varepsilon} \right) \frac{e^\varepsilon + 1}{e^\varepsilon - 1} \quad (16)$$

Unbiased: This estimator is *unbiased* because calculating its expectation cancels out the noise terms, resulting in $\mathbb{E}[T_n] = p$.

Convergence By the Law of Large Numbers, T_n converges to p almost surely:

$$T_n \xrightarrow{a.s.} p \quad (17)$$

(Note: “a.s.” stands for **almost sure** convergence, meaning the probability that the estimator converges to the true parameter p as $n \rightarrow \infty$ is 1.)

Deriving Sample Complexity (n) We want to find n such that the error is small (within α) with high probability. Using the tail bound:

$$\mathbb{P}(|T_n - p| > \alpha) \quad (18)$$

Substituting the definition of T_n :

$$\mathbb{P}\left(\left|\left(\frac{S_n}{n} - \frac{1}{1+e^\varepsilon}\right) \frac{e^\varepsilon + 1}{e^\varepsilon - 1} - p\right| > \alpha\right) = \mathbb{P}\left(\left|\frac{S_n}{n} - \mathbb{E}\left[\frac{S_n}{n}\right]\right| > \alpha \frac{e^\varepsilon - 1}{e^\varepsilon + 1}\right) \quad (19)$$

By applying Chernoff bound $\mathbb{P}(|\bar{X} - E| > t) \leq 2e^{-2nt^2}$, where our threshold is $\alpha \frac{e^\varepsilon - 1}{e^\varepsilon + 1}$:

$$\mathbb{P}(|T_n - p| > \alpha) \leq 2 \exp\left(-2n \left(\alpha \frac{e^\varepsilon - 1}{e^\varepsilon + 1}\right)^2\right) \quad (20)$$

Setting this probability to be at most β (significance level) and solving for n :

$$\begin{aligned} 2 \exp\left(-2n\alpha^2 \left(\frac{e^\varepsilon - 1}{e^\varepsilon + 1}\right)^2\right) &= \beta \\ -2n\alpha^2 \left(\frac{e^\varepsilon - 1}{e^\varepsilon + 1}\right)^2 &= \ln(\beta/2) \\ n &= \frac{-\ln(\beta/2)}{2\alpha^2 \left(\frac{e^\varepsilon - 1}{e^\varepsilon + 1}\right)^2} \end{aligned}$$

Sample Complexity (ε -LDP) [instructor notes; supporting Chernoff bound from MU Ch. 4]

$$n_{\varepsilon\text{-LDP}} = \frac{-\ln(\beta/2)}{2\alpha^2 \left(\frac{e^\varepsilon - 1}{e^\varepsilon + 1}\right)^2}$$

Interpretation:

- As $\varepsilon \rightarrow 0$ (stronger privacy), the term $\left(\frac{e^\varepsilon - 1}{e^\varepsilon + 1}\right)$ approaches 0. Since it is in the denominator, n increases significantly.
- As ε gets larger (weaker privacy), the term approaches 1, and the effect on n becomes minimal.

3 Local and Central Differential Privacy

Textbook Roadmap

The material of this section (ε -LDP, ε -CDP, the Laplace mechanism, and the LDP-vs-CDP sample-complexity comparison) is *not* covered in any of the three course textbooks; it follows the instructor's notes and the original differential-privacy literature (Dwork, McSherry, Nissim & Smith, 2006). The supporting probabilistic machinery is, however, fully developed in the textbooks:

- *Chernoff bound* for the sampling-error term: [Mor: Ch. 18, §§18.3–18.5]; [MU: Ch. 4, §§4.1–4.2].
- *Laplace tail bound* $\mathbb{P}(|Z| > t) = e^{-t/b}$: a one-line consequence of the Laplace pdf; not stated as a named theorem in the course books.

With the ε -DP framework and the randomized response mechanism in hand, the natural next question is quantitative: how much accuracy do we sacrifice when we insist on privacy? We now turn to the sample complexity of private estimation.

3.1 Cost of Privacy(ε -LDP)

We now analyze the *sample complexity*: how much data (n) do we need to estimate the true parameter p within an error margin α with high probability?

3.1.1 Baseline: Without Privacy

First, consider the non-private case (using clean data X_i). We want to find n such that the error is small (within α) with probability at least $1 - \beta$. From the Chernoff bound, we know:

$$\mathbb{P}(|\bar{X} - p| > \alpha) \leq 2e^{-2n\alpha^2} \quad (21)$$

Setting the right-hand side to β and solving for n , we obtain the standard sample complexity:

$$n_{\text{clean}} = \frac{-\ln(\beta/2)}{2\alpha^2} \quad (22)$$

3.1.2 With Privacy (ε -LDP)

Now consider the private case where we only observe Y_i . Since Y_i is a noisy version of X_i , simply taking the mean of Y would result in a biased estimate. We must construct an unbiased estimator T_n .

Let $S_n = \sum_{i=1}^n Y_i$. The expected value of a single observation is $\mathbb{E}[Y_i] = p(\frac{e^\varepsilon}{1+e^\varepsilon}) + (1-p)(\frac{1}{1+e^\varepsilon})$. By inverting this relation (solving for p in terms of $\mathbb{E}[Y_i]$), we define our unbiased estimator T_n :

$$T_n = \left(\frac{S_n}{n} - \frac{1}{1+e^\varepsilon} \right) \frac{e^\varepsilon + 1}{e^\varepsilon - 1} \quad (23)$$

We want to bound the error of this estimator: $\mathbb{P}(|T_n - p| > \alpha)$. Substituting the definition of T_n :

$$\mathbb{P}(|T_n - p| > \alpha) = \mathbb{P}\left(\left|\left(\frac{S_n}{n} - \frac{1}{1+e^\varepsilon}\right) \frac{e^\varepsilon + 1}{e^\varepsilon - 1} - p\right| > \alpha\right) \quad (24)$$

$$= \mathbb{P}\left(\left|\frac{S_n}{n} - \left(\frac{p(e^\varepsilon - 1)}{e^\varepsilon + 1} + \frac{1}{1+e^\varepsilon}\right)\right| > \alpha \frac{e^\varepsilon - 1}{e^\varepsilon + 1}\right) \quad (25)$$

$$= \mathbb{P}\left(\left|\frac{S_n}{n} - \mathbb{E}\left[\frac{S_n}{n}\right]\right| > \alpha \frac{e^\varepsilon - 1}{e^\varepsilon + 1}\right) \quad (26)$$

Notice that the inequality is now in the form $\mathbb{P}(|\bar{Y} - \mathbb{E}[\bar{Y}]| > t)$, where the threshold is scaled down to $t = \alpha \frac{e^\varepsilon - 1}{e^\varepsilon + 1}$. Applying the Chernoff bound:

$$\mathbb{P}(|T_n - p| > \alpha) \leq 2 \exp \left(-2n \left(\alpha \frac{e^\varepsilon - 1}{e^\varepsilon + 1} \right)^2 \right) \quad (27)$$

Setting this probability to be at most β and solving for n :

$$\begin{aligned} 2 \exp \left(-2n\alpha^2 \left(\frac{e^\varepsilon - 1}{e^\varepsilon + 1} \right)^2 \right) &= \beta \\ -2n\alpha^2 \left(\frac{e^\varepsilon - 1}{e^\varepsilon + 1} \right)^2 &= \ln(\beta/2) \\ n_{\varepsilon\text{-LDP}} &= \frac{-\ln(\beta/2)}{2\alpha^2 \left(\frac{e^\varepsilon - 1}{e^\varepsilon + 1} \right)^2} \end{aligned}$$

3.1.3 Effective Sample Size

Comparing n_{clean} and $n_{\varepsilon\text{-LDP}}$, we can define the *effective sample size* (n_{eff}). This represents how much “clean” information our noisy dataset contains:

$$n_{\text{eff}} = n_{\varepsilon\text{-LDP}} \cdot \left(\frac{e^\varepsilon - 1}{e^\varepsilon + 1} \right)^2 \quad (28)$$

This relationship highlights the cost of privacy. As $\varepsilon \rightarrow 0$ (high privacy), we can approximate the term using Taylor expansion ($e^\varepsilon \approx 1 + \varepsilon$):

$$\left(\frac{e^\varepsilon - 1}{e^\varepsilon + 1} \right)^2 \approx \left(\frac{(1 + \varepsilon) - 1}{(1 + \varepsilon) + 1} \right)^2 \approx \left(\frac{\varepsilon}{2} \right)^2 = \frac{\varepsilon^2}{4} \quad (29)$$

This implies that $n_{\varepsilon\text{-LDP}} \approx \frac{4}{\varepsilon^2} n_{\text{clean}}$. In other words, to maintain utility while satisfying strong privacy ($\varepsilon \rightarrow 0$), the required data size increases by a factor of $O(1/\varepsilon^2)$.

3.2 Local DP vs. Central DP

A fundamental distinction in differential privacy lies in *when* and *where* the noise is injected. In the previous section on Randomized Response(R.R.), we discussed **Local Differential Privacy (LDP)**. In the LDP setting, noise is added to each individual data point X_i by the user *before* it is sent to the central server. The aggregator never sees the true X_i , only the noisy Y_i . This requires no trust in the central server, but typically results in lower accuracy because the noise variance accumulates with the number of users.

In contrast, **Central Differential Privacy (CDP)** assumes a trusted curator who collects the raw data $S = \{X_1, \dots, X_n\}$. The curator computes the true aggregate function (e.g., the true mean S_n/n) and then adds noise to this *output* before releasing it to the public. In this setting, the noise is added to the query result (S_n/n) rather than the individual inputs. Because the noise is added only once to the final statistic, CDP generally achieves much higher accuracy (better utility) for the same privacy budget ε , provided the users trust the central aggregator.

3.3 ϵ -Central Differential Privacy

In the Central DP setting, we assume a trusted curator collects the dataset $D = \{X_1, \dots, X_n\} \in \{0, 1\}^n$. We wish to release a query result, such as the sample mean $f(D) = S_n/n$, with additive noise. The mechanism M is defined as:

$$M(D) = f(D) + Z \quad (30)$$

where Z is a random noise variable.

3.3.1 The Optimization Problem

Our goal is to choose a noise distribution Z that minimizes the error (variance) while satisfying the differential privacy constraint. Formally, we want to solve:

$$\min_{P_Z} \text{Var}(Z) \quad \text{s.t.} \quad \frac{p_M(t|D)}{p_M(t|D')} \leq e^\epsilon, \quad \forall t, \forall D \sim D' \quad (31)$$

Here, $p_M(t|D)$ represents the **probability density function (PDF)** of the mechanism outputting a specific value t given dataset D . The condition states that for any two neighboring datasets D and D' (differing by one person), the likelihood of seeing the same output t must be very close (bounded by e^ϵ).

Finding the optimal distribution for general queries is difficult. However, for real-valued queries like the mean, the standard solution is the **Laplace Mechanism**.

3.3.2 The Laplace Mechanism

To satisfy ϵ -DP, we draw the noise Z from a Laplace distribution, denoted as $Z \sim \text{Lap}(b)$. The probability density function is:

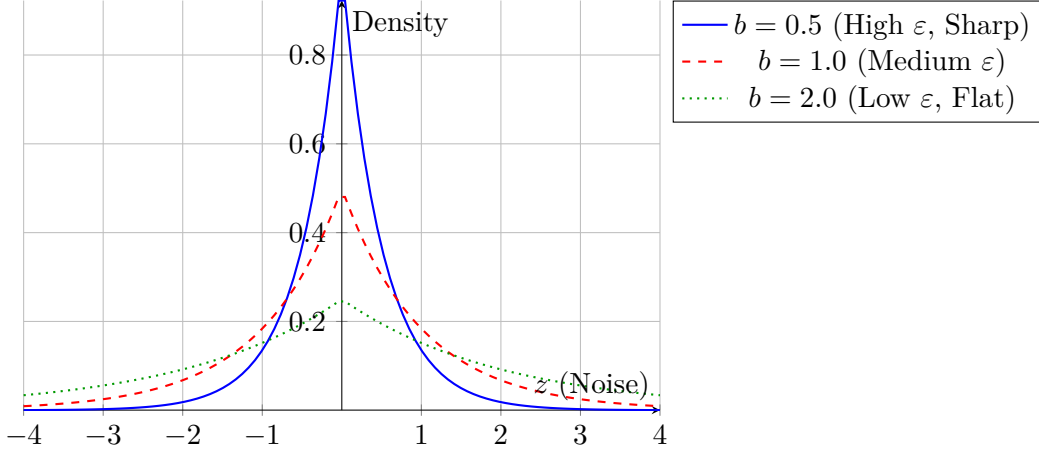
$$P_Z(z) = \frac{1}{2b} \exp\left(-\frac{|z|}{b}\right) \quad (32)$$

where b is the scale parameter.

Remark 16 (Why Laplace?). The choice of Laplace among all ϵ -DP noise distributions is not arbitrary. Theorem 6 of Koufogiannis et al. [6] shows that, among all input independent noise distributions g that satisfy ϵ -DP for a real valued identity query, the Laplace distribution achieves the minimum mean squared error, with $\mathbb{E}_{V \sim g} V^2 \geq \mathbb{E}_{V \sim \text{Lap}(1/\epsilon)} V^2 = 2/\epsilon^2$. The argument frames the variance minimization as an infinite dimensional linear program in which the privacy constraint becomes a pointwise Lipschitz bound on the log density, and weak duality singles out the Laplace density as the unique feasible solution that meets the dual bound. Papachristou and Rahimian [7] reuse this optimality result inside the proof of their Theorem 3.1 (Appendix A.1) to justify the Laplace choice for distributed private estimation, and the same reasoning carries over to the central DP setup considered here.

Visualizing the Noise and ϵ The shape of the Laplace distribution determines the privacy-utility trade-off.

- **Small b (High ϵ):** The distribution is sharp and peaked. The noise is likely to be small (high utility), but privacy is weaker.
- **Large b (Low ϵ):** The distribution is flat and wide. The noise can be large (low utility), but privacy is stronger because distributions for D and D' overlap more.



As shown in the graph:

- As $\varepsilon \rightarrow \infty$ (implying $b \rightarrow 0$), the curve becomes a spike at 0 (zero noise).
- As $\varepsilon \rightarrow 0$ (implying $b \rightarrow \infty$), the curve flattens out, meaning the output is very noisy and reveals almost nothing about the data.

3.3.3 Relation between b , n , and ε

We now formally derive the relationship between the noise scale b , the sample size n , and the privacy budget ε . Recall our mechanism:

$$M(D) = \frac{S_n}{n} + Z, \quad \text{where } Z \sim \text{Lap}(b) \quad (33)$$

Step 1: Sensitivity Analysis. First, we calculate how much the query output (the sample mean) changes between neighboring datasets D and D' . Since D and D' differ by exactly one individual's value (changing a 0 to 1 or vice versa), the sum S_n changes by at most 1. Consequently, the mean changes by at most $1/n$:

$$\left| \frac{S_n}{n} - \frac{S'_n}{n} \right| = \frac{|S_n - S'_n|}{n} \leq \frac{1}{n} \quad (34)$$

Step 2: Enforcing the DP Condition. To satisfy ε -DP, we require the ratio of the probability densities at any output t to be bounded by e^ε .

$$\frac{\mathbb{P}(M(D) = t)}{\mathbb{P}(M(D') = t)} \leq e^\varepsilon \quad (35)$$

Since $M(D) = \frac{S_n}{n} + Z$, observing an output of t implies that the noise was $Z = t - \frac{S_n}{n}$. We substitute the Laplace PDF, $P_Z(z) = \frac{1}{2b}e^{-|z|/b}$:

$$\frac{\frac{1}{2b} \exp\left(-\frac{|t - S_n/n|}{b}\right)}{\frac{1}{2b} \exp\left(-\frac{|t - S'_n/n|}{b}\right)} \leq e^\varepsilon \quad (36)$$

Canceling the $\frac{1}{2b}$ terms and using the exponent rule $\frac{e^A}{e^B} = e^{A-B}$:

$$\exp\left(\frac{|t - S'_n/n| - |t - S_n/n|}{b}\right) \leq e^\varepsilon \quad (37)$$

Step 3: Applying Triangle Inequality. To bound the exponent, we use the **Reverse Triangle Inequality**, which states that $|x| - |y| \leq |x - y|$. Let $x = t - S'_n/n$ and $y = t - S_n/n$. Then:

$$\begin{aligned} |t - S'_n/n| - |t - S_n/n| &\leq |(t - S'_n/n) - (t - S_n/n)| \\ &= \left| \frac{S_n}{n} - \frac{S'_n}{n} \right| \end{aligned}$$

From Step 1, we know this difference is bounded by the sensitivity $1/n$. Therefore:

$$\exp\left(\frac{1}{b} \left| \frac{S_n}{n} - \frac{S'_n}{n} \right| \right) \leq \exp\left(\frac{1}{bn}\right) \quad (38)$$

Conclusion. To satisfy the privacy condition, we set this upper bound equal to e^ε :

$$\exp\left(\frac{1}{bn}\right) \leq e^\varepsilon \implies \frac{1}{bn} \leq \varepsilon \implies b \geq \frac{1}{n\varepsilon} \quad (39)$$

Thus, to minimize the noise while guaranteeing privacy, we choose the smallest valid scale:

Laplace Mechanism for ε -CDP [Dwork et al., 2006; not in course textbooks]

$$b = \frac{1}{n\varepsilon} \quad (40)$$

We can now fully define our mechanism as:

$$M(D) = \frac{S_n}{n} + \text{Lap}\left(0, \frac{1}{n\varepsilon}\right) \quad (41)$$

Privacy-Utility Trade-off. The relationship $b = \frac{1}{n\varepsilon}$ explicitly links the noise magnitude to the dataset size and privacy budget. As illustrated in Figure 3, increasing the sample size n reduces the required noise scale b for a fixed ε , effectively concentrating the Laplace distribution around zero (improving utility). Similarly, a larger ε (weaker privacy) allows for a smaller b , resulting in less noise added to the output.

A direct verification that the randomized-response estimator satisfies ε -CDP, by aggregating its per-individual LDP guarantees, is the content of the following exercise.

Problem 2: Randomized Response and Central DP

Instructor-supplied.

Problem Statement: Consider the Randomized Response mechanism applied to a dataset of n individuals. For each individual i , let $X_i \in \{0, 1\}$ be the true response. The mechanism generates a noisy response Y_i according to the following probabilities:

$$Y_i = \begin{cases} X_i & \text{with probability } \frac{e^\varepsilon}{1 + e^\varepsilon} \\ 1 - X_i & \text{with probability } \frac{1}{1 + e^\varepsilon} \end{cases} \quad (42)$$

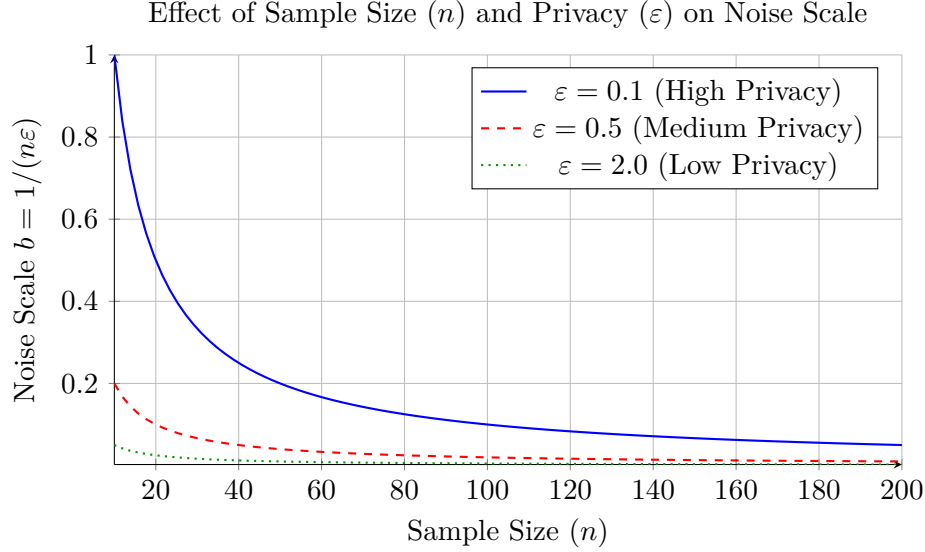


Figure 3: The trade-off between sample size n and noise scale b for different privacy budgets ε . As n increases, the noise required to maintain ε -CDP decreases ($1/n$ decay). Furthermore, for any fixed n , a larger ε results in a smaller noise scale b .

Let $S_n = \sum_{i=1}^n Y_i$ be the sum of the noisy responses. We define the unbiased estimator for the population mean, T_n , as in the main text:

$$T_n = \left(\frac{S_n}{n} - \frac{1}{1 + e^\varepsilon} \right) \frac{e^\varepsilon + 1}{e^\varepsilon - 1} \quad (43)$$

Prove that the mechanism $M(D) = T_n$ satisfies ε -Central Differential Privacy (ε -CDP).

- **Remark:** Note that this mechanism is originally designed for ε -Local Differential Privacy (ε -LDP). Since LDP is a stronger condition than CDP (because the noise is added locally before aggregation), any algorithm that satisfies ε -LDP automatically satisfies ε -CDP.

3.4 Sample Complexity for ε -CDP

We want to find the sample size n such that the error of our mechanism is bounded by α with high probability (at least $1 - \delta$).

$$\mathbb{P}(|M(D) - p| > \alpha) < \delta \quad (44)$$

Substituting the Central DP mechanism $M(D) = \frac{S_n}{n} + Z$, where $Z \sim \text{Lap}(1/n\varepsilon)$:

$$\mathbb{P}\left(\left|\frac{S_n}{n} + Z - p\right| > \alpha\right) < \delta \quad (45)$$

3.4.1 Error Decomposition

We cannot easily solve this directly, so we separate the error into two components: the *sampling error* (from S_n/n) and the *noise error* (from Z). Using the Triangle Inequality $|A + B| \leq |A| + |B|$,

we know that if $|S_n/n - p| + |Z| \leq \alpha$, then the total error is $\leq \alpha$. Conversely, for the error to exceed α , at least one of the components must be large. We split the threshold α using a parameter $x \in (0, 1)$:

$$\mathbb{P}\left(\left|\frac{S_n}{n} - p\right| + |Z| > \alpha\right) \leq \mathbb{P}\left(\left|\frac{S_n}{n} - p\right| > x\alpha\right) + \mathbb{P}(|Z| > (1-x)\alpha) \quad (46)$$

We set the sum of these probabilities to be less than δ . To do this, we split the failure budget δ using a parameter $y \in (0, 1)$:

$$1. \text{ Sampling Error: } \mathbb{P}\left(\left|\frac{S_n}{n} - p\right| > x\alpha\right) < y\delta$$

$$2. \text{ Noise Error: } \mathbb{P}(|Z| > (1-x)\alpha) < (1-y)\delta$$

We need to find n that satisfies both conditions simultaneously.

3.4.2 Solving for n

1. Sampling Condition (Chernoff Bound): From the Chernoff bound, $\mathbb{P}(|\bar{X} - p| > t) \leq 2e^{-nt^2}$. Setting $t = x\alpha$:

$$\begin{aligned} 2\exp(-2n(x\alpha)^2) &< y\delta \\ -2nx^2\alpha^2 &< \ln(y\delta/2) \\ n_1 &= \frac{-\ln(y\delta/2)}{2x^2\alpha^2} \end{aligned}$$

2. Noise Condition (Laplace Tail): The tail bound for $Z \sim \text{Lap}(b)$ is $\mathbb{P}(|Z| > t) = e^{-t/b}$. Here $b = \frac{1}{n\varepsilon}$ and $t = (1-x)\alpha$:

$$\begin{aligned} \exp\left(-\frac{(1-x)\alpha}{1/n\varepsilon}\right) &< (1-y)\delta \\ \exp(-n\varepsilon(1-x)\alpha) &< (1-y)\delta \\ -n\varepsilon(1-x)\alpha &< \ln((1-y)\delta) \\ n_2 &= \frac{-\ln((1-y)\delta)}{(1-x)\alpha\varepsilon} \end{aligned}$$

The required sample size is the maximum of the two: $n = \min_{x,y}(\max(n_1, n_2))$.

3.4.3 Optimization of Parameters

To find the optimal n , we balance the terms.

Optimizing y (Balancing Probability): We equate the probability terms relative to their pre-factors. A common heuristic used in class is:

$$\frac{y}{2} = 1 - y \implies y = \frac{2}{3} \quad (47)$$

Optimizing x (Balancing Convergence Rates): We equate the denominators of n_1 and n_2 to ensure both constraints scale similarly:

$$2\alpha^2x^2 \approx (1-x)\alpha\varepsilon \quad (48)$$

Rearranging gives a quadratic equation:

$$2\alpha x^2 + \varepsilon x - \varepsilon = 0 \quad (49)$$

Using the quadratic formula ($x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$):

$$x = \frac{-\varepsilon + \sqrt{\varepsilon^2 + 8\alpha\varepsilon}}{4\alpha} \quad (50)$$

Substituting these optimal values back into n_2 gives the approximate sample complexity for ε -CDP:

Sample Complexity (ε -CDP) [instructor notes; supporting Chernoff bound from MU Ch. 4 / Mor Ch. 18]

$$n_{\varepsilon\text{-CDP}} \approx \frac{-\ln(\delta/3)}{(1-x)\alpha\varepsilon}, \quad x = \frac{-\varepsilon + \sqrt{\varepsilon^2 + 8\alpha\varepsilon}}{4\alpha}$$

3.4.4 Comparison: CDP vs. LDP

There is a fundamental difference in how the sample complexity scales with ε (as $\varepsilon \rightarrow 0$):

- **CDP:** $n \propto \frac{1}{\varepsilon}$
- **LDP:** $n \propto \frac{1}{\varepsilon^2}$

This means LDP requires significantly more data to achieve the same accuracy as CDP when high privacy (low ε) is required.

Numerical Example: Consider the requirements $\delta = 0.05$, $\alpha = 0.01$, and $\varepsilon = 1$. The required sample sizes are approximately:

$$n_{\varepsilon\text{-LDP}} \approx 86,400$$

$$n_{\varepsilon\text{-CDP}} \approx 21,283$$

This illustrates that Central DP preserves utility much better than Local DP, effectively requiring ~ 4 times less data in this specific scenario.

Replacing Laplace noise with Gaussian noise leads to the more permissive (ε, δ) -DP relaxation; the corresponding sensitivity analysis and sample-complexity comparison are explored in the following exercise.

Problem 3: Gaussian Noise and Relaxed DP

Instructor-supplied.

Problem 1: Gaussian Noise and Pure DP. Consider a mechanism $M(D) = S_n/n + Z$, but instead of Laplace noise, we use Gaussian noise $Z \sim N(0, \sigma^2)$. Does this mechanism satisfy the original definition of ε -Differential Privacy (Pure DP)?

Problem 2: ε - θ Differential Privacy. Since Gaussian noise does not satisfy pure ε -DP, we relax the definition to allow for a small failure probability θ . A randomized algorithm M is (ε, θ) -differentially private if:

$$\mathbb{P}(M(D) \in r) \leq e^\varepsilon \mathbb{P}(M(D') \in r) + \theta \quad (51)$$

This is known as the “ θ -relaxation” of differential privacy. Verify that the Gaussian mechanism satisfies this relaxed (ε, θ) -DP definition for appropriate values of σ .

Problem 3: Sample Complexity Comparison. We previously calculated the sample size n_{Laplace} required for the Laplace mechanism. Now, consider the sample size n_{Gaussian} required when using the Gaussian mechanism with (ε, θ) -DP. Analyze the condition for θ . How large (or small) must θ be for the Gaussian mechanism to require fewer samples than the Laplace mechanism?

Further Reading

The two papers below take the ε -DP framework developed in this and the previous section in two complementary directions: one toward a new fundamental mechanism, the other toward stretching that mechanism to high-dimensional outputs.

- McSherry and Talwar [12], “Mechanism Design via Differential Privacy.” Introduces the *exponential mechanism*, a fundamental DP primitive complementing the Laplace mechanism studied above: instead of adding additive noise to a real-valued query, it samples an output b from a density $f(b) \propto \exp(\varepsilon \xi(b)/(2\Delta))$, where ξ is a utility function on the output space and Δ is its sensitivity. The paper also applies ε -DP as a solution concept in *mechanism design*, showing that DP mechanisms are approximately truthful and giving improved regret bounds for the unlimited-supply auction problem (digital goods, attribute auctions, and auctions with arbitrary structural constraints on the prices).
- Awan et al. [13], “Benefits and Pitfalls of the Exponential Mechanism with Applications to Hilbert Spaces and Functional PCA.” Extends the exponential mechanism to settings with infinite-dimensional outputs (functional data analysis, shape analysis, nonparametric statistics) by designing it with respect to a specified base measure, such as a Gaussian process. The paper derives a Central Limit Theorem for the exponential mechanism quite broadly, and develops an ε -DP mechanism for functional principal component analysis applicable in separable Hilbert spaces. A cautionary result: the magnitude of the privacy noise is shown to be asymptotically non-negligible relative to the statistical estimation error itself.
- Kairouz et al. [14], “The Composition Theorem for Differential Privacy.” Addresses how (ε, δ) -DP degrades when k mechanisms are composed adaptively, a question this section touches only briefly through linear composition. The paper proves a tight k -fold composition bound of $(\tilde{\varepsilon}, k\delta + \tilde{\delta})$ -DP with $\tilde{\varepsilon} = O(\varepsilon \sqrt{k \log(1/\tilde{\delta})} + k\varepsilon^2)$, improving on the naive $k\varepsilon$ guarantee and matching it with a converse construction. The proof reframes (ε, δ) -DP as a constraint on the privacy region of optimal hypothesis tests, an operational view that has since become the standard accounting tool for differentially private machine learning.
- Gupta et al. [15], “Differentially Private Combinatorial Optimization.” Applies the exponential mechanism introduced in the first entry above to classical combinatorial optimization problems where the input graph or set system encodes private information. The paper gives efficient ε -DP algorithms with bounds of $(2 + 16/\varepsilon) \cdot \text{OPT}$ for vertex cover, $\text{OPT} + O(\log n/\varepsilon)$ for min-cut,

and $6 \cdot \text{OPT} + O(k^2 \log^2 n / \varepsilon)$ for k -median, together with matching $\Omega(1/\varepsilon)$ approximation lower bounds. The min-cut algorithm bridges this section to the contraction-based Karger algorithm studied later in these notes: it samples a cut with probability proportional to $\exp(-\varepsilon \cdot |\text{cut}|)$, using Karger’s counting bound on the number of small cuts to control the partition function.

- Abadi et al. [17], “Deep Learning with Differential Privacy.” Carries the ε -DP framework into the modern machine-learning setting, training deep neural networks via stochastic gradient descent with formal (ε, δ) -DP guarantees on the released parameters. The DP-SGD algorithm clips each per-example gradient to a fixed ℓ_2 norm and adds calibrated Gaussian noise before each update. The privacy analysis is sharpened by a *moments accountant* that tracks the Rényi-divergence-style log-moment of the privacy loss and saves a $\sqrt{\log(1/\delta)}$ factor over the strong composition theorem of the previous entry. Experiments on MNIST and CIFAR-10 show that single-digit ε privacy budgets are compatible with modest accuracy loss, making the algorithm the de facto baseline for private deep learning.

4 Balls, Bins, and the Birthday Paradox

Textbook Roadmap

For a parallel treatment in the course textbooks, see:

- *Balls-and-bins model & Birthday Paradox*: [Mor: Ch. 19, §19.4]; [MU: Ch. 5, §5.1, §5.2.1].
- *Coupon Collector — expectation, variance, Chebyshev*: [MU: Ch. 2, §2.4.1 (geometric phases, $\mathbb{E}[X] = nH_n$)] and [MU: Ch. 3, §3.3.1 (Chebyshev bound $\pi^2/(6c^2)$)].
- *Coupon Collector via union bound on empty bins, Poisson approximation, limiting distribution $e^{-e^{-c}}$* : [MU: Ch. 5, §§5.3–5.4.1*].

The previous chapters concerned tail bounds for sums of independent random variables and their consequences for sample complexity. We now turn to a closely related combinatorial setting in which the same concentration principles operate, but the questions take a different shape. Rather than asking how a single average deviates from its mean, we ask about the joint occupancy pattern produced by random allocations, and we will see that this perspective unifies several classical problems in probabilistic algorithm analysis.

4.1 Balls and Bins

In the “Balls and Bins” model, we consider the process of throwing m balls sequentially into n bins. The placement is performed uniformly at random, meaning each ball is placed into any of the n bins with probability $1/n$, independently of other balls.

This simple model serves as a foundational framework for analyzing various problems in probability theory, such as:

- **The Birthday Paradox (collision)**: Analyzing the probability that two people share the same birthday.
- **The Coupon Collector’s Problem (occupancy)**: Estimating the number of samples required to collect all distinct items.
- **Maximum balls (load balance)**: Distributing tasks (balls) across servers (bins) to minimize the maximum load.

4.2 The Birthday Paradox

Consider the classic Birthday Problem where we want to find the number of people m (balls) required such that the probability of at least two people sharing a birthday (bins, $n = 365$) is at least $1/2$. We refer to two balls landing in the same bin as a **collision**.

4.2.1 Exact Probability of No Collision

We can derive the probability $\mathbb{P}(\text{No Collision})$ using the chain rule of conditional probability (often referred to as the Principle of Deferred Decisions). When placing the i -th ball (where i ranges from 0 to $m - 1$), it must not land in any of the i bins already occupied by the previous balls.

The probability is given by:

$$\mathbb{P}(\text{No Collision}) = 1 \times \left(1 - \frac{1}{n}\right) \times \left(1 - \frac{2}{n}\right) \times \cdots \times \left(1 - \frac{m-1}{n}\right)$$

This can be rewritten using factorials:

$$\mathbb{P}(\text{No Collision}) = \frac{n}{n} \times \frac{n-1}{n} \times \cdots \times \frac{n-m+1}{n} = \frac{n!}{n^m(n-m)!}$$

We want to find m such that the probability of a collision is at least $1/2$, which is equivalent to $\mathbb{P}(\text{No Collision}) \leq 1/2$.

4.2.2 Approximation using Exponential Inequality

Since the factorial form is difficult to compute for large n , we use the inequality $1 - x \leq e^{-x}$ (which holds for all real x) to derive a useful bound.

Applying this inequality to each term in the product:

$$\mathbb{P}(\text{No Collision}) = \prod_{i=0}^{m-1} \left(1 - \frac{i}{n}\right) \leq \prod_{i=0}^{m-1} e^{-i/n} = \exp\left(-\sum_{i=0}^{m-1} \frac{i}{n}\right)$$

The sum in the exponent is an arithmetic series $\sum_{i=0}^{m-1} i = \frac{m(m-1)}{2}$. Thus:

$$\mathbb{P}(\text{No Collision}) \leq \exp\left(-\frac{m(m-1)}{2n}\right)$$

To ensure a collision occurs with probability at least $1/2$, we need $\mathbb{P}(\text{No Collision}) \leq 1/2$:

$$e^{-\frac{m(m-1)}{2n}} \leq \frac{1}{2} \implies -\frac{m(m-1)}{2n} \leq -\ln 2$$

Multiplying by -1 (and reversing the inequality):

$$\frac{m(m-1)}{2n} \geq \ln 2 \implies m(m-1) \geq 2n \ln 2$$

To guarantee that the collision probability is at least $1/2$, it suffices to ensure:

$$\frac{m(m-1)}{2n} \geq \ln 2$$

Instead of approximating, we observe that $(m-1)^2 < m(m-1)$. Thus, if we satisfy the stricter condition $(m-1)^2 \geq 2n \ln 2$, the original inequality will also hold. Taking the square root:

$$m-1 \geq \sqrt{2n \ln 2} \implies m \geq \sqrt{2n \ln 2} + 1$$

Therefore, choosing an integer m such that:

Birthday Threshold (Exponential Bound) [Mor: §19.4] [MU: §5.1]

$$m = \lfloor \sqrt{2n \ln 2} \rfloor + 1$$

is sufficient to ensure the probability of collision is at least $1/2$.

4.2.3 Approximation using Union Bound (Deferred Decisions)

Now let's think about this from a different angle: How large can we make m while ensuring that $\mathbb{P}(\text{collision}) \leq 1/2$?

To analyze this, we use the principle of **Deferred Decisions**. Instead of throwing all balls at once, we verify the outcome ball by ball (checking each bin sequentially). We define the collision probability as the probability that *any* ball lands in a bin that is already occupied.

Let's use the **Union Bound**. The event of a collision is the union of the events where the i -th ball causes a collision.

$$\mathbb{P}(\text{collision}) = \mathbb{P}\left(\bigcup_{i=1}^m (\text{collision at step } i)\right)$$

Using the Union Bound inequality, which states $\mathbb{P}(\cup A_i) \leq \sum \mathbb{P}(A_i)$, we can sum the probabilities of a collision occurring at each specific step:

$$\mathbb{P}(\text{collision}) \leq \sum_{i=1}^m \mathbb{P}(\text{collision at step } i)$$

When we place the i -th ball, there are already $i - 1$ balls in the bins. The probability that the i -th ball hits one of the occupied bins is at most $\frac{i-1}{n}$ (since some of the previous $i - 1$ balls may share a bin, leaving fewer than $i - 1$ bins occupied).

$$\mathbb{P}(\text{collision}) \leq 0 + \frac{1}{n} + \frac{2}{n} + \cdots + \frac{m-1}{n}$$

We can calculate this sum using the arithmetic series formula:

$$\mathbb{P}(\text{collision}) \leq \frac{1}{n}(1 + 2 + \cdots + m - 1) = \frac{m(m-1)}{2n}$$

Now, to ensure the probability of collision is at most $1/2$:

$$\frac{m(m-1)}{2n} \leq \frac{1}{2}$$

Approximating $m(m-1) \approx m^2$:

$$\frac{m^2}{2n} \leq \frac{1}{2} \implies m^2 \leq n \implies m \leq \sqrt{n}$$

So, if $m \leq \sqrt{n}$, the probability of a collision remains small. Comparing the two results:

- $m \approx \sqrt{2(\ln 2)n} + 1$: The threshold where $\mathbb{P}(\text{collision})$ becomes significant (close to $1/2$).
- $m \leq \sqrt{n}$: A conservative bound to ensure $\mathbb{P}(\text{collision})$ stays low.

Ideally, m is an integer, so we often describe this bound using the ceiling function: $m = \lceil \sqrt{n} \rceil$.

4.2.4 Intuition: Exponential Decay over $\sqrt{n}$ Intervals

To gain further insight into why the threshold lies at $\sqrt{n}$, let us consider the scaling of m in terms of $\sqrt{n}$. Suppose we choose the number of balls to be a multiple of $\sqrt{n}$, that is, $m = c\sqrt{n}$ for some constant c . Recalling the exponential bound derived above (using the approximation $m(m-1) \approx m^2$), we have:

$$\mathbb{P}(\text{No Collision}) \leq \exp\left(-\frac{m^2}{2n}\right)$$

Substituting $m = c\sqrt{n}$ into this inequality:

$$\mathbb{P}(\text{No Collision}) \leq \exp\left(-\frac{(c\sqrt{n})^2}{2n}\right) = \exp\left(-\frac{c^2 n}{2n}\right) = e^{-c^2/2}$$

This result provides a crucial intuition regarding the “intervals” of balls:

- If we view the process as adding balls in fundamental units (or intervals) of size $\sqrt{n}$, the probability of avoiding a collision does not decrease linearly.
- Instead, it decays **exponentially** with the square of the number of intervals (c^2).

To visualize this *sharp* transition, consider how the bound changes as we increase the scaling factor c :

- **At $c = 1$ ($m = \sqrt{n}$):** The probability bound is $e^{-1/2} \approx 0.606$. There is still a reasonable chance ($\approx 60\%$) of having no collisions.
- **At $c = 2$ ($m = 2\sqrt{n}$):** The bound drops to $e^{-2^2/2} = e^{-2} \approx 0.135$. The probability of avoiding a collision has plummeted to roughly 13.5%.
- **At $c = 3$ ($m = 3\sqrt{n}$):** The bound becomes $e^{-3^2/2} \approx 0.011$. A collision is now essentially guaranteed ($> 98\%$ chance).

This comparison highlights why the threshold is so distinct: as soon as m exceeds $\sqrt{n}$ (i.e., $c > 1$), the probability of no collision vanishes rapidly.

Sharper constants in the birthday-paradox threshold, including the precise relationship between m and $\sqrt{n}$ at which the collision probability crosses $1/2$, are computed in the following exercise.

Problem 4: Birthday Paradox Constants

From [MU: Ex. 5.3].

In the previous section, we established that the number of balls m required for the collision probability to reach roughly $1/2$ is proportional to $\sqrt{n}$. The objective of this exercise is to tighten these bounds and determine the specific constant factors.

Problem Statement: Determine the precise constants C_1 and C_2 that define the transition of collision probabilities.

1. Bound for Low Collision Probability:

Find the constant C_1 such that if $m \approx C_1\sqrt{n}$, the probability of a collision is bounded

by approximately $1/e$.

$$\mathbb{P}(\text{collision}) \leq \frac{1}{e}$$

2. Bound for Majority Collision Probability:

Find the constant C_2 such that if $m \approx C_2\sqrt{n}$, the probability of a collision is at least $1/2$.

$$\mathbb{P}(\text{collision}) \geq \frac{1}{2}$$

Hint: To derive these constants rigorously, consider the following steps:

- **Upper Bounding via Inequality:**

Recall that the probability of no collision is $\prod(1 - i/n)$. Use the standard inequality $1 - x \leq e^{-x}$ to show:

$$\mathbb{P}(\text{No Collision}) \leq e^{-\frac{m(m-1)}{2n}}$$

- **Exact Bound (Avoiding Approximation):**

To guarantee $\mathbb{P}(\text{No Collision}) \leq 1/2$, we need $\frac{m(m-1)}{2n} \geq \ln 2$. Instead of using the loose approximation $m(m-1) \approx m^2$, observe that $(m-1)^2 < m(m-1)$. Use this strict inequality to show that it suffices to choose:

$$m \geq \lfloor \sqrt{2n \ln 2} \rfloor + 1$$

- **Exponential Decay Argument:**

Consider what happens when m scales as $c\sqrt{n}$. Substituting this into the bound yields $\mathbb{P}(\text{No Collision}) \approx e^{-c^2/2}$. This indicates that as we add “intervals” of balls of size $\sqrt{n}$, the probability of maintaining distinct values decays exponentially. Use this perspective to determine C_1 and C_2 .

4.3 Coupon Collector’s Problem

This problem shifts the perspective from “collision avoidance” to **occupancy**. The central question is: “How many balls (m) must be thrown randomly into n bins until *all* bins contain at least one ball?”

In the context of algorithm analysis, the result is often summarized as requiring $m \approx n \ln n + cn$ balls. The term cn acts as a scaling factor for the confidence level; as c increases, the probability that all bins are filled approaches 1.

4.3.1 Derivation of the Expected Number of Balls

Let random variable X be the total number of balls required to fill all n bins. We can decompose X into a sum of phases:

$$X = \sum_{i=1}^n X_i = X_1 + X_2 + \cdots + X_n$$

where X_i represents the number of *additional* balls needed to find the i -th distinct bin after $i-1$ distinct bins have already been found.

Geometric Distribution of Phases Consider the state where we have already found $i - 1$ distinct bins. The remaining number of empty bins is $n - (i - 1)$. The probability p_i that the next ball lands in an empty bin (a “success”) is:

$$p_i = \frac{n - (i - 1)}{n}$$

The random variable X_i follows a **Geometric Distribution** with success probability p_i , denoted as $X_i \sim \text{Geom}(p_i)$. The expected value of a geometric random variable is the inverse of the probability ($1/p_i$). Therefore:

$$\mathbb{E}[X_i] = \frac{1}{p_i} = \frac{n}{n - i + 1}$$

For example:

- X_1 : To find the 1st bin, any bin works. $p_1 = n/n = 1$, so $\mathbb{E}[X_1] = 1$.
- X_2 : To find the 2nd bin, we must avoid the 1 already filled. $p_2 = (n-1)/n$, so $\mathbb{E}[X_2] = n/(n-1)$.
- X_n : To find the last bin, only 1 is left. $p_n = 1/n$, so $\mathbb{E}[X_n] = n$.

Summation and Harmonic Series Using the Linearity of Expectation, the total expected number of balls is:

$$\mathbb{E}[X] = \sum_{i=1}^n \mathbb{E}[X_i] = \frac{n}{n} + \frac{n}{n-1} + \cdots + \frac{n}{1}$$

Factoring out n , we get:

$$\mathbb{E}[X] = n \left(\frac{1}{1} + \frac{1}{2} + \cdots + \frac{1}{n} \right) = nH_n$$

where H_n is the n -th Harmonic number.

4.3.2 Bounding the Harmonic Series

We can bound H_n using the natural logarithm by comparing the summation to the integral of $f(x) = 1/x$.

$$\ln n + \frac{1}{n} < H_n < \ln n + 1$$

From the integral test logic shown above:

- **Lower Bound:** The area under the curve $y = 1/x$ from 1 to $n + 1$ is $\ln(n + 1)$, which is roughly $\ln n$.
- **Upper Bound:** By shifting the rectangles, we can see $H_n \leq 1 + \ln n$.

Substituting this back into our expectation equation $\mathbb{E}[X] = nH_n$, we arrive at the final bounds:

Expected Coupon Collector Time [MU: §2.4.1]

$$n \ln n + 1 < \mathbb{E}[X] < n \ln n + n$$

Thus, the expected number of balls required to collect all coupons grows as $n \ln n$.

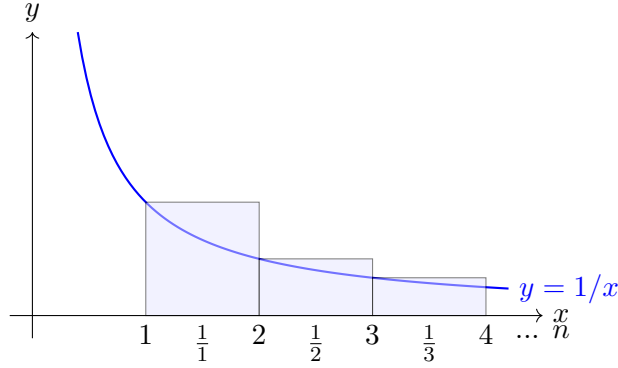


Figure 4: Visualizing the Harmonic Series. The sum of the areas of the rectangles $(1 + 1/2 + 1/3 + \dots)$ is greater than the area under the curve $\int_1^{n+1} \frac{1}{x} dx = \ln(n+1)$, illustrating the approximation.

4.3.3 Analysis of the Scaling Factor c

We now introduce a scaling factor c to analyze how likely the number of balls X is to exceed the expected value. We consider the threshold $n \ln n + cn$.

Approximation 1: Markov's Inequality First, we apply **Markov's Inequality**, which states that for a non-negative random variable X , $\mathbb{P}(X \geq a) \leq \frac{\mathbb{E}[X]}{a}$. Using our previously derived upper bound $\mathbb{E}[X] < n \ln n + n$, and setting the threshold $a = n \ln n + cn$:

$$\mathbb{P}(X > n \ln n + cn) \leq \frac{\mathbb{E}[X]}{n \ln n + cn} < \frac{n \ln n + n}{n \ln n + cn}$$

Dividing the numerator and denominator by $n \ln n$:

$$\mathbb{P}(X > n \ln n + cn) < \frac{1 + \frac{1}{\ln n}}{1 + \frac{c}{\ln n}}$$

Roughly speaking, as c increases, this probability bound decreases inversely proportional to c (i.e., $\propto 1/c$). However, this bound is quite loose.

Approximation 2: Chebyshev's Inequality To get a tighter bound, we use **Chebyshev's Inequality**, which bounds the probability based on the variance.

$$\mathbb{P}(|X - \mathbb{E}[X]| \geq \delta) \leq \frac{\text{Var}(X)}{\delta^2}$$

Here, we set the deviation $\delta = cn$ and approximate the mean as $n \ln n$. First, we need to calculate the variance of X . Since the phases X_i are independent, the variance of the sum is the sum of the variances:

$$\text{Var}(X) = \sum_{i=1}^n \text{Var}(X_i)$$

Recall that for a geometric distribution with probability p_i , the variance is $\text{Var}(X_i) = \frac{1-p_i}{p_i^2}$. We can bound this by:

$$\text{Var}(X_i) = \frac{1-p_i}{p_i^2} < \frac{1}{p_i^2}$$

Substituting $p_i = \frac{n-i+1}{n}$, or essentially $p \in \{\frac{1}{n}, \frac{2}{n}, \dots, \frac{n}{n}\}$:

$$\text{Var}(X) < \sum_{j=1}^n \frac{1}{(j/n)^2} = n^2 \sum_{j=1}^n \frac{1}{j^2}$$

Using the solution to the Basel problem, we know that the infinite sum converges to:

$$\sum_{j=1}^{\infty} \frac{1}{j^2} = \frac{\pi^2}{6}$$

Therefore, the variance is bounded by:

$$\text{Var}(X) < \frac{\pi^2 n^2}{6}$$

Now, applying Chebyshev's Inequality with $\delta = cn$:

$$\mathbb{P}(|X - n \ln n| > cn) \leq \frac{\text{Var}(X)}{(cn)^2} < \frac{\frac{\pi^2 n^2}{6}}{c^2 n^2}$$

Simplifying the expression:

Chebyshev Bound on Coupon Collector [MU: §3.3.1]

$$\mathbb{P}(|X - n \ln n| > cn) < \frac{\pi^2}{6c^2}$$

Comparison of Bounds Comparing the two results highlights the power of using the second moment (variance):

- **Markov's Inequality** provides a bound that decays roughly as $\frac{1}{c}$.
- **Chebyshev's Inequality** provides a bound that decays as $\frac{1}{c^2}$.

Since $1/c^2$ decreases much faster than $1/c$ for large c , Chebyshev's inequality gives us a much stronger guarantee that the number of balls required will be close to $n \ln n$.

A Chernoff-style high-probability tail bound for the coupon collector's stopping time, complementing the expectation bound above, is the subject of the following exercise.

Problem 5: Chernoff Bound for Coupon Collector

Adapted from [Mor: Ex. 18.7(c)(d)] (same setup, but with Markov / Chebyshev only — the homework asks for the stronger Chernoff version).

In the previous section, we used Chebyshev's inequality to show that the probability of exceeding the expected number of balls decays as $1/c^2$. The goal of this homework is to obtain a much stronger bound using the **Chernoff Bound**.

Problem Statement: Apply the Chernoff bound to the random variable $X = \sum X_i$ (where X_i are independent geometric random variables) to bound the probability:

$$\mathbb{P}(X > n \ln n + cn)$$

Find the rate at which this probability approaches 0 as c increases. You should expect an exponential decay, which is significantly tighter than the polynomial decay given by Chebyshev's inequality.

4.4 Alternative Analysis: Union Bound on Empty Bins

Instead of analyzing the sum of geometric variables (X), let's change our perspective back to the "Balls and Bins" model. This approach yields a surprisingly tight bound without using complex tools like Chernoff bounds.

Let m be the number of balls thrown, where $m = n \ln n + cn$. We are interested in the probability $\mathbb{P}(X > m)$.

- **Why examine this probability?** The event $X > m$ means "more than m balls are needed to fill all bins." This is logically equivalent to saying "After throwing m balls, **at least one bin remains empty**."

We want to bound the probability that *any* bin is empty:

$$\mathbb{P}(X > m) = \mathbb{P}\left(\bigcup_{i=1}^n (\text{Bin } i \text{ is empty})\right)$$

Using the **Union Bound** ($\mathbb{P}(\cup A_i) \leq \sum \mathbb{P}(A_i)$):

$$\mathbb{P}\left(\bigcup_{i=1}^n (\text{Bin } i \text{ is empty})\right) \leq \sum_{i=1}^n \mathbb{P}(\text{Bin } i \text{ is empty})$$

4.4.1 Calculating the Probability

First, consider a specific bin i . The probability that a single ball *misses* bin i is $1 - \frac{1}{n}$. Since the m balls are thrown independently, the probability that *all* m balls miss bin i is:

$$\mathbb{P}(\text{Bin } i \text{ is empty}) = \left(1 - \frac{1}{n}\right)^m$$

Summing this over all n bins:

$$\sum_{i=1}^n \mathbb{P}(\text{Bin } i \text{ is empty}) = n \times \left(1 - \frac{1}{n}\right)^m$$

Now, we apply the standard inequality $1 - x \leq e^{-x}$ (where $x = 1/n$):

$$n \left(1 - \frac{1}{n}\right)^m \leq n \left(e^{-1/n}\right)^m = ne^{-m/n}$$

Next, substitute $m = n \ln n + cn$:

$$ne^{-\frac{n \ln n + cn}{n}} = ne^{-(\ln n + c)} = ne^{-\ln n} e^{-c}$$

Since $e^{-\ln n} = \frac{1}{n}$:

$$n \cdot \frac{1}{n} \cdot e^{-c} = e^{-c}$$

Thus, we arrive at the bound:

Union Bound on Empty Bins [MU: §5.4.1*]

$$\mathbb{P}(X > n \ln n + cn) \leq e^{-c}$$

4.4.2 Remark: Is this bound tight?

You might wonder why a simple union bound gives such a tight result (e^{-c}), if it matches the actual limiting distribution behavior.

1. **Dependency:** The events “Bin i is empty” are **not independent**. If Bin 1 is empty, it means all balls landed in the other bins, which slightly increases the chance that Bin 2 is full. These events are negatively correlated.
2. **Inclusion-Exclusion Principle:** The Union Bound corresponds to the first term of the Inclusion-Exclusion principle:

$$\mathbb{P}(\cup(\text{Bin } i \text{ empty})) = \sum \mathbb{P}(\text{Bin } i \text{ empty}) - \sum \mathbb{P}(\text{Bin } i \text{ empty} \cap \text{Bin } j \text{ empty}) + \dots$$

When m is large (around $n \ln n$), the probability of *two* specific bins being empty simultaneously becomes negligibly small compared to the probability of a single bin being empty. Therefore, the intersection terms (the “overlaps”) are insignificant, making the first term (the Union Bound) a very good approximation of the true probability.

4.5 Poisson Approximation and Limiting Distribution

In this section, we refine our bounds by analyzing the exact distribution of balls in the bins. Direct calculation using combinatorics is difficult due to the dependencies between bins (since the total number of balls is fixed at m).

To simplify, we approximate the distribution using the **Poisson Distribution**. This allows us to treat the bins as if they were statistically independent.

Step 1: Binomial to Poisson Approximation Let Z_i be the number of balls that fall into a specific bin i . Since each of the m balls has a probability $1/n$ of landing in bin i , Z_i follows a Binomial distribution:

$$Z_i \sim \text{Binomial}\left(m, \frac{1}{n}\right)$$

The probability that bin i contains exactly k balls is given by:

$$\mathbb{P}(Z_i = k) = \binom{m}{k} \left(\frac{1}{n}\right)^k \left(1 - \frac{1}{n}\right)^{m-k}$$

Expanding the binomial coefficient and rearranging terms:

$$\mathbb{P}(Z_i = k) = \frac{m(m-1)\dots(m-k+1)}{k!} \cdot \frac{1}{n^k} \cdot \left(1 - \frac{1}{n}\right)^{m-k}$$

For large n and m , we can use the approximations $m(m-1)\cdots \approx m^k$ and $(1-1/n)^m \approx e^{-m/n}$. Also, $(1-1/n)^{-k} \approx 1$ for small k . Thus:

$$\mathbb{P}(Z_i = k) \approx \frac{m^k}{n^k k!} e^{-m/n} = \frac{(m/n)^k}{k!} e^{-(m/n)}$$

This is exactly the probability mass function of a Poisson distribution with parameter $\lambda = m/n$:

$$Z_i \approx \text{Poisson}(\lambda = m/n)$$

Step 2: Probability of an Empty Bin We are interested in whether a bin is empty ($k = 0$). Using the Poisson approximation:

$$\mathbb{P}(\text{Bin } i \text{ is empty}) = \mathbb{P}(Z_i = 0) \approx \frac{(m/n)^0}{0!} e^{-m/n} = e^{-m/n}$$

Substitute the threshold number of balls $m = n \ln n + cn$:

$$\mathbb{P}(\text{Bin } i \text{ is empty}) = e^{-\frac{n \ln n + cn}{n}} = e^{-(\ln n + c)} = e^{-\ln n} e^{-c} = \frac{e^{-c}}{n}$$

Step 3: Deriving the Limiting Distribution We want to find the probability that **all** bins are non-empty (i.e., the coupon collector is done).

$$\mathbb{P}(\text{Bin } i \text{ is non-empty}) = 1 - \mathbb{P}(\text{Bin } i \text{ is empty}) = 1 - \frac{e^{-c}}{n}$$

Here, we apply a **heuristic assumption**: Under the Poisson approximation, the bin counts $Z_1, \dots, Z_n$ are treated as approximately independent (in reality they are dependent since the total $\sum Z_i = m$ is fixed, but for large n this constraint has a negligible effect on any single bin). Therefore, the probability that *all* n bins are non-empty is approximately the product of their individual probabilities:

$$\mathbb{P}(\text{All bins non-empty}) \approx \left(1 - \frac{e^{-c}}{n}\right)^n$$

Taking the limit as $n \rightarrow \infty$, and using the definition $e^x = \lim_{n \rightarrow \infty} (1 + x/n)^n$:

$$\lim_{n \rightarrow \infty} \mathbb{P}(\text{All bins non-empty}) = e^{-e^{-c}}$$

4.5.1 Conclusion

The event “All bins are non-empty” is equivalent to “The number of balls X needed is less than or equal to m ”. Therefore:

Limiting Distribution of Coupon Collector [MU: §§5.4–5.4.1*]

$$\lim_{n \rightarrow \infty} \mathbb{P}(X \leq n \ln n + cn) = e^{-e^{-c}}$$

Conversely, the probability that we are **not** done (we need more balls) is:

$$\lim_{n \rightarrow \infty} \mathbb{P}(X > n \ln n + cn) = 1 - e^{-e^{-c}}$$

Interpretation of c : The constant c measures how far we are from the threshold $n \ln n$.

- **If c is large positive ($c > 0$):** We are throwing $m > n \ln n$ balls. The term e^{-c} becomes small, so $\mathbb{P}(X \leq m) \approx e^0 = 1$. With high probability, all bins are filled.
- **If c is large negative ($c < 0$):** We are throwing $m < n \ln n$ balls. The term e^{-c} becomes large, so $\mathbb{P}(X \leq m) \approx e^{-\text{large}} \approx 0$. With high probability, we have not yet filled all bins.

This behavior characterizes a **sharp threshold**: the probability mass of X is concentrated in a narrow interval around the mean $n \ln n$, and a small shift in the number of balls (changing c) causes the probability of success to transition rapidly from near 0 to near 1.

Further Reading

This section developed the balls-and-bins framework, the birthday paradox, and the sharp threshold $m \approx n \ln n$ for the Coupon Collector problem. The reading below shows how these same concentration phenomena reappear in a modern online matching problem, where free servers behave like “empty bins” that fill up over time.

- Gupta et al. [16], “Stochastic Online Metric Matching.” Studies min-cost online perfect matching where n servers sit at fixed metric points and n requests arrive online i.i.d. from a known distribution $\mathcal{D}$. The paper gives an $O((\log \log n)^2)$ -competitive algorithm on general metrics (separating the i.i.d. setting from the $\Omega(\log n)$ bound that holds against adversarial and random-order arrivals), and a 9-competitive algorithm on tree and line metrics. The analysis is anchored on a balls-and-bins concentration argument: the imbalance between the set of free servers and the request distribution is controlled by bounding the load of the top- k heaviest bins, a direct descendant of the techniques developed in this section.

5 Coupon Collector and Load Balancing

Textbook Roadmap

For a parallel treatment in the course textbooks, see:

- *Independent-Poisson model, Poisson-multinomial conditioning, sandwiching argument:* [MU: Ch. 5, §5.4].
- *Load balancing — max load $\leq 3 \ln n / \ln \ln n$ with probability $\geq 1 - 1/n$:* [MU: Ch. 5, §5.2.1, Lem. 5.1]; [Mor: Ch. 19, §§19.4–19.5].

The previous derivation rested on a heuristic independence assumption among the bin counts. To justify this rigorously, we now reanalyze the same problem under a fully independent model and confirm that the answer is unchanged.

5.1 Poisson Approximation with Independent Bins

In the exact Coupon Collector problem, the total number of balls m is fixed. This introduces dependencies between bins (if one bin contains a ball, it is slightly less likely for another to contain one). To simplify the analysis, we consider a model where the number of balls in each bin is determined independently.

Let $\mu = m/n$ be the expected number of balls per bin. Since $m = n \ln n + cn$, we can write μ as:

$$\mu = \frac{n \ln n + cn}{n} = \ln n + c$$

We define random variables $Y_1^{(\mu)}, Y_2^{(\mu)}, \dots, Y_n^{(\mu)}$ representing the number of balls in each bin i . We assume each $Y_i^{(\mu)}$ follows a Poisson distribution with parameter μ :

$$Y_i^{(\mu)} \sim \text{Poisson}(\mu)$$

Under this model, the random variables $Y_i^{(\mu)}$ are **independent**. The sum of these variables, $Y = \sum Y_i^{(\mu)}$, follows a Poisson distribution with parameter $n\mu = m$.

Probability of All Bins Being Full: Let $\mathcal{E}$ be the event that all bins are full. In terms of our variables, this corresponds to the event where every bin contains at least one ball:

$$\mathcal{E} = \{\forall i : Y_i^{(\mu)} > 0\}$$

Since the variables $Y_i^{(\mu)}$ are independent, the probability of the intersection of these events is the product of their individual probabilities:

$$\mathbb{P}(\mathcal{E}) = \mathbb{P}(Y_1^{(\mu)} > 0) \times \mathbb{P}(Y_2^{(\mu)} > 0) \times \dots \times \mathbb{P}(Y_n^{(\mu)} > 0)$$

For a Poisson random variable, the probability of observing zero is $\mathbb{P}(Y_i^{(\mu)} = 0) = e^{-\mu}$. Therefore, the probability of observing a non-zero value is:

$$\mathbb{P}(Y_i^{(\mu)} > 0) = 1 - e^{-\mu}$$

Substituting this into the product equation:

$$\mathbb{P}(\mathcal{E}) = (1 - e^{-\mu})^n$$

Limit Analysis: Substituting $\mu = \ln n + c$ into the expression:

$$\mathbb{P}(\mathcal{E}) = \left(1 - e^{-(\ln n + c)}\right)^n$$

Using the property $e^{-(\ln n + c)} = e^{-\ln n} e^{-c} = \frac{1}{n} e^{-c}$:

$$\mathbb{P}(\mathcal{E}) = \left(1 - \frac{e^{-c}}{n}\right)^n$$

Taking the limit as $n \rightarrow \infty$ and applying the standard limit definition $\lim_{n \rightarrow \infty} \left(1 - \frac{x}{n}\right)^n = e^{-x}$:

$$\lim_{n \rightarrow \infty} \left(1 - \frac{e^{-c}}{n}\right)^n = e^{-e^{-c}}$$

This matches the limiting distribution we derived earlier under the heuristic independence assumption, confirming that the rigorous independent-bin analysis yields the same answer.

5.1.1 Equivalence of the Models

We have shown that for the independent Poisson variables, the probability of filling all bins converges to $e^{-e^{-c}}$:

$$\lim_{n \rightarrow \infty} \mathbb{P}(\mathcal{E}) = e^{-e^{-c}}$$

However, the original Coupon Collector problem relies on a fixed number of balls, m . We denote the event that m balls are sufficient to fill all bins as $\{X \leq m\}$. To be rigorous, we must establish that the probability calculated under the Poisson assumption is asymptotically equivalent to the probability in the fixed-ball model:

$$\lim_{n \rightarrow \infty} \mathbb{P}(X \leq n \ln n + cn) = \lim_{n \rightarrow \infty} \mathbb{P}(\mathcal{E})$$

Justification: The Poisson model assumes the total number of balls is a random variable $Y \sim \text{Poisson}(m)$, rather than a fixed number m . However, as m becomes large, the Poisson distribution becomes highly concentrated around its mean. Consequently, the relative difference between the “fixed m ” model and the “Poisson fluctuating around m ” model vanishes as $n \rightarrow \infty$.

To prove this rigorously, we need to show two things:

1. **Linkage:** $\mathbb{P}(\mathcal{E} \mid Y = m) = \mathbb{P}(X \leq m)$
2. **Convergence:** $\lim_{n \rightarrow \infty} \mathbb{P}(\mathcal{E}) = \lim_{n \rightarrow \infty} \mathbb{P}(\mathcal{E} \mid Y = m)$

Step 1: Conditioning independent Poissons on their sum yields multinomial (Poisson-multinomial link). First, we justify why conditioning the Poisson model on $Y = m$ recovers the exact Coupon Collector model. We analyze the conditional probability:

$$\mathbb{P}(Y_1^{(\mu)} = k_1, \dots, Y_n^{(\mu)} = k_n \mid Y = k)$$

where $Y_i^{(\mu)}$ are the independent Poisson counts for each bin, and $Y = \sum Y_i^{(\mu)}$ is the total number of balls. Using the definition of conditional probability:

$$\mathbb{P}(Y_1^{(\mu)} = k_1, \dots \mid Y = k) = \frac{\mathbb{P}(Y_1^{(\mu)} = k_1) \times \dots \times \mathbb{P}(Y_n^{(\mu)} = k_n)}{\mathbb{P}(Y = k)}$$

Substituting the Poisson probability mass functions (where the numerator is a product of independent Poissons and the denominator is a Poisson with mean $n\mu$):

$$= \frac{\frac{e^{-\mu} \mu^{k_1}}{k_1!} \times \dots \times \frac{e^{-\mu} \mu^{k_n}}{k_n!}}{\frac{e^{-n\mu} (n\mu)^k}{k!}}$$

Canceling the exponential terms ($e^{-n\mu}$) and grouping the powers of μ :

$$= \frac{k!}{k_1! \dots k_n!} \left(\frac{1}{n} \right)^k$$

This is exactly the **Multinomial Distribution** probability for throwing k balls into n bins uniformly. Therefore, the distribution of balls in the Poisson model, *given* that the total number of balls is fixed to m , is identical to the fixed-ball model. Thus:

$$\mathbb{P}(\mathcal{E} \mid Y = m) = \mathbb{P}(X \leq m)$$

Step 2: Concentration and Sandwiching. Now we must show that $\mathbb{P}(\mathcal{E})$ converges to $\mathbb{P}(\mathcal{E} \mid Y = m)$. We do this by defining a narrow range around the mean m where the Poisson variable Y is almost guaranteed to land.

Intuition for Concentration: Why does the Poisson distribution concentrate? Recall that for a Poisson variable Y with mean m , the standard deviation is $\sigma = \sqrt{m}$. As m grows large, the ratio of the spread to the mean is $\frac{\sigma}{m} = \frac{1}{\sqrt{m}}$, which tends to 0. Visually, if we plot the probability mass function, it looks like a bell curve that becomes relatively narrower and taller as m increases. This means Y rarely deviates from m by more than a few standard deviations.

Let $\Delta = \sqrt{2m \ln m}$. We consider the concentration of Y within the interval $[m - \Delta, m + \Delta]$.

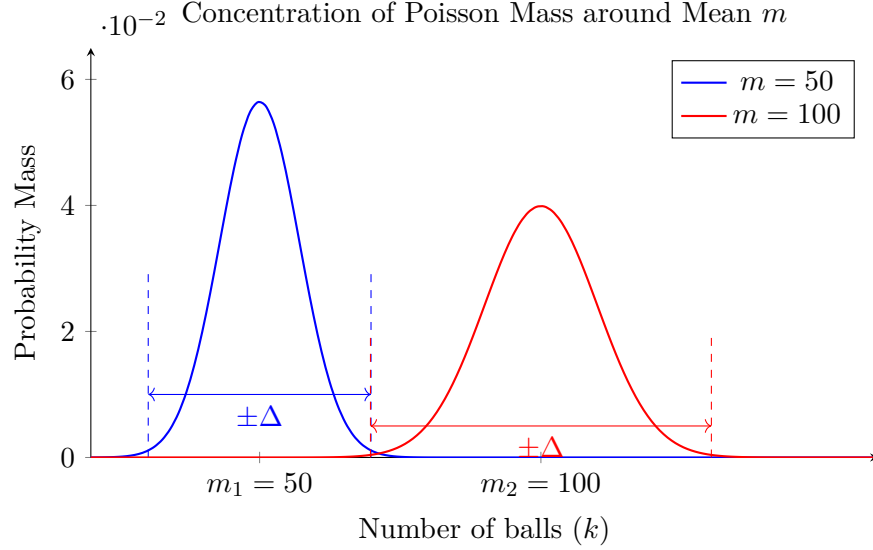


Figure 5: Poisson PMFs for increasing m . Note that while the variance m increases, the distribution becomes sharper *relative* to the mean. The dashed lines indicate the concentration range $m \pm \Delta$. The total probability mass outside this range vanishes as $m \rightarrow \infty$.

Using the Law of Total Probability, we can split $\mathbb{P}(\mathcal{E})$ based on whether Y falls within the concentration bound $A = \{|Y - m| \leq \Delta\}$:

$$\mathbb{P}(\mathcal{E}) = \mathbb{P}(\mathcal{E} \mid A)\mathbb{P}(A) + \mathbb{P}(\mathcal{E} \mid A^c)\mathbb{P}(A^c)$$

We analyze the limits of these terms as $n \rightarrow \infty$:

- From Chebyshev's inequality (proven below), we know $\mathbb{P}(A^c) \rightarrow 0$. Since $\mathbb{P}(\mathcal{E} \mid A^c) \leq 1$, the second term vanishes:

$$\mathbb{P}(\mathcal{E} \mid A^c)\mathbb{P}(A^c) \rightarrow 0$$

- Consequently, $\mathbb{P}(A) \rightarrow 1$. Thus, the first term simplifies:

$$\mathbb{P}(\mathcal{E} \mid A)\mathbb{P}(A) \approx \mathbb{P}(\mathcal{E} \mid A) \cdot 1$$

Combining these, we get the approximation:

$$\mathbb{P}(\mathcal{E}) \approx \mathbb{P}(\mathcal{E} \mid |Y - m| \leq \Delta)$$

Concentration via Chebyshev's inequality. Since $Y \sim \text{Poisson}(m)$, $\text{Var}(Y) = m$.

$$\mathbb{P}(|Y - m| > \sqrt{2m \ln m}) \leq \frac{\text{Var}(Y)}{(\sqrt{2m \ln m})^2} = \frac{m}{2m \ln m} = \frac{1}{2 \ln m}.$$

As $m \rightarrow \infty$, this probability goes to 0. □

Thus, we can approximate $\mathbb{P}(\mathcal{E}) \approx \mathbb{P}(\mathcal{E} \mid |Y - m| \leq \Delta)$. To proceed, we utilize **Monotonicity**. The event $\mathcal{E}$ (all bins full) becomes *more likely* as the number of balls increases. Therefore:

$$\mathbb{P}(\mathcal{E} \mid Y = m - \Delta) \leq \mathbb{P}(\mathcal{E} \mid |Y - m| \leq \Delta) \leq \mathbb{P}(\mathcal{E} \mid Y = m + \Delta)$$

Since $\mathbb{P}(\mathcal{E} \mid Y = m)$ lies between these bounds as well, we have “sandwiched” the probabilities.

Step 3: Collapsing the Sandwich. To prove the limits are equal, we must show that the gap between the upper and lower bounds vanishes:

$$\lim_{n \rightarrow \infty} [\mathbb{P}(\mathcal{E} \mid Y = m + \Delta) - \mathbb{P}(\mathcal{E} \mid Y = m - \Delta)] = 0$$

The difference corresponds to the probability that the event $\mathcal{E}$ occurs with $m + \Delta$ balls but *not* with $m - \Delta$ balls.

$$\mathbb{P}(m - \Delta < X \leq m + \Delta)$$

Consider the “gap” of 2Δ balls. For the event status to change, at least one of these 2Δ extra balls must land in a bin that was empty at step $m - \Delta$. To bound this rigorously, we first estimate the expected number of empty bins at $m - \Delta$ balls. Each bin is empty with probability $(1 - 1/n)^{m-\Delta}$, so by linearity of expectation:

$$\mathbb{E}[\# \text{ empty bins at } m - \Delta] \leq n \left(1 - \frac{1}{n}\right)^{m-\Delta} \leq n \cdot e^{-(m-\Delta)/n}$$

Substituting $m = n \ln n + cn$:

$$n \cdot e^{-(m-\Delta)/n} = n \cdot e^{-\ln n - c + \Delta/n} = e^{-c} \cdot e^{\Delta/n}$$

Since $\Delta = \sqrt{2m \ln m}$ and $m \leq 2n \ln n$ for large n , we have $\Delta \leq \sqrt{12n \cdot \ln n}$, so $\Delta/n \leq \sqrt{12 \ln n / \sqrt{n}} \rightarrow 0$. Thus:

$$\mathbb{E}[\# \text{ empty bins at } m - \Delta] \leq e^{-c} \cdot e^{o(1)} = O(1)$$

Each of the 2Δ extra balls lands in any given empty bin with probability $1/n$, so the probability that any extra ball hits an empty bin is bounded by the expected number of empty bins times $2\Delta/n$:

$$\mathbb{P}(\text{Gap event}) \leq \frac{2\Delta \cdot O(1)}{n} = O\left(\frac{\ln n}{\sqrt{n}}\right) \rightarrow 0 \quad \text{as } n \rightarrow \infty$$

Since the difference between the upper and lower bounds goes to 0, the sandwich collapses. Therefore:

$$\lim_{n \rightarrow \infty} \mathbb{P}(\mathcal{E}) = \lim_{n \rightarrow \infty} \mathbb{P}(\mathcal{E} \mid Y = m) = \lim_{n \rightarrow \infty} \mathbb{P}(X \leq m)$$

This completes the proof that the Poisson limit applies to the fixed-ball model.

5.2 Load Balancing

Consider the problem of throwing n balls into n bins uniformly at random. Let X_i be the number of balls in bin i . We are interested in bounding the maximum load, denoted as $\max_i X_i$.

We aim to show that with high probability (whp), the maximum load is bounded by a constant K :

$$\max_i X_i \leq K \quad \text{whp}$$

In this context, “with high probability” means the probability is at least $1 - \frac{1}{n}$ as $n \rightarrow \infty$.

5.2.1 Bounding the Probability

First, consider the probability that a specific bin receives at least k balls. We can bound this by the probability of choosing k specific balls to land in that bin:

$$\mathbb{P}(X_i \geq k) \leq \binom{n}{k} \left(\frac{1}{n}\right)^k$$

To bound the maximum load across all n bins, we apply the **Union Bound**:

$$\mathbb{P}(\max_i X_i \geq k) \leq \sum_{i=1}^n \mathbb{P}(X_i \geq k) = n \cdot \mathbb{P}(X_1 \geq k)$$

Substituting the individual bound:

$$\mathbb{P}(\max_i X_i \geq k) \leq n \binom{n}{k} \left(\frac{1}{n}\right)^k$$

We can simplify the term $\binom{n}{k} \left(\frac{1}{n}\right)^k$ using the definition of the binomial coefficient:

$$\binom{n}{k} \frac{1}{n^k} = \frac{n!}{k!(n-k)!} \cdot \frac{1}{n^k} = \frac{1}{k!} \cdot \underbrace{\frac{n!}{(n-k)!n^k}}_{<1} < \frac{1}{k!}$$

Note that $\frac{n!}{(n-k)!} = n(n-1)\dots(n-k+1)$, which is strictly less than n^k . Therefore, the probability is bounded by:

$$\mathbb{P}(\max_i X_i \geq k) < \frac{n}{k!}$$

5.2.2 Applying the Taylor Bound

We use the standard inequality derived from the Taylor expansion of e^k :

$$\frac{k^k}{k!} < e^k \implies \frac{1}{k!} < \left(\frac{e}{k}\right)^k$$

Applying this to our bound:

$$\mathbb{P}(\max_i X_i \geq k) < n \left(\frac{e}{k}\right)^k$$

5.2.3 Determining the Threshold K

How do we choose the value of K ? We want the probability term $n(e/k)^k$ to be small, say less than $1/n$. If we set $(e/k)^k \approx 1/n^2$, taking the logarithm gives roughly $k \ln k \approx 2 \ln n$. Since k grows slower than $\ln n$, an approximate solution to $k \ln k = \Theta(\ln n)$ involves the Lambert W function behavior, which gives $k \approx \frac{c \ln n}{\ln \ln n}$ as the target asymptotic form to plug in.

The value of c that rigorously bounds the probability will be clear from plugging $\frac{c \ln n}{\ln \ln n}$ in the expression for the probability. In this case, $c = 3$ is large enough to get a $1/n$ bound on the probability. We claim that the maximum load is bounded by:

$$K = \frac{3 \ln n}{\ln \ln n}$$

Substituting $k = K$ into the inequality:

$$\mathbb{P}(\max_i X_i \geq K) < n \left(\frac{e}{K} \right)^K$$

Substituting $K = \frac{3 \ln n}{\ln \ln n}$:

$$n \left(\frac{e \ln \ln n}{3 \ln n} \right)^{\frac{3 \ln n}{\ln \ln n}}$$

Since $e < 3$, we have $\frac{e}{3} < 1$, so we can loosely bound the base as $\frac{\ln \ln n}{\ln n}$:

$$< n \left(\frac{\ln \ln n}{\ln n} \right)^{\frac{3 \ln n}{\ln \ln n}}$$

To analyze the asymptotic behavior, we convert this expression to the exponential form $\exp(\dots)$:

$$= \exp \left(\ln n + \frac{3 \ln n}{\ln \ln n} (\ln(\ln \ln n) - \ln(\ln n)) \right)$$

Simplifying the exponent:

$$\ln n + \frac{3 \ln n}{\ln \ln n} (\ln \ln \ln n - \ln \ln n) = \ln n + \frac{3 \ln n \cdot \ln \ln \ln n}{\ln \ln n} - 3 \ln n$$

For large n , the term involving $\ln \ln \ln n$ is negligible compared to $\ln n$. The dominant terms are $\ln n - 3 \ln n = -2 \ln n$.

$$\approx \exp(-2 \ln n) = \frac{1}{n^2}$$

Since $\frac{1}{n^2} \leq \frac{1}{n}$ for $n \geq 1$:

Load Balancing Bound [MU: Lem. 5.1] [Mor: §19.4]

$$\mathbb{P} \left(\max_i X_i \geq K \right) \leq \frac{1}{n}$$

Thus, with probability at least $1 - 1/n$, the maximum load in any bin is at most $\frac{3 \ln n}{\ln \ln n}$.

Establishing $\mathbb{E}[X] = \Omega(n^{2/3})$ for the number of heavy bins is the content of the next exercise; a companion problem that follows it boosts this expectation bound to a high-probability statement using the variance bound $\text{Var}(X) \leq n$.

Problem 6: Heavy Bins in Expectation

From [Mor: Ex. 19.7].

Problem Statement: Consider throwing n balls into n bins uniformly at random. Let k be defined as:

$$k = \frac{c \ln n}{\ln \ln n} \quad \text{where } c = \frac{1}{3}$$

Prove that the expected number of bins with at least k balls is $\Omega(n^{2/3})$ for sufficiently large n .

Hint: To derive this bound, you may follow these steps:

1. **Lower Bound Probability for a Single Bin:**

Show that for a specific bin j , the probability it contains at least k balls satisfies:

$$\mathbb{P}(\text{Bin } j \geq k \text{ balls}) \geq \frac{1}{2ek^k}$$

(Note: You can use the fact that $(1 - 1/n)^n$ is increasing and $\binom{n}{k} > (n/k)^k$.)

2. **Bound the Term k^k :**

Using the definition of $k = \frac{c \ln n}{\ln \ln n}$, prove the algebraic lemma:

$$k^k \leq n^c$$

3. **Combine to bound Expectation:**

Let X be the number of bins with at least k balls. Use Linearity of Expectation and the bounds derived above to show:

$$\mathbb{E}[X] \geq \Omega(n^{1-c})$$

Substitute $c = 1/3$ to obtain the final result $\Omega(n^{2/3})$.

Problem 7: Heavy Bins with High Probability

From [Mor: Ex. 19.8].

Problem Statement: Using the same setup as Problem 6 (with $c = 1/3$ and $k = \frac{c \ln n}{\ln \ln n}$), our goal is to show that with reasonably high probability, **at least some bin** has $\geq k$ balls. Let X denote the number of bins with at least k balls. We want to prove that the probability of having *no* such bins approaches zero:

$$\mathbb{P}(X = 0) \leq O\left(\frac{1}{n^{1/3}}\right)$$

Hint: Use **Chebyshev's Inequality** to bound the probability that X deviates from its mean.

1. **Chebyshev Application:**

Use Chebyshev's inequality to upper bound $\mathbb{P}(X = 0)$ in terms of $\text{Var}(X)$ and $\mathbb{E}[X]$:

$$\mathbb{P}(X = 0) \leq \frac{\text{Var}(X)}{(\mathbb{E}[X])^2}$$

2. Bound the Variance:

Prove that $\text{Var}(X) \leq n$.

To do this, recall that $\text{Var}(X) = \sum \text{Var}(X_i) + \sum_{i \neq j} \text{Cov}(X_i, X_j)$, where X_i is the indicator variable for bin i having $\geq k$ balls.

- Argue that the covariance term $\text{Cov}(X_i, X_j)$ is negative (negative correlation between bins).
- Therefore, $\text{Var}(X) \leq \sum \text{Var}(X_i) \leq \mathbb{E}[X] \leq n$. (Note: Simply bounding $\text{Var}(X) \leq n$ is sufficient given the denominator grows faster).

3. Final Bound:

Combine the result from Problem 6 ($\mathbb{E}[X] = \Omega(n^{2/3})$) and the variance bound ($\text{Var}(X) \leq n$) to show:

$$\mathbb{P}(X = 0) \leq \frac{n}{(n^{2/3})^2} = \frac{1}{n^{1/3}}$$

Further Reading

This section analysed the maximum load when n balls are thrown into n bins uniformly at random, establishing $\max_i X_i \leq 3 \ln n / \ln \ln n$ with high probability. A natural next question is what happens once each ball is allowed a *choice* between two or more random bins, a question with a rich follow-up literature.

- [MU: Ch. 17] (*Balanced Allocations and Cuckoo Hashing*). The textbook chapter on the *power of two choices*: when each of n balls is placed into the lesser-loaded of two uniformly random bins, the maximum load drops from $\Theta(\log n / \log \log n)$ to $\Theta(\log \log n)$. The chapter also develops cuckoo hashing as an algorithmic application.
- Georgakopoulos et al. [18], “The Power of Two Choices for Random Walks.” Carries the power-of-two-choices paradigm from balls-into-bins to random walks on graphs: at each step the walker is offered two independent uniform random neighbours and may pick one. The cover time becomes linear in the number of vertices on discrete tori and bounded-degree trees, of order $O(n \log \log n)$ on bounded-degree expanders, and of order $O(n(\log \log n)^2)$ on the Erdős–Rényi random graph in a certain sparsely connected regime.
- Papachristou and Kleinberg [19], “The Power of Choice in Random Sampling.” Introduces the *Choice Random Walk* (CRW), a generalization of Markov Chain Monte Carlo in which the sampler receives a set of k candidate moves and selects among them according to an optimizable policy. The formulation casts sampling as a convex optimization problem over transition policies, enabling systematic improvement of key metrics such as the average acceptance rate, the mixing time, and a differentiable relaxation of maximum congestion.
- Ashlagi et al. [20], “The Power of Two in Token Systems.” Applies the power-of-two-choices paradigm in an economic setting: in a marketplace where one agent requests service and a small set of d agents become available to provide it, the agent with the fewest tokens is selected to provide service. The token distribution is unstable when $d = 1$ but already stable when $d = 2$, with numerical simulations on kidney-exchange data confirming the prediction.

6 Sorting, Quicksort, and Monte Carlo Min-Cut

Textbook Roadmap

For a parallel treatment in the course textbooks, see:

- *Bucket Sort expected runtime $O(n)$ (balls into bins, $\mathbb{E}[X_i^2]$ calculation): [MU: Ch. 5, §5.2.2].*
- *Sorting lower bound $\Omega(n \log n)$ via the decision-tree argument: [Mor: Ch. 21, §21.3].*
- *Deterministic vs. randomized Quicksort, indicator-variable analysis $\mathbb{E}[X_{ij}] = 2/(j-i+1)$: [Mor: Ch. 21, §§21.3–21.4]; [MU: Ch. 2, §2.5].*
- *Monte Carlo vs. Las Vegas; randomized Min-Cut by edge contraction (probability $\geq 2/[n(n-1)]$, amplification, the $\binom{n}{2}$ corollary): [Mor: Ch. 22, §§21.2, 22.3]; [MU: Ch. 1, §1.5].*

Up to this point we used randomness to analyze quantities such as load, occupancy, and concentration. We now shift the role of randomness from the analysis to the algorithm itself. The next examples introduce randomized algorithms whose runtime or whose correctness probability is the random object of interest, and the same balls-and-bins style of reasoning that drove the previous chapters reappears here as the right tool for analyzing them.

6.1 Balls into Bins

We begin by analyzing the problem of throwing m balls into n bins. Let X_i be an indicator random variable defined as:

$$X_i = \begin{cases} 1 & \text{if collision at bin } i \\ 0 & \text{otherwise} \end{cases} \quad (52)$$

The total number of collisions is given by $X = \sum_{i=1}^n X_i$. By the linearity of expectation, we have:

$$\mathbb{E}[X] = \sum_{i=1}^n \mathbb{E}[X_i] \quad (53)$$

Since X_i is an indicator, $\mathbb{E}[X_i] = \mathbb{P}(\text{bin}_i \geq 2)$. We calculate this probability by subtracting the probabilities of having exactly 0 or 1 ball:

$$\mathbb{P}(\text{bin} \geq 2) = 1 - \mathbb{P}(\text{bin} = 0) - \mathbb{P}(\text{bin} = 1) \quad (54)$$

Assuming the number of balls in a given bin follows a binomial distribution, $\text{bin} \sim \text{Bin}(m, 1/n)$, we have:

$$\mathbb{P}(\text{bin} = 0) = \left(1 - \frac{1}{n}\right)^m \quad (55)$$

$$\mathbb{P}(\text{bin} = 1) = m \left(\frac{1}{n}\right) \left(1 - \frac{1}{n}\right)^{m-1} \quad (56)$$

Suppose $m = n$ and $n \rightarrow \infty$. We can approximate these probabilities using the definition of e . The probability of a collision in a specific bin converges to:

$$\mathbb{P}(\text{bin} \geq 2) \approx 1 - 2e^{-1} \approx 0.264 \text{ (or } 26.4\%) \quad (57)$$

Thus, the expected number of colliding bins is $\mathbb{E}[X] = n \cdot \mathbb{P}(\text{bin} \geq 2) \approx 0.264n$. This analysis will be applied directly when analyzing Bucket Sort.

6.2 Sorting Algorithms and Lower Bounds

6.2.1 The $\Omega(n \log n)$ Lower Bound

For any comparison-based sorting algorithm sorting an array of size n , a lower bound of $\Omega(n \log n)$ is required. A successful sorting algorithm must be able to generate any sequence through permutation. There are $n!$ possible permutations. Let T be the total number of comparisons in the worst case. Each comparison has two possible outcomes (less or greater), so the algorithm's decision process is a **binary tree** of depth T , which has at most 2^T leaves. Since the algorithm must distinguish all $n!$ permutations, each leaf corresponds to one permutation, giving:

$$2^T \geq n! \quad (58)$$

Using Stirling's approximation, $n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$, we take the logarithm of both sides:

Comparison Sort Lower Bound [Mor: §21.3]

$$T \geq \log(n!) \sim n \log n$$

Thus, the worst-case time complexity is bounded by $\Omega(n \log n)$. Note that this bound applies only to *comparison-based* algorithms; Bucket Sort, discussed next, is not comparison-based and can therefore achieve $O(n)$ expected time.

6.2.2 Bucket Sort

Consider n random draws from the interval $[0, 2^k)$. We can distribute these n elements (balls) into n bins (buckets). For instance, if $n = 2^m$ (e.g., $m = 7$), we can use k bits to determine placement into buckets.

The time complexity for this initial distribution phase is $O(n)$. This is because we only need to iterate through the array once and check the most significant bits of each element to assign it to the correct bucket. Let X_i be the number of elements in bucket i . To fully sort the array, we must sort the elements within each bucket (e.g., using a simple algorithm like Bubble Sort) and then concatenate them.

6.2.3 Expected Total Number of Comparisons in Bucket Sort

Let c be a constant factor representing the overhead of the sorting algorithm used within each bucket (e.g., cX_i^2 for Bubble Sort). The expected total number of comparisons is:

$$\mathbb{E} \left[\sum_{i=1}^n cX_i^2 \right] = c \sum_{i=1}^n \mathbb{E}[X_i^2] \quad (59)$$

Using the definition of variance, we know that $\mathbb{E}[X_i^2] = \text{Var}(X_i) + (\mathbb{E}[X_i])^2$. Since $X_i \sim \text{Bin}(n, 1/n)$:

$$\mathbb{E}[X_i] = n \left(\frac{1}{n} \right) = 1 \quad (60)$$

$$\text{Var}(X_i) = n \left(\frac{1}{n} \right) \left(1 - \frac{1}{n} \right) = 1 - \frac{1}{n} \quad (61)$$

Therefore, $\mathbb{E}[X_i^2] = (1 - 1/n) + 1^2 = 2 - 1/n$. Substituting this back into our expectation yields:

Bucket Sort Expected Runtime [MU: §5.2.2]

$$\mathbb{E} \left[\sum_{i=1}^n cX_i^2 \right] = cn \left(2 - \frac{1}{n} \right) = O(n)$$

This demonstrates that Bucket Sort operates in an expected $O(n)$ time when the inputs are uniformly distributed.

We now return to comparison-based algorithms and examine whether they can match the $\Omega(n \log n)$ lower bound.

6.3 Comparison-Based Sorting Algorithms

6.3.1 Merge Sort

Merge sort is a classic divide-and-conquer algorithm. The sorting process divides the array recursively, requiring:

- $n/2$ pairs of comparisons at the first layer.
- $n/4$ pairs of comparisons at the second layer.
- ...
- 1 pair at the last layer.

The number of comparisons satisfies the recurrence $T(n) = 2T(n/2) + O(n)$, which solves to $T(n) = O(n \log n)$ in all cases. To see why, consider the *recursion tree*: at each level of the tree, the total merge work across all subproblems is $O(n)$, because every element participates in exactly one merge per level. Since the array is halved at every step, the tree has $\log_2 n$ levels. Multiplying the per-level cost by the number of levels gives $O(n) \times \log_2 n = O(n \log n)$ total.

6.3.2 Quicksort

Quicksort is an efficient algorithm for sorting a list of n numbers. In Deterministic Quicksort, the first element in the list is designated as the pivot. All elements in the list are then compared with the pivot. Those elements less than the pivot are put into list L1, and those greater than the pivot are put into list L2. Quicksort is then recursively applied to these sublists.

Let $C(n)$ denote the total number of comparisons made on an input of size n . A worst-case scenario occurs when the input list is already sorted, meaning the pivot is always the smallest element in the list. In this bad input list, all elements end up in just one of the sublists, which is bad because the size of the problem shrinks too slowly. In the first step, we compare the pivot with $n - 1$ elements. We then end up with a sublist of length $n - 1$, which requires $C(n - 1)$ comparisons to sort. This yields the recurrence relation:

$$C(n) = (n - 1) + C(n - 1) \tag{62}$$

Since $C(1) = 0$, this means that $C(n) = O(n^2)$ on this worst-case input list.

To prevent this and achieve an optimal runtime on every input, we use Randomized Quicksort. This algorithm is identical to Deterministic Quicksort, except that the pivot position is chosen at random in each step. This randomness makes it impossible for an adversary to force a bad input list. Given any input list of n distinct elements, Randomized Quicksort will make $O(n \log n)$ comparisons in expectation. This expectation is taken over all sequences of random pivot positions.

Algorithm 1 Randomized Quicksort**Require:** Array $A[1..n]$ of n distinct elements; indices $\ell \leq r$

```

1: procedure RANDQSORT( $A, \ell, r$ )
2:   if  $\ell \geq r$  then return
3:   end if
4:   Choose pivot index  $p$  uniformly at random from  $\{\ell, \dots, r\}$ 
5:   Partition  $A[\ell..r]$  around  $A[p]$ ; let  $i$  be the final position of the pivot
6:   RANDQSORT( $A, \ell, i - 1$ )
7:   RANDQSORT( $A, i + 1, r$ )
8: end procedure

```

Expected Runtime Analysis of Randomized Quicksort To mathematically prove that the expected running time of Randomized Quicksort is $O(n \log n)$, we can analyze the total number of comparisons made.

Let the input list of n distinct elements be sorted such that $s_1 < s_2 < \dots < s_n$. For any $i < j$, we define an indicator random variable X_{ij} that takes the value 1 if s_i and s_j are ever compared during the sorting process, and 0 otherwise:

$$X_{ij} = \begin{cases} 1 & \text{if } s_i \text{ and } s_j \text{ are compared} \\ 0 & \text{otherwise} \end{cases} \quad (63)$$

Since any two elements are compared at most once, the total number of comparisons, X , is the sum of all X_{ij} for all possible pairs:

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} \quad (64)$$

By Linearity of Expectation, the expected number of total comparisons is:

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \mathbb{E}[X_{ij}] \quad (65)$$

The expected value $\mathbb{E}[X_{ij}]$ is exactly the probability that s_i and s_j get compared. This happens only if either s_i or s_j is chosen as a pivot before any other element between them in the sorted list. Since there are $j - i + 1$ elements in this range, and each has an equal chance of being chosen as the pivot first, we have:

$$\mathbb{E}[X_{ij}] = \frac{2}{j - i + 1} \quad (66)$$

Substituting this back into our expectation formula yields:

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j - i + 1} \quad (67)$$

To simplify this double summation, we can perform a change of variables by letting $k = j - i + 1$. The inner sum's index changes from j to k , where k ranges from 2 to $n - i + 1$:

$$\mathbb{E}[X] = \sum_{i=1}^{n-1} \sum_{k=2}^{n-i+1} \frac{2}{k} \quad (68)$$

We can change the order of summation to group the terms by k instead of i . If we sum over k from 2 to n , the index i will run from 1 up to $n - k + 1$:

$$\mathbb{E}[X] = \sum_{k=2}^n \sum_{i=1}^{n-k+1} \frac{2}{k} \quad (69)$$

Since the inner sum no longer depends on i , we simply multiply by the number of terms, which is $(n - k + 1)$:

$$\mathbb{E}[X] = \sum_{k=2}^n \frac{2}{k} \times (n - k + 1) \quad (70)$$

We can establish a strict upper bound by replacing $(n - k + 1)$ with n and starting the sum from $k = 1$:

$$\mathbb{E}[X] \leq \sum_{k=1}^n \frac{2}{k} \cdot n = 2n \sum_{k=1}^n \frac{1}{k} \quad (71)$$

Using the well-known harmonic series bound $\sum_{k=1}^n \frac{1}{k} < \ln n + 1$, we arrive at our final upper bound:

Expected Runtime of Randomized Quicksort [Mor: §21.4] [MU: §2.5]

$$\mathbb{E}[X] \leq 2n(\ln n + 1) \quad (72)$$

Thus, the expected runtime of Randomized Quicksort is $O(n \log n)$.

Tightening the analysis from an expectation bound to a high-probability tail bound on the runtime of Randomized Quicksort is the focus of the following exercise.

Problem 8: Bounding the Tail of Randomized Quicksort

From [Mor: Ex. 21.13].

Problem Statement: Use Chernoff bounds to show that, w.h.p. $\left(1 - \frac{1}{n}\right)$, Randomized Quicksort requires only $O(n \log n)$ comparisons to sort a list of length n . Here are some steps to help you:

- (a) Consider a particular run of Randomized Quicksort as shown in Figure 21.3. The tree shows the list L at each stage and then shows the sublists L_1 and L_2 under that, separated by the pivot, p . You can imagine drawing such a tree for any instance of Randomized Quicksort. Let T denote the total number of comparisons made by Randomized Quicksort. Explain why T is upper-bounded by the sum of the lengths of all root-to-leaf paths in the ternary tree. Note that pivots count as leaves as well, so every element is eventually a leaf.

[Hint: For each leaf, think about the number of comparisons that it's involved in.]

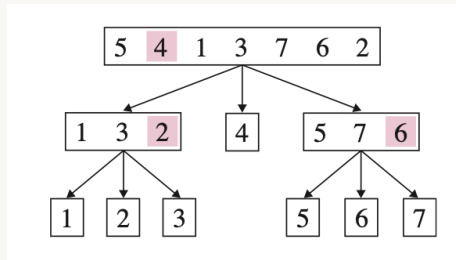


Figure 6: Randomized Quicksort tree. The randomly selected pivot is in pink.

- (b) Now we'll argue that w.h.p. each root-to-leaf path is of length $O(\log n)$. Note: It's fine that some quantities are not integers.
- (i) Let's say that a node of the tree is "good" if the randomly chosen pivot separates the current list at the node into two sublists, each of size at most $\frac{3}{4}$ the size of the current list. Otherwise we say that the node is "bad." What is the probability that a node is "good"?
 - (ii) Let g denote the maximum number of "good" nodes possible along a single root-to-leaf path. What is g as a function of n ?
 - (iii) Consider an arbitrary leaf i . We want to prove that the root-to-leaf path ending in i is not very long. Specifically, show that

$$\mathbb{P}\{\text{The root-to-leaf path ending in } i \text{ has length } \geq 6g\} \leq \frac{1}{n^2}$$

Here you're using the g from part (ii). Note that as soon as we see the first g "good" nodes, we'll be down to a single leaf.

- (c) We have seen that with probability at least $1 - \frac{1}{n^2}$ a given root-to-leaf path is no longer than $6g$. What probabilistic statement about T follows from this?

6.4 Monte Carlo Algorithms

Randomized Quicksort illustrates one face of randomness: an algorithm that always returns the correct answer, but whose runtime depends on the random pivot choices. We now turn to graph algorithms and meet a complementary face of randomness, where the runtime is fixed but the answer itself may occasionally be wrong. A *Monte Carlo algorithm* runs in bounded time but may return an incorrect answer with small probability; success probability is amplified by running the algorithm independently multiple times.

Monte Carlo vs. Las Vegas. Randomized algorithms generally fall into one of two families. *Las Vegas algorithms* always produce a *correct* answer; only their running time is random. Randomized Quicksort is a canonical example, since every pivot sequence ultimately yields a correctly sorted array. *Monte Carlo algorithms*, by contrast, have a *deterministic* time bound but may occasionally return a wrong answer. The min-cut algorithm below is of this type: it always finishes in polynomial time, but with some probability it may output a cut that is not minimal.

6.4.1 Min-Cut

What is a Min-Cut?

A cut-set of a graph is a set of edges whose removal breaks the graph into two or more connected components. A min-cut is simply a cut-set with the minimum possible number of edges (minimum cardinality).

This is the *global* min-cut: we minimize over **all** partitions $(S, V \setminus S)$ of the vertex set, with no designated source or sink. It differs from the (s,t) -min-cut familiar from network flows, where $s, t \in V$ are fixed and only partitions separating them are considered. The (s,t) -version is dual to maximum flow via the classical **max-flow / min-cut theorem**, which states that the value of a maximum s to t flow equals the capacity of a minimum s to t cut (see Theorem 26.6 of Cormen et al. [8] for a textbook statement and proof). This duality yields polynomial-time algorithms, and the global min-cut can in turn be reduced to $n - 1$ instances of the (s,t) -version, so it too is solvable deterministically in polynomial time. The reason we still study a randomized algorithm here is that the contraction algorithm below is dramatically simpler than any flow-based method, and its analysis showcases the probabilistic-method ideas central to this course.

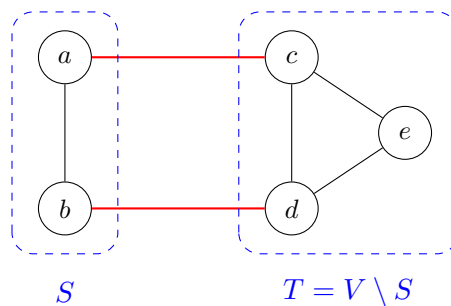


Figure 7: A cut (S, T) partitions the vertices into two sets. The **red edges** $\{\{a, c\}, \{b, d\}\}$ form the cut-set; removing them disconnects S from T . A *min-cut* minimizes the number of such crossing edges.

Randomized Min-Cut Algorithm

The randomized algorithm for finding a min-cut works through a process called *edge contraction*: two endpoints of a randomly chosen edge are merged into a single super-vertex, and this is repeated until only two super-vertices remain.

Algorithm 2 Contraction Algorithm for Min-Cut

Require: Connected multigraph $G = (V, E)$ with $|V| \geq 2$

Ensure: A cut-set (the edges between the two surviving super-vertices)

- 1: **while** $|V| > 2$ **do**
 - 2: Choose an edge $e = \{u, v\}$ uniformly at random from E
 - 3: Merge u and v into a single super-vertex w
 - 4: Remove all self-loops on w ; retain all parallel edges
 - 5: **end while**
 - 6: **return** the set of remaining edges
-

Probability of Finding a Min-Cut

Let C be a specific min-cut of the original graph, and let its size be $|C| = k$. We want to find the probability that C survives the entire contraction process (i.e., no edge of C is ever contracted).

If the min-cut size is k , then every vertex in the graph must have a degree of at least k . Therefore, a graph with n vertices must have at least $\frac{nk}{2}$ edges.

When we pick an edge uniformly at random in the first round, the probability that we do *not* pick an edge from C is:

$$\mathbb{P}(\text{no edge of } C \text{ selected in 1st round}) \geq \frac{\frac{nk}{2} - k}{\frac{nk}{2}} = \frac{n-2}{n} \quad (73)$$

Assuming C survives the first round, the contracted graph now has $n-1$ vertices. It still has a min-cut of size k , so it has at least $\frac{(n-1)k}{2}$ edges. The probability of C surviving the second round is:

$$\mathbb{P}(\text{no edge of } C \text{ selected in 2nd round}) \geq \frac{\frac{(n-1)k}{2} - k}{\frac{(n-1)k}{2}} = \frac{n-3}{n-1} \quad (74)$$

If we generalize this and multiply the probabilities across all $n-2$ iterations needed to reduce the graph to 2 vertices, we get:

Min-Cut Survival Probability [Mor: §22.3] [MU: §1.5]

$$\mathbb{P}(C \text{ survives}) \geq \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdots \frac{3}{5} \cdot \frac{2}{4} \cdot \frac{1}{3} = \frac{2}{n(n-1)} \quad (75)$$

Remark 17 (A combinatorial corollary [MU: §1.5]). The bound (75) silently delivers a structural fact that has nothing to do with the algorithm itself. Each distinct min-cut C of the graph is the algorithm's output with probability at least $2/[n(n-1)]$, and the events “ C is the output” for distinct min-cuts C are disjoint. Their probabilities therefore sum to at most 1:

$$(\text{number of distinct min-cuts}) \cdot \frac{2}{n(n-1)} \leq 1 \implies (\text{number of distinct min-cuts}) \leq \binom{n}{2}.$$

That is, *any* graph on n vertices has at most $\binom{n}{2}$ minimum cuts. The randomized algorithm doubles as a counting argument.

Amplification of Success Probability

A probability of $\frac{2}{n(n-1)}$ is quite small. To reduce the probability of error, we can run this Monte Carlo algorithm multiple times and return the smallest cut-set produced.

Let x be the number of times we run the algorithm. If we choose $x = n(n-1) \ln n$, the probability that the min-cut fails to survive in *all* x independent runs is bounded by:

$$\mathbb{P}(\text{failure}) \leq \left(1 - \frac{2}{n(n-1)}\right)^{n(n-1) \ln n} \quad (76)$$

Using the standard approximation $(1-z) < e^{-z}$ for small z , this simplifies nicely:

Amplification Result [Mor: §22.3]

$$\left[\left(1 - \frac{2}{n(n-1)}\right)^{n(n-1)} \right]^{\ln n} < (e^{-2})^{\ln n} = e^{-2 \ln n} = \frac{1}{n^2} \quad (77)$$

Thus, by running the algorithm $n(n-1) \ln n$ times, the probability of failure drops to $\frac{1}{n^2}$,

meaning we find the min-cut with high probability.

Further Reading

This section introduced the Las Vegas / Monte Carlo dichotomy through Randomized Quicksort and the contraction-based Min-Cut algorithm. The textbook below treats both topics, and indeed almost every theme covered in these notes, at greater length, and serves as a general reference for the course.

- Motwani and Raghavan [28], *Randomized Algorithms*. The standard reference text on the area, organized into two parts. Part I (*Tools and Techniques*) develops the core machinery: game-theoretic techniques (Ch. 2), moments and deviations (Ch. 3, including occupancy problems, randomized selection, and the coupon collector), tail inequalities (Ch. 4, including the Chernoff bound and martingales), the probabilistic method (Ch. 5, including MAX-SAT and the Lovász Local Lemma), Markov chains and random walks (Ch. 6, including the 2-SAT example and expanders), and algebraic techniques (Ch. 7, including verifying polynomial identities). Part II (*Applications*) treats data structures, geometric algorithms, graph algorithms (including a deeper treatment of the Min-Cut problem in Ch. 10.2), approximate counting (Ch. 11, including the DNF counting problem), parallel and distributed algorithms, online algorithms, and number theory. Chapter 1 introduces the Min-Cut contraction algorithm (§1.1) and the Las Vegas / Monte Carlo distinction (§1.2) in essentially the same form used in this section.

7 Randomized Selection and Polynomial Identity Testing

Textbook Roadmap

For a parallel treatment in the course textbooks, see:

- *k-Select / Randomized Median-Select* with $\mathbb{E}[C(n)] \leq 4n$: [Mor: Ch. 21, §21.5].
- *Approximate median via sampling* ($n^{3/4}$ sample, failure events E_1, E_2, E_3 , Chebyshev bounds, failure prob. $\leq 1/n^{1/4}$): [MU: Ch. 3, §§3.5–3.5.2].
- *P, NP, NP-hard framing for randomized algorithms*: [Mor: Ch. 22, §22.1 (intro)]; the formal complexity definitions are standard CS theory background, not detailed in any of the three books.
- *Randomized polynomial identity testing* (roots vs. coefficients form, sample from $\{0, \dots, 100d\}$, error $< 1/100$): [Mor: Ch. 22, §22.2]; [MU: Ch. 1, §1.1] (Schwartz–Zippel-style argument).

7.1 The k -Select Problem

Randomized Quicksort, which we analyzed above, sorts a list of n elements in $2n \ln n$ comparisons in expectation. Now let us compare this with a related but different problem: finding the k -th largest (or k -th smallest) element of an unsorted list.

Note that the k -largest element means the element with rank k from the top. For example, the 1-largest is the maximum, and the n -largest is the minimum. Equivalently, the k -largest is the $(n - k + 1)$ -th smallest.

One obvious approach is to sort the entire list using Quicksort and then pick the k -th element. This costs $O(n \log n)$. But sorting the entire list is wasteful if we only need a single element.

The key idea is to use a pivot, just as in Quicksort, but only recurse on the side that contains our target element. Consider the sorted order of the elements on a number line. When we pick a

pivot s_i , it splits the list into two sublists: L_1 (elements smaller than s_i) and L_2 (elements larger than s_i). Since we know which side our target lies on, we can **discard** the other side entirely.

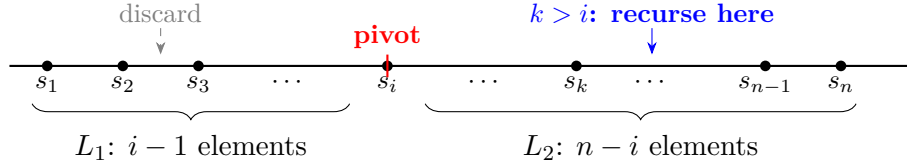


Figure 8: The pivot s_i partitions the sorted number line into L_1 and L_2 . If the target rank $k > i$, we recurse only on L_2 and discard L_1 . If $k < i$, the roles are reversed.

This idea leads directly to the following recursive algorithm.

Algorithm 3 Randomized k -Select

Require: Array $A[\ell..r]$; target rank k with $\ell \leq k \leq r$

Ensure: The element of rank k in the sorted order of A

```

1: procedure KSELECT( $A, \ell, r, k$ )
2:   if  $\ell = r$  then return  $A[\ell]$ 
3:   end if
4:   Choose pivot index  $p$  uniformly at random from  $\{\ell, \dots, r\}$ 
5:   Partition  $A[\ell..r]$  around  $A[p]$ ; let  $i$  be the final position of the pivot
6:   if  $k = i$  then return  $A[i]$ 
7:   else if  $k < i$  then return KSELECT( $A, \ell, i - 1, k$ )
8:   elsereturn KSELECT( $A, i + 1, r, k$ )
9:   end if
10: end procedure

```

Suppose we pick a pivot that happens to split the list roughly in half. After one round of $n - 1$ comparisons, we only need to search in a sublist of size $\approx n/2$. The next round costs $\approx n/2$ comparisons and reduces the problem to size $\approx n/4$, and so on. The total number of comparisons is then:

$$C(n) = (n - 1) + C(n/2) < n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots \leq 2n \quad (78)$$

This is a **geometric series** that sums to at most $2n$, giving us $O(n)$ comparisons, a dramatic improvement over the $2n \ln n$ of Quicksort.

Remark 18. Why does Quicksort require $O(n \log n)$ while k -Select requires only $O(n)$? The difference comes from how many sublists we process at each step:

- **Quicksort** must recurse on **both** sublists L_1 and L_2 . At every level of the recursion tree, the total work across all subproblems remains $O(n)$. Since there are $O(\log n)$ levels, the total is $O(n \log n)$.
- **k -Select** only recurses on **one** sublist. The problem size shrinks geometrically: $n \rightarrow n/2 \rightarrow n/4 \rightarrow \dots$, so the total work is the geometric series $n + n/2 + n/4 + \dots \leq 2n = O(n)$.

Processing only one side is what turns the harmonic-type sum into a geometric sum.

Figure 9 illustrates this contrast visually.

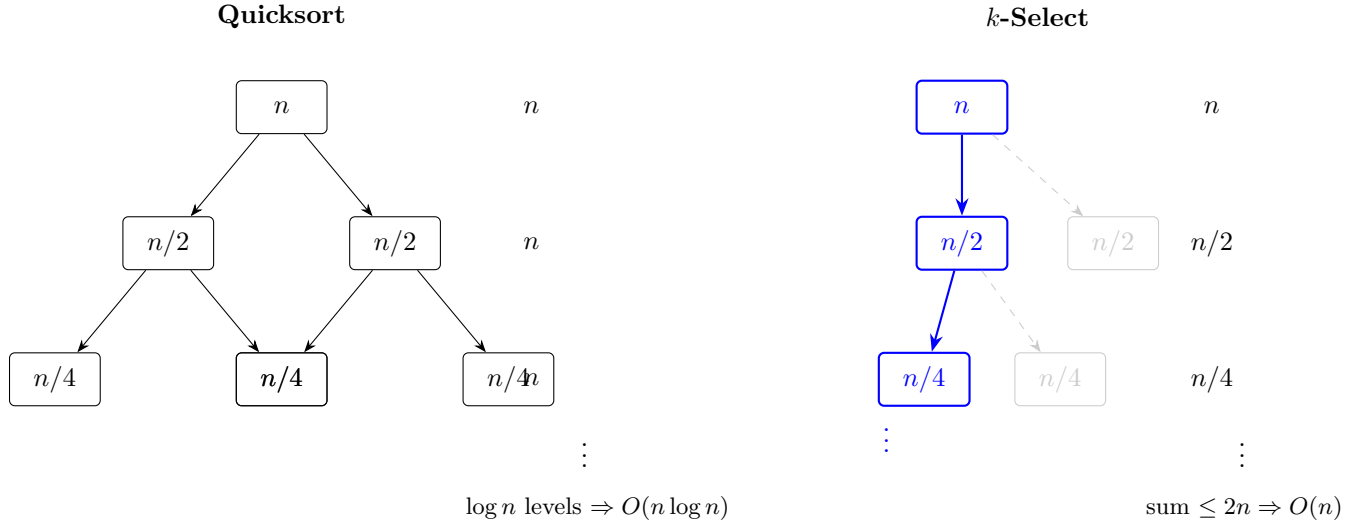


Figure 9: **Left:** Quickselect processes both sublists at each level; total work per level is $O(n)$, and there are $O(\log n)$ levels. **Right:** k -Select processes only one sublist (blue); the grayed-out sublists are discarded. The work per level shrinks geometrically.

7.2 Randomized Median-Select

We now focus on a specific instance of k -Select: finding the median, i.e., $k = \lceil n/2 \rceil$. We present a randomized algorithm (Randomized Median-Select) that finds the median in expected $O(n)$ time using random pivots, just as in Randomized Quickselect.

7.2.1 Expected Complexity

Let $C(n)$ denote the number of comparisons on a list of size n . The pivot is chosen uniformly at random; suppose it turns out to be the i -th smallest element. We need $n - 1$ comparisons to partition, and then we recurse on whichever sublist contains the median. This gives:

$$\mathbb{E}[C(n)] = (n - 1) + \frac{1}{n} \sum_{i=1}^n \mathbb{E}[C(\text{sublist size when pivot is } s_i)] \quad (79)$$

Why can we upper-bound this by summing only over the larger sublist sizes? For any pivot s_i , the sublist we recurse on has size $\max\{i - 1, n - i\}$ in the worst case. Since $\max\{i - 1, n - i\} \geq \lfloor n/2 \rfloor$ always, and each size from $\lfloor n/2 \rfloor$ to $n - 1$ is hit by exactly two choices of i (one from each side of the list), we obtain:

$$\mathbb{E}[C(n)] \leq (n - 1) + \frac{2}{n} \sum_{i=\lfloor n/2 \rfloor}^{n-1} \mathbb{E}[C(i)] \quad (80)$$

7.2.2 Proof that $\mathbb{E}[C(n)] \leq 4n$

We want to show $\mathbb{E}[C(n)] \leq a \cdot n$ for some constant a , and then determine the smallest a that makes the induction go through. The value $a = 4$ will emerge naturally from the algebra.

Inductive hypothesis: Assume $\mathbb{E}[C(i)] \leq a \cdot i$ for all $i \leq n - 1$.

Substituting into the recurrence:

$$\begin{aligned}\mathbb{E}[C(n)] &\leq (n-1) + \frac{2}{n} \sum_{i=\lfloor n/2 \rfloor}^{n-1} a \cdot i \\ &= (n-1) + \frac{2a}{n} \cdot \frac{(n-1 - \lfloor n/2 \rfloor + 1)((n-1) + \lfloor n/2 \rfloor)}{2}\end{aligned}$$

Ignoring floors for clarity, the sum of consecutive integers evaluates to:

$$\sum_{i=n/2}^{n-1} i = \frac{n/2 \cdot (3n/2 - 1)}{2} \approx \frac{3n^2}{8}$$

Substituting:

$$\mathbb{E}[C(n)] \leq (n-1) + \frac{2a}{n} \cdot \frac{3n^2}{8} = (n-1) + \frac{3}{4}an$$

For the induction to close, we need $(n-1) + \frac{3}{4}an \leq an$, which simplifies to $(n-1) \leq \frac{1}{4}an$, i.e., $a \geq 4\left(1 - \frac{1}{n}\right)$. The smallest integer satisfying this for all n is $a = 4$. Setting $a = 4$:

$$\mathbb{E}[C(n)] \leq (n-1) + 3n = 4n - 1 \leq 4n$$

Randomized Median-Select [Mor: §21.5]

For any list of n distinct elements, Randomized Median-Select finds the median in at most $4n$ expected comparisons. More generally, Randomized k -Select finds any order statistic in $O(n)$ expected time.

Remark 19. Recall that Randomized Median-Select is a **Las Vegas algorithm**: it always returns the correct answer, but its runtime varies depending on the random pivot choices. As shown in Exercise 21.13 [3], the tail of the runtime is well-behaved: $\mathbb{P}(T > cn \ln n) < e^{-c}$, which follows from analyzing the recursion tree where each “good” pivot (splitting at most $3/4$ on each side) reduces the problem by a factor of $3/4$. The maximum number of good nodes on any root-to-leaf path is $g = \log_{4/3} n$, and Chernoff bounds show the path length is $O(\log n)$ with high probability.

7.3 Approximate Median via Sampling

The Randomized Median-Select algorithm above uses $O(n)$ expected comparisons but has variable runtime. We now consider a different approach: a **sampling-based** algorithm that finds the median with high probability using a small random sample. This approach is closer in spirit to a *Monte Carlo algorithm*: it runs in a predictable amount of time but may occasionally fail.

Remark 20 (Las Vegas vs. Monte Carlo). A **Las Vegas algorithm** always returns the correct answer; only its runtime varies. A **Monte Carlo algorithm** runs in a fixed (deterministic) amount of time but may return an incorrect answer with some small probability. If we run a Monte Carlo algorithm repeatedly until it succeeds, we obtain a Las Vegas algorithm.

Algorithm 4 Approximate Median via Sampling**Require:** Array S of n distinct elements**Ensure:** The median of S , or FAIL

```

1: Draw a sample  $R$  of  $n^{3/4}$  elements from  $S$  uniformly at random with replacement
2: Sort  $R$ 
3:  $d \leftarrow R[\frac{1}{2}n^{3/4} - \sqrt{n}]$  ▷ lower sample pivot
4:  $u \leftarrow R[\frac{1}{2}n^{3/4} + \sqrt{n}]$  ▷ upper sample pivot
5:  $\ell_d \leftarrow |\{s \in S \mid s < d\}|$ ,  $\ell_u \leftarrow |\{s \in S \mid s > u\}|$ 
6: if  $\ell_d > n/2$  or  $\ell_u > n/2$  then
7:   return FAIL ▷ event  $E_2$  or  $E_1$ : median lies outside  $[d, u]$ 
8: end if
9:  $C \leftarrow \{s \in S \mid d \leq s \leq u\}$ 
10: if  $|C| > 4n^{3/4}$  then
11:   return FAIL ▷ event  $E_3$ : candidate set too large to sort cheaply
12: end if
13: Sort  $C$ 
14: return the element at position  $\lceil n/2 \rceil - \ell_d$  in sorted  $C$ 

```

7.3.1 The Sampling AlgorithmLet $S = \{s_1, s_2, \dots, s_n\}$ be the original unsorted array.

The five steps in detail:

1. **Sample:** Draw $R = n^{3/4}$ elements uniformly at random (with replacement) from S . Call this sample $r_1, r_2, \dots, r_{n^{3/4}}$.
2. **Sort the sample:** Sort R to obtain the sorted sample.
3. **Identify bounds:** In the sorted sample, let:
 - d = the element at rank $\frac{1}{2}n^{3/4} - \sqrt{n}$ (“down”)
 - u = the element at rank $\frac{1}{2}n^{3/4} + \sqrt{n}$ (“up”)

These are $\sqrt{n}$ positions below and above the sample median.

4. **Collect candidates:** Define $C = \{s \in S \mid d \leq s \leq u\}$.
5. **Sort C and extract the median:** Sort C . The median of S is the element at position $\lceil n/2 \rceil - \ell_d$ within the sorted C (1-indexed), where ℓ_d is the number of elements of S that are strictly less than d .

The algorithm succeeds if the true median of S lies within C and C is small enough to sort cheaply. We define three failure events.

7.3.2 Failure Events

- E_1 (**Failure 1**): $\ell_u > n/2$, i.e., too many elements of S lie above u . This means u is below the true median, so the median is not in C .
- E_2 (**Failure 2**): $\ell_d > n/2$, i.e., too many elements of S lie below d . This means d is above the true median, so the median is not in C .

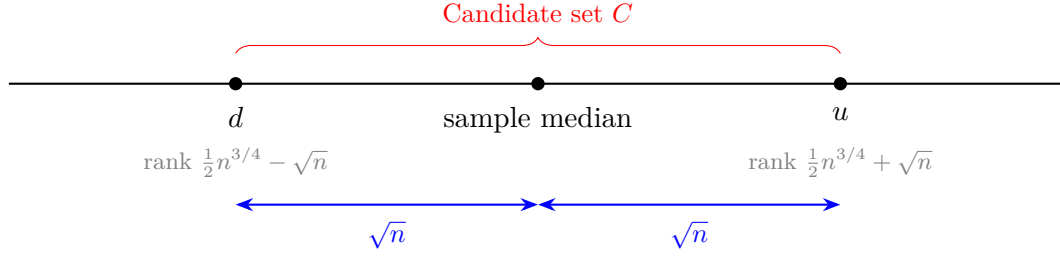


Figure 10: Ranks in the sorted sample of size $n^{3/4}$. The bounds d and u are placed $\sqrt{n}$ positions below and above the sample median. All elements of S falling between d and u form the candidate set C .

- E_3 (**Failure 3**): $|C| > 4n^{3/4}$. Even if the median is in C , sorting C would be too expensive.

If none of these events occur, the algorithm correctly finds the median.

7.3.3 Bounding the Failure Probability

Bounding $\mathbb{P}(E_1)$. Define indicator random variables for each sample element:

$$X_i = \begin{cases} 1 & \text{if the } i\text{-th sample element is below the true median} \\ 0 & \text{otherwise} \end{cases}$$

Since each sample element is drawn uniformly from S , we have $\mathbb{P}(X_i = 1) = 1/2$. Let $X = \sum_{i=1}^{n^{3/4}} X_i$, so $X \sim \text{Bin}(n^{3/4}, 1/2)$ with $\mathbb{E}[X] = n^{3/4}/2$.

Recall that u is the element at sample-rank $\frac{1}{2}n^{3/4} + \sqrt{n}$. The event E_1 (u lies below the true median) is equivalent to: at least $\frac{1}{2}n^{3/4} + \sqrt{n}$ of the sampled elements lie below the true median (because if so, the first $\frac{1}{2}n^{3/4} + \sqrt{n}$ sample-ranks are all below the true median, which forces u below the median as well). Therefore:

$$\begin{aligned} \mathbb{P}(E_1) &= \mathbb{P}\left(X > \frac{1}{2}n^{3/4} + \sqrt{n}\right) \\ &\leq \mathbb{P}(|X - \mathbb{E}[X]| > \sqrt{n}) \\ &\leq \frac{\text{Var}(X)}{n} = \frac{n^{3/4} \cdot \frac{1}{4}}{n} = \frac{1}{4n^{1/4}} \end{aligned}$$

where the last step uses Chebyshev's inequality.

Bounding $\mathbb{P}(E_2)$. By a symmetric argument, $\mathbb{P}(E_2) \leq \frac{1}{4n^{1/4}}$.

Bounding $\mathbb{P}(E_3)$. We decompose E_3 into two sub-events:

- E'_3 : at least $2n^{3/4}$ elements of S lie above the true median but below u .
- E''_3 : at least $2n^{3/4}$ elements of S lie below the true median but above d .

If both E'_3 and E''_3 fail, then $|C| \leq 2 \cdot 2n^{3/4} = 4n^{3/4}$, so E_3 does not occur. Hence $\mathbb{P}(E_3) \leq \mathbb{P}(E'_3) + \mathbb{P}(E''_3)$.

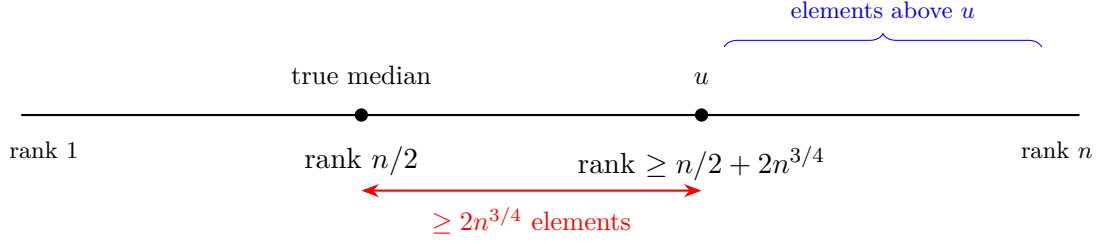


Figure 11: Analysis of E'_3 : if u is at rank $\geq n/2 + 2n^{3/4}$ in S , then at least $2n^{3/4}$ elements of S lie between the true median and u , making $|C|$ large.

We bound $\mathbb{P}(E'_3)$. For E'_3 to occur, u must be at rank at least $n/2 + 2n^{3/4}$ in the original array S . The sorted-sample positions *at or above* u are positions $\frac{1}{2}n^{3/4} + \sqrt{n}, \dots, n^{3/4}$, of which there are $\frac{1}{2}n^{3/4} - \sqrt{n} + 1$. The event E'_3 holds iff every one of these top sample positions is filled by an element of S -rank $\geq n/2 + 2n^{3/4}$, equivalently, at least $\frac{1}{2}n^{3/4} - \sqrt{n} + 1$ of the sampled elements have S -rank $\geq n/2 + 2n^{3/4}$. Define:

$$X_i = \begin{cases} 1 & \text{if the } i\text{-th sample element has rank } \geq n/2 + 2n^{3/4} \text{ in } S \\ 0 & \text{otherwise} \end{cases}$$

Then $\mathbb{P}(X_i = 1) = \frac{n/2 - 2n^{3/4}}{n} = \frac{1}{2} - \frac{2}{n^{1/4}}$, and $X = \sum_{i=1}^{n^{3/4}} X_i$ follows:

$$X \sim \text{Bin}\left(n^{3/4}, \frac{1}{2} - \frac{2}{n^{1/4}}\right), \quad \mathbb{E}[X] = n^{3/4} \left(\frac{1}{2} - \frac{2}{n^{1/4}}\right) = \frac{1}{2}n^{3/4} - 2\sqrt{n}$$

For E'_3 to occur, we need $X > \frac{1}{2}n^{3/4} - \sqrt{n} = \mathbb{E}[X] + \sqrt{n}$. By Chebyshev:

$$\mathbb{P}(E'_3) \leq \mathbb{P}(|X - \mathbb{E}[X]| > \sqrt{n}) \leq \frac{\text{Var}(X)}{n} \leq \frac{n^{3/4}/4}{n} = \frac{1}{4n^{1/4}}$$

By a symmetric argument, $\mathbb{P}(E''_3) \leq \frac{1}{4n^{1/4}}$.

7.3.4 Total Failure Probability

Combining by the union bound:

$$\mathbb{P}(\text{Failure}) \leq \mathbb{P}(E_1) + \mathbb{P}(E_2) + \mathbb{P}(E'_3) + \mathbb{P}(E''_3) \leq \frac{4}{4n^{1/4}} = \frac{1}{n^{1/4}}$$

Approximate Median via Sampling [MU: §§3.5–3.5.2]

$$\mathbb{P}(\text{Failure}) \leq \frac{1}{n^{1/4}}$$

By sampling $n^{3/4}$ elements, sorting the sample, and collecting the candidate set C , the algorithm finds the exact median with probability at least $1 - 1/n^{1/4}$.

7.3.5 Computational Complexity

The total cost of the algorithm (when it does not fail) consists of:

1. **Sorting R :** The sample has $n^{3/4}$ elements. Using a comparison-based sort (e.g., Quicksort), this costs $O(n^{3/4} \log n^{3/4}) = O(n^{3/4} \log n)$. Using a quadratic sorting algorithm (e.g., Bubble Sort), this would cost $O(n^{3/2})$.
2. **Collecting C :** A single pass through S to check which elements lie between d and u : $O(n)$.
3. **Sorting C :** Since $|C| \leq 4n^{3/4}$ (no failure), this costs $O(n^{3/4} \log n)$.

The dominant cost is the $O(n)$ scan, giving an overall $O(n)$ runtime. A natural question is: *can we improve the sample size $n^{3/4}$?* That is, can we use a smaller sample and still guarantee a low failure probability? This is explored in the homework below.

Remark 21. If the sampling algorithm fails, we can simply run it again. Since $\mathbb{P}(\text{Failure}) \leq 1/n^{1/4} \rightarrow 0$, the expected number of trials until success is $1/(1 - \mathbb{P}(\text{Failure})) \approx 1$. Running the Monte Carlo algorithm repeatedly until no failure occurs yields a Las Vegas algorithm with expected runtime $O(n)$.

Reducing the sample size in this approximate-median routine while preserving the failure-probability guarantee is the subject of the following exercise.

Problem 9: Improving the Sample Size

Instructor-supplied.

Problem Statement: In the approximate median via sampling algorithm above, we used a sample of size $R = n^{3/4}$. Can the sample size be made smaller while still guaranteeing $\mathbb{P}(\text{Failure}) \rightarrow 0$ as $n \rightarrow \infty$?

Determine how small R can be (as a function of n) so that the failure probability still vanishes. Specifically, re-derive the bounds on $\mathbb{P}(E_1)$, $\mathbb{P}(E_2)$, and $\mathbb{P}(E_3)$ for a general sample size R , and find the smallest R (up to constant factors) for which $\mathbb{P}(\text{Failure}) \rightarrow 0$.

7.4 Computational Complexity: P, NP, and NP-Hard

The Approximate Median algorithm trades exactness for speed, accepting a small probability of failure in exchange for a fixed running time. To understand when such trade-offs become unavoidable, and what role randomness plays once a deterministic polynomial-time algorithm is out of reach, we briefly recall the framework of computational complexity. This will set the stage for a new class of randomized algorithms.

7.4.1 Min-Cut vs. Max-Cut

Consider a graph $G = (V, E)$. Two natural optimization problems arise:

- **Min-Cut:** Find the minimum number of edges whose removal disconnects the graph. This problem is solvable in **polynomial time** (e.g., via max-flow algorithms).

- **Max-Cut:** Partition the vertices into exactly two groups so as to **maximize** the number of edges between the groups. This problem is **NP-hard**.

The contrast is striking: two problems that sound similar, both about cutting edges in a graph, yet one is efficiently solvable and the other is believed to be intractable. This motivates a closer look at the complexity classes P, NP, and NP-hard.

7.4.2 NP and Certificates

A key distinction in computational complexity is between **solving** a problem and **verifying** a proposed solution. *Verification* means checking whether a given candidate answer is correct, whereas *optimization* (or computation) means finding the answer from scratch. This gap is precisely what separates the complexity classes: a problem is in **P** if it can be *solved* in polynomial time, while it is in **NP** if a proposed solution can be *verified* in polynomial time. The famous $P \stackrel{?}{=} NP$ question asks whether every problem whose solutions are easy to check is also easy to solve.

More precisely, a decision problem (one with a yes/no answer) belongs to the class **NP** if every “yes” instance has a **certificate** (also called a *witness*) that can be *verified* in polynomial time.

Example 22 (Hamiltonian Cycle). Does a graph G contain a cycle that visits every vertex exactly once? If the answer is “yes,” the cycle itself serves as a certificate: one can verify in polynomial time that it is indeed a valid Hamiltonian cycle.

Example 23 (Independent Set). An **independent set** in a graph is a set of vertices with no edges between them. Finding the maximum independent set is NP-hard. However, the *decision* version, namely “does G have an independent set of size $\geq k$?”, is in NP. A “yes” answer has a certificate (the set of k vertices), and checking that no two are adjacent takes polynomial time.

Remark 24. Whether a problem is in NP or is NP-hard can depend on precisely what question is being asked. The *optimization* version (“find the largest independent set”) may be NP-hard, while the *decision* version (“does an independent set of size $\geq k$ exist?”) is in NP. An NP-hard problem is one that is “at least as hard” as every problem in NP; if any NP-hard problem could be solved in polynomial time, then **every** problem in NP could be, which would imply $P = NP$.

7.5 Randomized Polynomial Identity Testing

The gap between *computing* a solution and *verifying* one is at the heart of the P vs. NP question. We now see a concrete example of this gap, and how randomization can exploit it.

7.5.1 The Problem

Suppose we are given two representations of a degree- d polynomial:

- **Roots form:** $b_1, b_2, \dots, b_d$, defining $F(x) = (x - b_1)(x - b_2) \cdots (x - b_d)$.
- **Coefficients form:** $a_0, a_1, \dots, a_d$, defining $G(x) = \sum_{i=0}^d a_i x^i$.

The question is: *is* $F(x) = G(x)$? That is, does expanding the product of the roots yield exactly the given coefficients?

7.5.2 Deterministic Approach: $O(d^2)$

The straightforward method is to expand $F(x)$ by multiplying one factor at a time. Starting from $(x - b_1)$, each multiplication by a new factor $(x - b_{k+1})$ requires $O(k)$ operations. The total cost is:

$$1 + 2 + \cdots + d = \frac{d(d+1)}{2} = O(d^2)$$

After expanding, we compare the resulting coefficients with $a_0, \dots, a_d$. The entire process takes $O(d^2)$ multiplications.

7.5.3 Verification at a Single Point: $O(d)$

In contrast, **evaluating** both $F(r)$ and $G(r)$ at a specific value r takes only $O(d)$ operations:

- $F(r) = (r - b_1)(r - b_2) \cdots (r - b_d)$: d multiplications.
- $G(r) = \sum_{i=0}^d a_i r^i$: d multiplications (via Horner's method).

This is a dramatic gap: *computing* whether $F = G$ deterministically costs $O(d^2)$, but *checking* the identity at a single point costs only $O(d)$. This mirrors the P vs. NP phenomenon: verification can be fundamentally cheaper than computation.

7.5.4 A Randomized Checker

We exploit this gap with a Monte Carlo algorithm.

Algorithm 5 Randomized Polynomial Identity Tester

Require: Roots $b_1, \dots, b_d$ and coefficients $a_0, \dots, a_d$

Ensure: “equal” or “not equal”; false-positive probability $< 1/100$

1: Pick r uniformly at random from $\{0, 1, \dots, 100d\}$

2: Compute $F(r) = \prod_{i=1}^d (r - b_i)$

3: Compute $G(r) = \sum_{i=0}^d a_i r^i$

4: **if** $F(r) = G(r)$ **then return** “equal”

5: **elsereturn** “not equal”

6: **end if**

Correctness analysis. If $F \equiv G$ (they are the same polynomial), then $F(r) = G(r)$ for every r , and the checker always outputs “equal”. There is no error in this case.

If $F \not\equiv G$, define $H(x) = F(x) - G(x)$. Since $F \neq G$, H is a nonzero polynomial of degree at most d . By the **Fundamental Theorem of Algebra**, H has at most d roots. Since r is chosen uniformly from a set of $100d + 1$ values:

$$\mathbb{P}(\text{false “equal”}) = \mathbb{P}(H(r) = 0) \leq \frac{d}{100d + 1} < \frac{1}{100}$$

Randomized Polynomial Checker [Mor: §22.2] [MU: §1.1]

Given roots $(b_1, \dots, b_d)$ and coefficients $(a_0, \dots, a_d)$, the randomized checker verifies whether $(x - b_1) \cdots (x - b_d) = \sum a_i x^i$ in $O(d)$ time with error probability less than $1/100$. This is a **Monte Carlo algorithm** with one-sided error: if it says “not equal,” it is always correct; errors can only occur when it says “equal.”

Remark 25. The error probability $1/100$ can be made arbitrarily small by repeating the check with independent random values of r . Running k independent checks reduces the error probability to $(1/100)^k$, while the total cost remains $O(kd)$, still far below the $O(d^2)$ deterministic approach.

Further Reading

The randomized polynomial-identity checker analysed above used a single random evaluation point r drawn uniformly from $\{0, 1, \dots, 100d\}$ and bounded its error probability by $1/100$ via the Fundamental Theorem of Algebra (a nonzero polynomial of degree at most d has at most d roots in a set of size $100d + 1$). The remark above observed that k independent checks reduce the error to $(1/100)^k$. A more careful look at *how* those k checks should be carried out, and the trade-offs they imply, is given in:

- [MU: Ch. 1, pp. 3–5 and p. 7] (within §1.2 *Axioms of Probability*). Uses the same polynomial-identity checker as the running example of the chapter, and separates the analysis of repeated trials into two regimes:
 - *Sampling with replacement.* Each iteration draws r uniformly from $\{1, \dots, 100d\}$ independently of previous draws. By Definition 1.3 (independence), the probability of a wrong answer after k iterations is $\prod_{i=1}^k \mathbb{P}(E_i) \leq (d/100d)^k = (1/100)^k$.
 - *Sampling without replacement.* Once r has been chosen it cannot be reused. Using the definition of conditional probability, the probability of a wrong answer after $k \leq d$ iterations is bounded by $\prod_{j=1}^k \frac{d-(j-1)}{100d-(j-1)} \leq (1/100)^k$, and is strictly tighter than the with-replacement bound whenever $j > 1$.

Two observations from page 7 are worth pulling out. First, $d + 1$ samples *without* replacement deterministically guarantee finding a root of $F(x) - G(x)$ when the polynomials disagree; but the resulting $\Theta(d^2)$ cost matches the canonical-form approach and so buys nothing over a deterministic algorithm. Second, despite its slightly looser bound, sampling with replacement is often the analysis of choice because the independence of the events E_i makes the algebra significantly easier. The same chapter (Lemma 1.2 on page 4) also states the *union bound* that recurs throughout these notes.

8 Min-Cut Amplification, Max-Cut, and Submodular Greedy

Textbook Roadmap

For a parallel treatment in the course textbooks, see:

- *Min-Cut amplification by independent repetitions* ($\ell = n(n-1) \ln n$ runs $\Rightarrow$ failure $\leq 1/n^2$): [Mor: Ch. 22, §22.3]; [MU: Ch. 1, §1.5]. (The two-phase contraction strategy is an instructor extension; the optimal k^* is the content of **Problem 10**.)
- *Maximum Cut — coin-flip* $\mathbb{E}[C(A, B)] = m/2$ and probability splitting $p \geq 2/(m+2)$: [MU: Ch. 6, §6.2.1, Thm. 6.3]; [AS: Ch. 2, §2.1 (Basics)].
- *Derandomization* $\rightarrow$ greedy Max-Cut (place each vertex on the side with fewer current neigh-

bors): [MU: Ch. 6, §6.3].

- *Local search improvement for Max-Cut (iterate single-vertex flips until no flip improves the cut, $m/2$ -guarantee at any local optimum)*: not covered in any of the three textbooks; included as an instructor extension. The $m/2$ -bound at termination follows from the cut value acting as a monovariant under improving flips, and the local-search style of analysis for combinatorial optimization is classically associated with Sahni & Gonzalez (1976).
- *Greedy $(1 - 1/e)$ -approximation for monotone submodular maximization*: not covered in any of the three textbooks. The classical result is Nemhauser, Wolsey & Fisher (1978); supporting tools (linearity of expectation, union bounds) are from [MU: Ch. 6].

8.1 Min-Cut: Amplifying the Contraction Algorithm

The contraction algorithm we just analyzed outputs any fixed minimum cut with probability at least $2/[n(n-1)]$ (Equation (75)). For large n this success probability is tiny, but it is nonzero, which is precisely the kind of guarantee that admits amplification by repetition.

8.1.1 Boosting Success by Independent Repetitions

A success probability of $2/[n(n-1)]$ is small for large n , but we can amplify it by running the algorithm independently many times and keeping the smallest cut found. If we run the contraction algorithm ℓ independent times, the probability that *every* trial fails to output a given min-cut is:

$$\left(1 - \frac{2}{n(n-1)}\right)^\ell \leq e^{-2\ell/[n(n-1)]}, \quad (81)$$

where we used the inequality $1 - x \leq e^{-x}$ for $x \geq 0$.

How many repetitions ℓ do we need to make this failure probability small? Setting $\ell = \frac{1}{2} \ln(n) \cdot n(n-1)$, we obtain:

$$e^{-2\ell/[n(n-1)]} = e^{-\ln n} = \frac{1}{n}.$$

More generally, setting $\ell = c \cdot n(n-1)/2$ gives a failure probability of at most e^{-c} , which can be made as small as desired.

Remark 26. Each run of the contraction algorithm performs $n-2$ contractions, and each contraction takes $O(m)$ time in the worst case (where $m = |E|$). Thus each trial costs $O(nm)$, and $\ell = O(n^2 \ln n)$ trials give a total runtime of $O(n^3 m \ln n)$. This motivates the question: can we do better?

8.1.2 A Smarter Amplification Strategy

Observe the chain of probabilities in (75) more carefully:

$$\underbrace{\left(1 - \frac{2}{n}\right) \left(1 - \frac{2}{n-1}\right) \cdots}_{\text{early contractions: high survival prob.}} \cdot \underbrace{\cdots \frac{2}{4} \cdot \frac{1}{3}}_{\text{late contractions: low survival prob.}}$$

The factors near the beginning are close to 1, so the min-cut survives the early contractions with high probability. The danger lies in the *late* contractions, when the graph is small and each contraction has a significant chance of hitting a min-cut edge.

Why are late contractions so much more dangerous? Intuitively, the minimum cut consists of a fixed set of k edges. When the graph still has many vertices, it also has many edges in total (at least

$(n - i)k/2$ when $n - i$ vertices remain), so the min-cut edges are a tiny fraction of the whole. But as vertices are merged and the graph shrinks, the total number of edges decreases while the k min-cut edges persist (assuming they have not been contracted). This means each random contraction is increasingly likely to land on one of those critical edges. By the time only a handful of vertices remain, the min-cut edges may constitute a substantial fraction of the remaining edges, making each step a high-stakes gamble.

This observation motivates a natural strategy: **split the algorithm at a threshold k^*** : run the contraction algorithm from n vertices down to k^* vertices (where the survival probability is still decent), and then **branch into ℓ independent copies** from that point onward, each continuing the contraction from k^* vertices down to 2.

- **Phase 1 (cheap, high survival)**: Contract from n vertices to k^* vertices. This costs $n - k^*$ contractions.
- **Phase 2 (expensive, amplified)**: From the k^* -vertex graph, run ℓ independent copies of the contraction algorithm, each contracting from k^* down to 2. Each copy costs $k^* - 2$ contractions.

The total number of edge contractions is:

$$(n - k^*) + \ell \cdot (k^* - 2) = n - 2 + (\ell - 1)(k^* - 2). \quad (82)$$

Optimizing the threshold k^* in this two-phase contraction strategy to minimize the total runtime is the subject of the following exercise.

Problem 10: Optimizing the Two-Phase Contraction

From [MU: Ex. 1.25(b)(c)].

Problem Statement: Consider the two-phase contraction strategy described above. The error probability of this scheme is:

$$\left(1 - \frac{2}{k^*(k^* - 1)}\right)^\ell,$$

since each of the ℓ independent Phase 2 runs outputs the min-cut with probability at least $2/[k^*(k^* - 1)]$ (applying the contraction bound to the k^* -vertex graph). The total budget constraint is that we perform no more than $2(n - 2)$ edge contractions overall (i.e., twice the cost of a single run):

$$(n - k^*) + \ell(k^* - 2) \leq 2(n - 2).$$

Choose k^* and ℓ to **minimize the error probability** subject to this budget constraint.

8.2 Maximum Cut

We now turn from min-cut to a fundamentally different problem. While min-cut is solvable in polynomial time (e.g., via max-flow), the **maximum cut problem** is NP-hard. This means that no polynomial-time algorithm is known (or believed to exist) for finding an exact solution. Consequently, we shift our focus to *approximation algorithms*, efficient procedures that may not find the optimal solution but can provably come close.

8.2.1 Problem Definition

Given a graph $G = (V, E)$ with m edges, a **cut** is a partition of the vertex set V into two disjoint subsets A and $B = V \setminus A$. The **cut value** $C(A, B)$ is the number of edges with one endpoint in A and the other in B :

$$C(A, B) = |\{e = \{u, v\} \in E : u \in A, v \in B\}|.$$

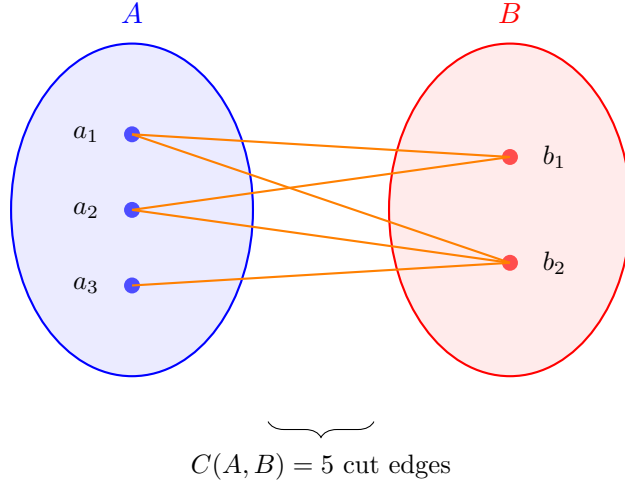


Figure 12: A cut (A, B) of a graph. The orange edges between A and B form the cut; the cut value $C(A, B)$ counts these edges.

The **max-cut problem** asks: find the partition (A, B) that maximizes $C(A, B)$. We denote the optimal value by OPT_G .

Why is max-cut NP-hard while min-cut is in P? Intuitively, min-cut has a dual characterization via maximum flow (the max-flow min-cut theorem), which yields efficient algorithms. Max-cut lacks such a dual structure; one must search over all 2^{n-1} possible partitions, and no shortcut is known.

8.2.2 A Randomized Approximation: Coin-Flipping Algorithm

Since we cannot efficiently find the exact maximum cut, we settle for a good approximation. A remarkably simple randomized algorithm achieves a $1/2$ -approximation.

We now analyze the expected cut value produced by this algorithm.

Claim. *Any graph with m edges has a cut of size at least $m/2$.*

To prove this, define indicator random variables for each edge $e = \{u, v\} \in E$:

$$X_e = \begin{cases} 1 & \text{if } e \in C(A, B) \quad (\text{i.e., } u \text{ and } v \text{ are on different sides}), \\ 0 & \text{otherwise.} \end{cases}$$

Then the cut value can be written as:

$$C(A, B) = \sum_{e \in E} X_e.$$

For any edge $e = \{u, v\}$, the two endpoints are placed independently and uniformly. There are four equally likely outcomes for the pair (u, v) :

Algorithm 6 Coin-Flipping Max-Cut**Require:** Graph $G = (V, E)$ **Ensure:** Partition (A, B) of V with $\mathbb{E}[C(A, B)] \geq m/2$

```

1:  $A \leftarrow \emptyset, \quad B \leftarrow \emptyset$ 
2: for each vertex  $v \in V$  do
3:   Flip a fair coin
4:   if heads then
5:      $A \leftarrow A \cup \{v\}$ 
6:   else
7:      $B \leftarrow B \cup \{v\}$ 
8:   end if
9: end for
10: return  $(A, B)$ 

```

u	v	e crosses the cut?
A	A	No
A	B	Yes
B	A	Yes
B	B	No

So $\mathbb{P}(X_e = 1) = 2/4 = 1/2$ for every edge e . By linearity of expectation:

$$\mathbb{E}[C(A, B)] = \mathbb{E}\left[\sum_{e \in E} X_e\right] = \sum_{e \in E} \mathbb{E}[X_e] = \sum_{e \in E} \frac{1}{2} = \frac{m}{2}. \quad (83)$$

Since $\mathbb{E}[C(A, B)] = m/2$, there must *exist* a partition achieving $C(A, B) \geq m/2$ (a random variable cannot always be below its expectation). This is a classic application of the **probabilistic method**: a randomized construction proves the existence of a combinatorial object.

8.2.3 Concentration and the Probability of a Good Cut

Equation (83) tells us the expected cut size, but how likely is it that a random partition actually achieves $C(A, B) \geq m/2$? Could it be that the expectation is $m/2$ but the probability of reaching it is tiny?

A first instinct is to invoke Chebyshev's inequality:

$$\mathbb{P}(C(A, B) = 0) \leq \mathbb{P}(|C(A, B) - \mathbb{E}[C(A, B)]| \geq \mathbb{E}[C(A, B)]) \leq \frac{\text{Var}(C(A, B))}{\mathbb{E}[C(A, B)]^2}.$$

This is suggestive but not yet rigorous: the indicators X_e are *not* independent (edges sharing a vertex are correlated through the placement of that shared endpoint), so $\text{Var}(C(A, B))$ is not simply $\sum_e \text{Var}(X_e)$ and a careful covariance computation would be needed. Rather than pursue that route, we obtain the stronger conclusion we actually need—that achieving a cut of size at least $m/2$ has probability *bounded away from zero*—by a direct expectation-splitting argument.

Let $p = \mathbb{P}(C(A, B) \geq m/2)$. Splitting the expectation:

$$\frac{m}{2} = \mathbb{E}[C(A, B)] = \sum_{j < m/2} j \mathbb{P}(C(A, B) = j) + \sum_{j \geq m/2} j \mathbb{P}(C(A, B) = j).$$

Assume for the moment that m is even. Then the first sum is at most $(\frac{m}{2} - 1)(1 - p)$ (each term is an integer below $m/2$, hence at most $m/2 - 1$, and the probabilities sum to $1 - p$), and the second sum is at most $m \cdot p$ (each term is at most m). Therefore:

$$\frac{m}{2} \leq \left(\frac{m}{2} - 1\right)(1 - p) + mp = \frac{m}{2} - 1 + p\left(\frac{m}{2} + 1\right),$$

which gives:

$$1 \leq p\left(\frac{m}{2} + 1\right) \implies p \geq \frac{1}{\frac{m}{2} + 1} = \frac{2}{m + 2}.$$

For odd m , the same argument with the bound $j \leq (m - 1)/2$ in place of $j \leq m/2 - 1$ gives the slightly weaker $p \geq 1/(m + 1)$. In either case, the probability of achieving a cut of size at least $m/2$ is positive for any graph.

8.2.4 Approximation Guarantee

Since $\text{OPT}_G \leq m$ (every edge can be cut at most once), the coin-flipping algorithm satisfies:

$$\mathbb{E}[C_M(A, B; G)] = \frac{m}{2} \geq \frac{1}{2} \text{OPT}_G,$$

where C_M denotes the cut produced by algorithm M on graph G .

Coin-Flipping Max-Cut Approximation [MU: Thm. 6.3] [AS: §2.1]

The randomized coin-flipping algorithm produces a cut whose expected size is at least $\frac{1}{2} \text{OPT}_G$. This is a $\frac{1}{2}$ -**approximation algorithm**.

Remark 27. One might wonder: if max-cut is NP-hard, how can we even *claim* that the algorithm achieves half the optimum? We do not need to compute OPT_G explicitly. Instead, we use the simple upper bound $\text{OPT}_G \leq m$ (since no cut can include more than m edges). This gives $\mathbb{E}[C_M] = m/2 \geq \text{OPT}_G/2$ without ever computing OPT_G . Approximation ratios are typically established through such *universal bounds* on the optimum.

8.3 Greedy Max-Cut

The coin-flipping algorithm is simple but entirely oblivious to the graph structure. Can we do better by making *informed* decisions? The **greedy algorithm** assigns vertices one at a time, placing each vertex on the side that maximizes the current cut value.

8.3.1 The Algorithm

Process the vertices in some fixed order $x_1, x_2, \dots, x_n$. When assigning vertex x_{k+1} , vertices $x_1, \dots, x_k$ have already been placed in A or B . Place x_{k+1} on the side that maximizes the number of edges from x_{k+1} to vertices already on the *opposite* side. That is, given only the current state of the partition, make the locally optimal (greedy) choice.

Remark 28. Each vertex x_{k+1} has edges going to some already-placed vertices. By choosing the better side, we guarantee that at least half of these “encountered” edges cross the cut. After processing all n vertices, every edge has been encountered exactly once (when the later of its two endpoints was placed). Since we capture at least half of the edges at each step, the total cut is at least $m/2$.

This gives a **deterministic** $1/2$ -approximation guarantee, matching the coin-flipping algorithm in expectation but without any randomness.

8.3.2 Local Search Improvement

After the greedy pass, we can try to improve the solution by **local search**: revisit each vertex and check whether moving it to the other side increases the cut.

Algorithm 7 Local Search Improvement for Max-Cut

Require: Graph $G = (V, E)$; initial cut (A, B) with $C(A, B) \geq m/2$

Ensure: A locally optimal cut (A, B)

```

1: repeat
2:   for each vertex  $v \in V$  do
3:     if moving  $v$  to the opposite side strictly increases  $C(A, B)$  then
4:       Move  $v$ 
5:     end if
6:   end for
7: until no vertex move improves  $C(A, B)$ 
8: return  $(A, B)$ 

```

The procedure unfolds as follows:

1. Complete the greedy assignment to obtain a cut of size at least $m/2$.
2. Return to x_1 . With all other vertices fixed, check if flipping x_1 to the other side increases $C(A, B)$. If so, flip it.
3. Repeat for $x_2, x_3, \dots, x_n$, and continue cycling through the vertices.
4. Stop when a full pass through all vertices produces no improvement (a **local optimum**).

Each pass examines n vertices; vertex v 's flip-check inspects only its incident edges, so a full pass takes $\sum_{v \in V} \deg(v) = 2m = O(m)$ time. The algorithm converges in at most $O(m)$ passes (since the cut value increases by at least 1 each time a flip occurs, and $C(A, B) \leq m$), giving a worst-case total runtime of $O(m^2)$. The resulting solution is a local optimum that is at least $m/2$.

8.4 Greedy Algorithm for Monotone Submodular Maximization

We now step back from max-cut to study greedy algorithms in a broader framework. Many combinatorial optimization problems, including **maximum coverage**, share a common structure: they involve maximizing a *submodular* function. The greedy algorithm turns out to provide a strong approximation guarantee for such problems.

8.4.1 Maximum Coverage

The **maximum coverage problem** is: given a collection of sets and a budget k , choose k sets to maximize the number of elements covered (i.e., the size of their union). A natural instance arises on graphs: given a graph $G = (V, E)$, each vertex v “covers” all edges incident to it. Selecting a set $S \subseteq V$ of k vertices covers all edges that have at least one endpoint in S . The goal is to choose k vertices to cover as many edges as possible.

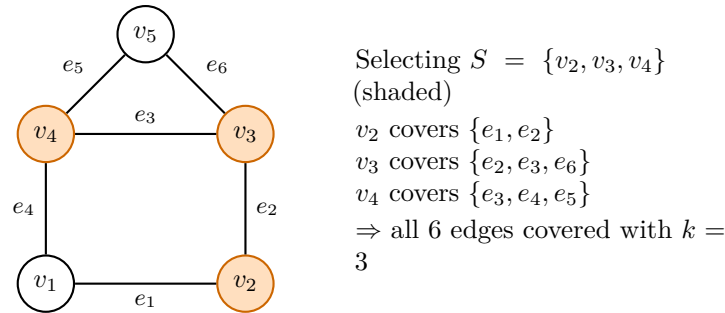


Figure 13: Maximum coverage on a graph with 5 vertices and 6 edges (a square plus a triangle sharing an edge). Each vertex covers its incident edges. Choosing $k = 3$ vertices $\{v_2, v_3, v_4\}$ suffices to cover all 6 edges.

8.4.2 Monotonicity and Submodularity

Let $f : 2^V \rightarrow \mathbb{R}$ be a set function (e.g., $f(A)$ = number of elements covered by the sets in A).

Definition 29 (Monotonicity). A set function f is **monotone** if $A \subseteq B \implies f(A) \leq f(B)$.

In the coverage setting, adding more sets to our selection can only increase (or maintain) the number of covered elements, so the coverage function is monotone.

Definition 30 (Submodularity). A set function f is **submodular** if for all $A \subseteq B$ and any element $e \notin B$:

$$f(A \cup \{e\}) - f(A) \geq f(B \cup \{e\}) - f(B). \quad (84)$$

This is the **diminishing returns** property: the marginal gain from adding e is larger when the current set is smaller. In the coverage context, adding a new set to a small collection is likely to cover many new elements, but adding the same set to a large collection covers fewer new elements (since more are already covered).

An equivalent characterization of submodularity in terms of pairwise marginal returns is established in the following exercise.

Problem 11: Equivalent Characterization of Submodularity

Instructor-supplied.

Problem Statement: Show that the following two conditions on a set function $f : 2^V \rightarrow \mathbb{R}$ are equivalent:

- (a) **Diminishing returns:** For all $A \subseteq B \subseteq V$ and any $e \notin B$,

$$f(A \cup \{e\}) - f(A) \geq f(B \cup \{e\}) - f(B).$$

- (b) **Subadditivity of unions:** For all $A, B \subseteq V$,

$$f(A) + f(B) \geq f(A \cup B) + f(A \cap B).$$

8.5 The Greedy Algorithm and Its Guarantee

Now that we have the language of submodularity, let us analyze the greedy algorithm for maximizing a monotone submodular function subject to a cardinality constraint: $\max_{|S| \leq k} f(S)$.

8.5.1 The Greedy Strategy

The greedy algorithm is simple: at each step, add the element with the **largest marginal gain**.

Algorithm 8 Greedy Algorithm for Monotone Submodular Maximization

Require: Ground set V ; monotone submodular $f: 2^V \rightarrow \mathbb{R}$ with $f(\emptyset) = 0$; budget k

Ensure: A set $G_k \subseteq V$ with $|G_k| = k$

```

1:  $G_0 \leftarrow \emptyset$ 
2: for  $i = 1$  to  $k$  do
3:    $g_i \leftarrow \arg \max_{e \notin G_{i-1}} [f(G_{i-1} \cup \{e\}) - f(G_{i-1})]$ 
4:    $G_i \leftarrow G_{i-1} \cup \{g_i\}$ 
5: end for
6: return  $G_k$ 

```

In the maximum coverage setting, this means: at each step, pick the set that covers the most new (previously uncovered) elements. This is the “largest degree” heuristic.

8.5.2 Warm-Up: Greedy Achieves $\frac{1}{2}$ OPT

Before proving the tight $(1 - 1/e)$ bound, let us first show the simpler result that greedy achieves at least half the optimal value.

Let $O_k = \{o_1, o_2, \dots, o_k\}$ denote an optimal solution with $f(O_k) = \text{OPT}$, and let $G_k = \{g_1, \dots, g_k\}$ denote the greedy solution. We have:

$$f(O_k) \leq f(O_k \cup G_k) \tag{85}$$

$$\leq f(G_k) + \sum_{o \in O_k} [f(G_k \cup \{o\}) - f(G_k)] \tag{86}$$

where (85) uses monotonicity ($O_k \subseteq O_k \cup G_k$) and (86) uses submodularity (adding the elements of O_k one at a time, each marginal gain is at most the singleton gain over G_k).

Now, by submodularity again, each $f(G_k \cup \{o_i\}) - f(G_k)$ is at most $f(G_{i-1} \cup \{o_i\}) - f(G_{i-1})$ (since $G_{i-1} \subseteq G_k$). And since greedy chose g_i over o_i at step i , we have $f(G_{i-1} \cup \{o_i\}) - f(G_{i-1}) \leq f(G_i) - f(G_{i-1})$. Therefore:

$$\sum_{o \in O_k} [f(G_k \cup \{o\}) - f(G_k)] \leq \sum_{i=1}^k [f(G_i) - f(G_{i-1})] = f(G_k) - f(\emptyset) = f(G_k).$$

Substituting back:

$$f(O_k) \leq f(G_k) + f(G_k) = 2f(G_k) \quad \implies \quad f(G_k) \geq \frac{1}{2} f(O_k).$$

Greedy 1/2-Approximation [Nemhauser–Wolsey–Fisher, 1978; not in course textbooks]

For any monotone submodular function f with $f(\emptyset) = 0$, the greedy algorithm with budget k satisfies $f(G_k) \geq \frac{1}{2} \text{OPT}$.

8.5.3 The Tight Bound: Greedy Achieves $(1 - 1/e)$ OPT

We can improve the analysis by tracking the **gap** between the optimal value and the greedy solution at each step. Define:

$$\Delta_i = f(O_k) - f(G_i),$$

the gap remaining after i greedy steps. Note that $\Delta_0 = f(O_k) - f(\emptyset) = f(O_k)$ (the initial gap is the full optimal value, since $f(\emptyset) = 0$).

Key inequality. By monotonicity and submodularity:

$$f(O_k) \leq f(G_i \cup O_k) \leq f(G_i) + \sum_{o_j \in O_k} [f(G_i \cup \{o_j\}) - f(G_i)].$$

Since $|O_k| = k$, there exists some $o_j \in O_k$ with marginal gain at least $\frac{1}{k} \sum_{o_j \in O_k} [f(G_i \cup \{o_j\}) - f(G_i)]$. The greedy choice g_{i+1} has *at least* this marginal gain (since greedy picks the best element). Therefore:

$$f(G_{i+1}) - f(G_i) \geq \frac{1}{k} (f(O_k) - f(G_i)) = \frac{\Delta_i}{k}.$$

Since $\Delta_i - \Delta_{i+1} = f(G_{i+1}) - f(G_i) \geq \Delta_i/k$, we obtain:

$$\Delta_{i+1} \leq \Delta_i - \frac{\Delta_i}{k} = \left(1 - \frac{1}{k}\right) \Delta_i.$$

Applying this recursively:

$$\Delta_k \leq \left(1 - \frac{1}{k}\right)^k \Delta_0 = \left(1 - \frac{1}{k}\right)^k f(O_k).$$

Using the well-known inequality $(1 - 1/k)^k \leq 1/e$ for all $k \geq 1$:

$$f(O_k) - f(G_k) = \Delta_k \leq \frac{1}{e} f(O_k),$$

which gives:

$$f(G_k) \geq \left(1 - \frac{1}{e}\right) f(O_k) \approx 0.632 \text{ OPT}.$$

Greedy $(1 - 1/e)$ -Approximation for Monotone Submodular Maximization [Nemhauser–Wolsey–Fisher, 1978; not in course textbooks]

For any monotone submodular function f with $f(\emptyset) = 0$ and any cardinality constraint k , the greedy algorithm satisfies:

$$f(G_k) \geq \left(1 - \frac{1}{e}\right) \text{OPT}.$$

This bound is **tight**: there exist instances where greedy achieves exactly $(1 - 1/e)$ OPT and

no better.

Remark 31 (Tightness as NP-hardness). The tightness in the result above is not just an artifact of bad instances for the greedy heuristic. Feige [9] proved that approximating the maximum coverage problem within any ratio strictly better than $1 - 1/e$ is NP-hard, that is, no polynomial time algorithm achieves a $(1 - 1/e + \alpha)$ -approximation for any constant $\alpha > 0$ unless $P = NP$. Since maximum coverage is a special case of monotone submodular maximization under a cardinality constraint, this rules out any improvement over the greedy guarantee even by more sophisticated algorithms in the same complexity class.

8.6 Beyond Monotone Submodularity

Remark 32. The max-cut objective function is submodular but **not** monotone: adding a vertex to one side of the partition can decrease the cut value. The $(1 - 1/e)$ -guarantee above relies on monotonicity, so it does not directly apply to max-cut. For non-monotone submodular maximization, a technique known as **double greedy** can be used: one process starts from the empty set (adding elements greedily) while another starts from the full set (removing elements greedily). The two processes converge to a solution that achieves a $1/2$ -approximation for unconstrained non-monotone submodular maximization.

Further Reading

This section established the classical $(1 - 1/e)$ approximation guarantee of the greedy algorithm for monotone submodular maximization under a cardinality constraint. The reading below shows how the same approximation ratio is preserved while making the algorithm dramatically faster.

- Mirzasoleiman et al. [21], “Lazier Than Lazy Greedy.” Develops STOCHASTIC-GREEDY, the first *linear-time* algorithm for maximizing a monotone submodular function subject to a cardinality constraint. The algorithm replaces the full marginal-gain scan of each greedy iteration with a small uniform random subsample of size $s = (n/k) \log(1/\varepsilon)$, and achieves a $(1 - 1/e - \varepsilon)$ approximation guarantee in expectation to the optimum with only $O(n \log(1/\varepsilon))$ function evaluations, independent of the cardinality k . Experiments on data summarization tasks (training large-scale kernel methods, exemplar-based clustering, sensor placement) show that STOCHASTIC-GREEDY practically matches the utility of lazy greedy at a fraction of the cost; in many practical scenarios it does not even evaluate the whole dataset once.
- Buchbinder et al. [22], “A Tight Linear Time $(1/2)$ -Approximation for Unconstrained Submodular Maximization.” Removes both the cardinality constraint and the monotonicity assumption studied in this section. For unconstrained, possibly non-monotone submodular maximization, the paper introduces the *Double Greedy* algorithm, which maintains two solutions $A = \emptyset$ and $B = V$ in lockstep and at each element either inserts it into A or removes it from B based on marginal gains. The deterministic version achieves a $1/3$ -approximation in linear time, and a randomized choice (probability proportional to the two marginal gains) sharpens this to a tight $1/2$ -approximation matching the known hardness bound. The same framework also yields linear-time $3/4$ -approximations for Submodular MAX-SAT and the two-player Submodular Welfare problem.
- Kumar et al. [23], “Fast Greedy Algorithms in MapReduce and Streaming.” Carries the greedy submodular-maximization framework from the sequential model studied above into the *distributed* (MapReduce) and *streaming* models, in which each machine has only sublinear memory and the algorithm is restricted to a small number of parallel rounds or sequential passes. The SAMPLE&PRUNE technique repeatedly samples a sublinear set of candidate elements, runs centralized greedy on each shard, and prunes elements whose marginal gain falls below the

current threshold. For hereditary constraint families this yields $O(1/\delta)$ -round MapReduce algorithms that match the centralized greedy approximation: $1/(2 + \varepsilon)$ for matroid constraints, $1/2 - \varepsilon$ for one-knapsack monotone submodular maximization, and $1/(p + 1 + \varepsilon)$ for p -system constraints, all using $O(kn^\delta)$ memory per machine.

9 Independent Sets, Satisfiability, and Random Walks

Textbook Roadmap

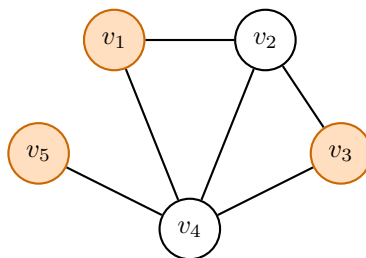
For a parallel treatment in the course textbooks, see:

- *Independent sets via sample-and-modify* ($p^* = 1/d$, $\mathbb{E}[X - Y] = n^2/(4m)$): [MU: Ch. 6, §§6.4–6.4.1, Thm. 6.5]; [AS: Ch. 1, §1.6]; [AS: Ch. 3, §3.2] (sample-then-delete).
- *SAT, MAX-SAT, random assignment satisfies $\geq m(1 - 2^{-k})$ clauses*: [MU: Ch. 6, §6.2.2, Thm. 6.4]; [AS: Ch. 1, §1.1].
- *Lovász Local Lemma (symmetric version, $4dp \leq 1$) and k -SAT bound $T = 2^k/4k$* : [MU: Ch. 6, §6.7, Thm. 6.11; Thm. 6.13]; [AS: Ch. 5, §§5.1–5.2].
- *2-SAT random walk* ($h_j = n^2 - j^2$, expected steps $\leq n^2$, success $\geq 1 - 2^{-m}$): [MU: Ch. 7, §7.1.1, Alg. 7.1; Lem. 7.1 & Thm. 7.2].
- *3-SAT random walk (biased $1/3$ up, $2/3$ down, $h_0 = 2^{n+2} - 3n - 4 = \Theta(2^n)$)*: [MU: Ch. 7, §7.1.2, Alg. 7.2].
- *Minimum vertex cover and the complementarity $\alpha(G) + \beta(G) = n$* : standard graph theory background; see [AS: Ch. 1, §1.6] for the independent-set context.

The previous chapter showed how randomization combined with submodular structure delivers sharp approximation guarantees for max-cut and maximum coverage. We now move to combinatorial problems whose underlying objective lacks such structure, so the tools shift accordingly. The probabilistic method gives existence guarantees for independent sets, simple randomized rounding handles the satisfiability variants, and random walks turn the search itself into a stochastic process that we analyze by hitting-time methods.

9.1 Independent Sets

An **independent set** in a graph $G = (V, E)$ is a subset $I \subseteq V$ such that no two vertices in I are adjacent. That is, for all $u, v \in I$, $\{u, v\} \notin E$.



Shaded vertices $\{v_1, v_3, v_5\}$ form an independent set: no edge connects any pair within the set.

Figure 14: An independent set $\{v_1, v_3, v_5\}$ in a graph with 5 vertices and 6 edges. No two shaded vertices share an edge.

The **maximum independent set problem** asks: given G , find an independent set of largest

possible size, denoted $\alpha(G)$. This problem is **NP-hard**: no polynomial-time algorithm is known, and even approximating $\alpha(G)$ within a constant factor is NP-hard. This motivates the use of randomized algorithms and the probabilistic method to obtain guaranteed lower bounds on $\alpha(G)$.

9.2 A Randomized Algorithm for Independent Sets

The probabilistic method can establish that every graph has an independent set of a certain guaranteed size. The idea is a *sample-and-modify* technique: first sample a random subset of vertices, then repair the result by removing problematic vertices.

The algorithm. Let $G = (V, E)$ have n vertices and m edges. Let $\bar{d} = 2m/n$ be the average degree. Consider:

1. Delete each vertex of G (together with its incident edges) independently with probability $1 - p$, where p is a sampling probability.
2. For each remaining edge, remove it and one of its adjacent vertices.

The remaining vertices form an independent set, since all edges have been removed.

Let X be the number of vertices that survive the first step, and Y the number of edges that survive. Then:

$$\mathbb{E}[X] = np, \quad \mathbb{E}[Y] = mp^2.$$

The second step removes at most Y vertices, so the independent set has size at least $X - Y$:

$$\mathbb{E}[X - Y] = np - mp^2.$$

To find the optimal p , differentiate and set the derivative to zero:

$$\frac{d}{dp}(np - mp^2) = n - 2mp = 0 \quad \implies \quad p^* = \frac{n}{2m} = \frac{1}{\bar{d}}.$$

Substituting p^* back:

$$\mathbb{E}[X - Y] = \frac{n^2}{2m} - \frac{n^2}{4m} = \frac{n^2}{4m}.$$

Independent Set via Sample-and-Modify [MU: Thm. 6.5] [AS: §3.2]

Every graph G on n vertices with m edges has an independent set of size at least $n^2/4m$.

9.3 Minimum Vertex Cover

A **vertex cover** of $G = (V, E)$ is a subset $C \subseteq V$ such that every edge has at least one endpoint in C . The **minimum vertex cover** problem asks for the smallest such set.

Minimum vertex cover and maximum independent set are **complementary** problems. If I is an independent set, then $V \setminus I$ is a vertex cover: any edge not covered by $V \setminus I$ would have both endpoints in I , contradicting independence. Conversely, the complement of any vertex cover is an independent set. Therefore, if we denote the size of a minimum vertex cover by $\beta(G)$:

$$\beta(G) + \alpha(G) = n.$$

Finding the maximum independent set is equivalent to finding the minimum vertex cover: solving one immediately gives the other. This complementarity between two seemingly different combinatorial

problems is itself a hint that NP-hard problems share a deeper common structure. We will see this structure most clearly in the canonical logic problem we turn to next, satisfiability, which captures the difficulty of every problem in NP through a single Boolean formalism.

A greedy alternative to this randomized algorithm and its approximation guarantee for independent sets are explored in the following exercise.

Problem 12: Greedy Algorithm for Independent Sets

Adapted from [MU: Ex. 6.3] (random-permutation algorithm with the same Caro–Wei lower bound $\sum 1/(d_i + 1)$); the homework analyses a smallest-degree greedy variant instead.

Problem Statement: We have seen a randomized sample-and-modify algorithm for finding independent sets. Now consider a **greedy** approach: at each step, pick the vertex that is currently the “best” candidate to add to the independent set (e.g., the one with the smallest degree in the remaining graph), add it to the set, and remove it along with all its neighbors from the graph. Repeat until no vertices remain.

Analyze this greedy algorithm and establish a lower bound on the size of the independent set it produces.

9.4 Satisfiability

We now shift from graph problems to Boolean logic. The **satisfiability (SAT)** problem is one of the most fundamental problems in computer science, and the first problem proven to be NP-complete.

9.4.1 Historical Context

Alan Turing formalized computation through the *Turing machine* in 1936 and showed that the *halting problem*, which asks whether a given program eventually stops, is **undecidable**: no algorithm can solve it in general.

In 1971, Stephen Cook proved that SAT is **NP-complete**: every problem in NP can be reduced to SAT in polynomial time. Any operation of a Turing machine can be expressed as a Boolean formula, and checking whether the machine accepts an input in polynomial time corresponds to finding a satisfying assignment. This is what it means for a solution to be *verifiable in polynomial time*. Shortly after, in 1972, Richard Karp identified 21 NP-complete problems spanning combinatorics, graph theory, and logic, demonstrating that NP-completeness is pervasive.

9.4.2 Conjunctive Normal Form and SAT

A **literal** is a Boolean variable x_i or its negation $\bar{x}_i$. A **clause** is a disjunction (OR) of literals, e.g., $(x_1 \vee \bar{x}_2 \vee x_3)$. A formula in **conjunctive normal form (CNF)** is a conjunction (AND) of clauses:

$$\Phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m,$$

where each clause $C_i = (\ell_{i,1} \vee \ell_{i,2} \vee \cdots \vee \ell_{i,k_i})$ is the disjunction of k_i literals. The **SAT problem** asks: given n variables and m clauses, does there exist a True/False assignment to the variables such that every clause contains at least one True literal?

9.4.3 The Phase Transition

The difficulty of a random SAT instance depends critically on the **clause-to-variable ratio** m/n . When m/n is small, there are few constraints relative to the number of variables, and satisfying assignments are abundant, so the problem is easy. When m/n is large, the formula is heavily constrained and almost certainly unsatisfiable, which is also easy to determine. The hardest instances arise near a **critical threshold ratio**, where the problem transitions sharply from almost-always satisfiable to almost-always unsatisfiable. Near this threshold, both heuristic and exact solvers struggle.

9.5 Maximum Satisfiability (MAX-SAT)

When a SAT formula may not be fully satisfiable, a natural optimization variant asks: find an assignment that satisfies as many clauses as possible. This is the **MAX-SAT** problem, which is NP-hard.

A simple random assignment provides a strong guarantee. Assign values independently and uniformly at random to the n variables (each True with probability $1/2$). The probability that the i -th clause (with k_i literals) is **not** satisfied equals the probability that all k_i literals evaluate to False:

$$\mathbb{P}(\text{clause } i \text{ not satisfied}) = \frac{1}{2^{k_i}}.$$

By linearity of expectation, the expected number of satisfied clauses is:

$$\mathbb{E}[\text{satisfied clauses}] = \sum_{i=1}^m \left(1 - \frac{1}{2^{k_i}}\right) \geq m \left(1 - \frac{1}{2^k}\right),$$

where $k = \min_i k_i$. Since the expectation is at least $m(1 - 1/2^k)$, there must exist an assignment achieving this bound. This is a direct application of the **probabilistic method**: a random construction (coin flip for each variable) proves the existence of a good solution. The argument can be converted into a Las Vegas algorithm or derandomized via the method of conditional expectations (Section 6.3 of [2]).

MAX-SAT Random Assignment Bound [MU: Thm. 6.4] [AS: §1.1]

Given m clauses with k_i literals each and $k = \min_i k_i$, there exists a truth assignment satisfying at least $m(1 - 1/2^k)$ clauses.

Improving the basic $1/2$ -approximation for MAX-SAT to a $3/4$ -approximation, by combining randomized rounding with a smarter assignment, is the subject of the following exercise.

Problem 13: Improving the MAX-SAT Bound

From [Mor: Ex. 22.5(b)(c)].

Problem Statement: Let $C^* = m(1 - 1/2^k)$ denote the expected number of satisfied clauses under a uniform random assignment, where $k = \min_i k_i$. Let W be the (random)

number of satisfied clauses. By splitting the expectation at C^* :

$$C^* = \mathbb{E}[W] = \sum_{j < C^*} j \mathbb{P}(W = j) + \sum_{j \geq C^*} j \mathbb{P}(W = j).$$

Let $p = \mathbb{P}(W \geq C^*)$. Using the bounds $j < C^*$ in the first sum and $j \leq m$ in the second:

$$C^* \leq (C^* - 1)(1 - p) + mp.$$

Derive a lower bound on p from this inequality. Use this to bound the expected number of random trials needed before finding an assignment that satisfies at least C^* clauses.

9.6 k -SAT and the Lovász Local Lemma

We now consider k -SAT, where every clause has exactly k literals. For $k \geq 3$, the problem is NP-complete. However, if no variable appears in too many clauses, the Lovász Local Lemma guarantees the existence of a satisfying assignment.

9.6.1 The Lovász Local Lemma

The Lovász Local Lemma shows that the probability of avoiding a collection of “bad” events is positive, even when the events are *not* mutually independent, provided the dependency among them is limited.

Setup. Let $E_1, \dots, E_n$ be events in some probability space. The **dependency graph** has an edge between E_i and E_j whenever they share underlying randomness (e.g., common variables). Event E_i is mutually independent of all non-adjacent events.

Theorem 33 (Lovász Local Lemma, symmetric version [MU: Thm. 6.11] [AS: §5.1]). *If $\mathbb{P}(E_i) \leq p$ for all i , the degree of the dependency graph is bounded by d , and $4dp \leq 1$, then*

$$\mathbb{P}\left(\bigcap_{i=1}^n \bar{E}_i\right) > 0,$$

where $\bar{E}_i$ denotes the complement of E_i (i.e., event E_i does not occur). In other words, with positive probability, **none** of the bad events occur.

Remark 34. If the events were mutually independent, we could write $\mathbb{P}(\bigcap \bar{E}_i) = \prod (1 - \mathbb{P}(E_i)) > 0$. But when events are dependent, i.e., $\mathbb{P}(E_i \cap E_j) \neq \mathbb{P}(E_i)\mathbb{P}(E_j)$, this product formula fails. The Lovász Local Lemma handles this by requiring only that the dependency is bounded ($4dp \leq 1$).

Remark 35 (Sharper constants). The condition $4dp \leq 1$ stated above is the most commonly cited form of the symmetric Lovász Local Lemma, but it is not the tightest constant possible. Shearer [10] pinned down the exact threshold: writing $f(d)$ for the supremum of values p such that $\mathbb{P}(E_i) \leq p$ on a dependency graph of maximum degree d still guarantees $\mathbb{P}(\bigcap \bar{E}_i) > 0$, the sharp bound is $f(1) = 1/2$ and

$$f(d) = \frac{(d-1)^{d-1}}{d^d}, \quad d \geq 2,$$

with $\lim_{d \rightarrow \infty} d \cdot f(d) = 1/e$. The well known intermediate improvement $ep(d+1) \leq 1$ noted by Spencer falls between the loose $4dp \leq 1$ used above and the sharp Shearer threshold. Asymptotically, Shearer’s bound saves a factor of about $4e \approx 10.87$ over the constant $4dp \leq 1$.

Remark 36 (Constructive proof). The original Lovász Local Lemma is non constructive: it shows that a satisfying configuration exists with positive probability but offers no efficient procedure for finding one. Moser and Tardos [11] closed this gap by exhibiting a simple resampling algorithm that, under essentially the same hypothesis as the lemma, finds a satisfying configuration in expected polynomial time. The resulting algorithmic Lovász Local Lemma was awarded the 2020 Gödel Prize and now underlies almost all algorithmic applications of the lemma in modern combinatorics.

9.6.2 Application to k -SAT

Assign a random value to each variable independently and uniformly. For each clause C_i , define the bad event $E_i = \{\text{clause } C_i \text{ is not satisfied}\}$. Since each clause has k literals:

$$\mathbb{P}(E_i) = \frac{1}{2^k} =: p.$$

Now suppose each variable appears in at most T clauses. Clause i contains k variables, and each of these variables can link clause i to at most $T - 1$ other clauses, so the degree of the dependency graph is bounded by $d \leq k(T - 1)$. The Lovász Local Lemma condition becomes:

$$4dp \leq 4 \cdot k(T - 1) \cdot \frac{1}{2^k} = \frac{4k(T - 1)}{2^k} \leq 1 \implies T \leq \frac{2^k}{4k} + 1.$$

Satisfiability via Lovász Local Lemma [MU: Thm. 6.13] [AS: Ch. 5, §5.1]

If no variable in a k -SAT formula appears in more than $T = 2^k/(4k) + 1$ clauses, then the formula has a satisfying assignment.

Remark 37. This condition is non-trivial (i.e., $T \geq 1$) when $2^k > 4k$, equivalently $2^{k-2} > k$. This holds for $k \geq 5$.

9.7 2-SAT

We now turn to **2-SAT**, where every clause has exactly two literals. Unlike general SAT ($k \geq 3$), 2-SAT is solvable in **polynomial time**.

9.7.1 The Implication Graph

A 2-SAT clause $(\ell_1 \vee \ell_2)$ can be rewritten as two logical implications:

$$\bar{\ell}_1 \Rightarrow \ell_2 \quad \text{and} \quad \bar{\ell}_2 \Rightarrow \ell_1.$$

If ℓ_1 is False, then ℓ_2 must be True for the clause to be satisfied (and vice versa).

Example 38. Consider the clause $(\bar{x} \vee y)$:

x	y	$\bar{x} \vee y$	Satisfied?
F	T	T	Yes
T	F	F	No
T	T	T	Yes
F	F	T	Yes

The only failing case is $x = T, y = F$. The clause is equivalent to the implications $x \Rightarrow y$ and $\bar{y} \Rightarrow \bar{x}$.

Why use an implication graph? The key insight is that every 2-SAT clause naturally decomposes into exactly two implications. By encoding all of these implications as directed edges, we transform the logical satisfiability question into a *graph-reachability* question: if a chain of implications forces both x_i and $\bar{x}_i$ to be simultaneously true, the formula is unsatisfiable. This happens precisely when x_i and $\bar{x}_i$ lie in the same strongly connected component. The power of this reduction is that strongly connected components can be found in $O(n + m)$ time (e.g., via Tarjan's or Kosaraju's algorithm), giving us a polynomial-time decision procedure for 2-SAT.

Formally, the **implication graph** is a directed graph on $2n$ nodes (one for each literal x_i and $\bar{x}_i$) with $2m$ directed edges (each clause contributes 2 edges). A 2-SAT formula is satisfiable if and only if no variable and its negation belong to the same strongly connected component.

9.7.2 A Randomized Algorithm for 2-SAT

Although 2-SAT can be solved deterministically, it also admits a simple randomized algorithm based on random walks.

Algorithm 9 Random Walk Algorithm for 2-SAT

Require: 2-SAT formula φ on n variables with m clauses

Ensure: A satisfying assignment, or “unsatisfiable”

```

1:  $A \leftarrow$  an arbitrary truth assignment
2: for  $t = 1$  to  $2mn^2$  do
3:   if  $A$  satisfies  $\varphi$  then return  $A$ 
4:   end if
5:   Pick any clause  $C$  in  $\varphi$  not satisfied by  $A$ 
6:   Pick one of the two literals of  $C$  uniformly at random
7:   Flip the chosen literal's variable in  $A$ 
8: end for
9: return “unsatisfiable”

```

9.7.3 Analysis via Random Walk

Fix a satisfying assignment S . Let A_i denote the current assignment after i steps, and define X_i as the number of variables in A_i that agree with S . When $X_i = n$, we have found S .

At each step, we pick an unsatisfied clause. Since S satisfies it but A_i does not, at least one of the two variables disagrees with S . When we flip a randomly chosen variable:

- With probability $\geq 1/2$: we flip a disagreeing variable, so $X_{i+1} = X_i + 1$.
- With probability $\leq 1/2$: we flip an agreeing variable, so $X_{i+1} = X_i - 1$.

To bound the expected time to reach n , we analyze a pessimistic random walk on $\{0, 1, \dots, n\}$ where from any state $j \in \{1, \dots, n-1\}$, we move up or down with equal probability $1/2$, and from state 0 we always move to 1.

9.7.4 Hitting Time Analysis

Let h_j be the expected number of steps to reach state n from state j . The boundary conditions and recurrence are:

$$h_n = 0, \quad h_0 = 1 + h_1, \quad h_j = \frac{1}{2} h_{j-1} + \frac{1}{2} h_{j+1} + 1 \quad (1 \leq j \leq n-1).$$

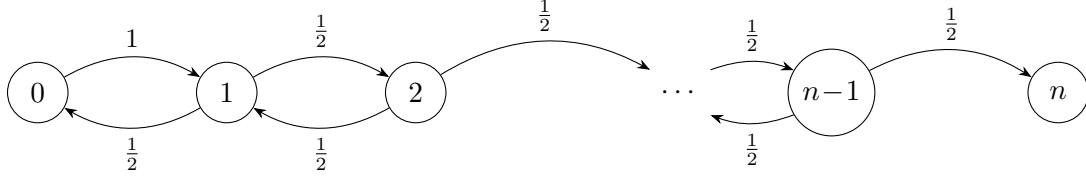


Figure 15: Random walk on $\{0, 1, \dots, n\}$ for the 2-SAT analysis. State n is absorbing.

The solution is $h_j = n^2 - j^2$. We verify for $1 \leq j \leq n - 1$:

$$\frac{h_{j-1} + h_{j+1}}{2} + 1 = \frac{(n^2 - (j-1)^2) + (n^2 - (j+1)^2)}{2} + 1 = n^2 - j^2 = h_j. \quad \checkmark$$

And $h_0 = n^2 = (n^2 - 1) + 1 = h_1 + 1. \quad \checkmark$

2-SAT Random Walk [MU: Lem. 7.1; Thm. 7.2]

If a 2-SAT formula has a satisfying assignment, the random walk algorithm finds one in at most $h_0 = n^2$ expected steps.

9.7.5 Bounding the Failure Probability

The algorithm runs for $2mn^2$ steps divided into segments of $2n^2$ steps. By Markov's inequality, within a single segment:

$$\mathbb{P}(Z > 2n^2) \leq \frac{\mathbb{E}[Z]}{2n^2} \leq \frac{n^2}{2n^2} = \frac{1}{2},$$

where Z is the number of steps to reach a satisfying assignment. Each segment succeeds with probability at least $1/2$, so after m segments the failure probability is at most $(1/2)^m$.

9.8 3-SAT

We extend the random walk approach to **3-SAT**, where each clause has exactly three literals. Unlike 2-SAT, 3-SAT is NP-complete, so the expected running time becomes exponential.

9.8.1 The Random Walk for 3-SAT

The algorithm is identical to the 2-SAT version, but now we randomly flip one of *three* literals. Fix a satisfying assignment S and define X_i as before (the number of agreements between the current assignment and S). The transition probabilities change: at each step, at least one of three variables in the unsatisfied clause disagrees with S , so:

- $\mathbb{P}(X_{i+1} = j + 1 \mid X_i = j) \geq 1/3$
- $\mathbb{P}(X_{i+1} = j - 1 \mid X_i = j) \leq 2/3$

The probability of going up has decreased from $1/2$ to $1/3$, making the walk biased away from the satisfying assignment.

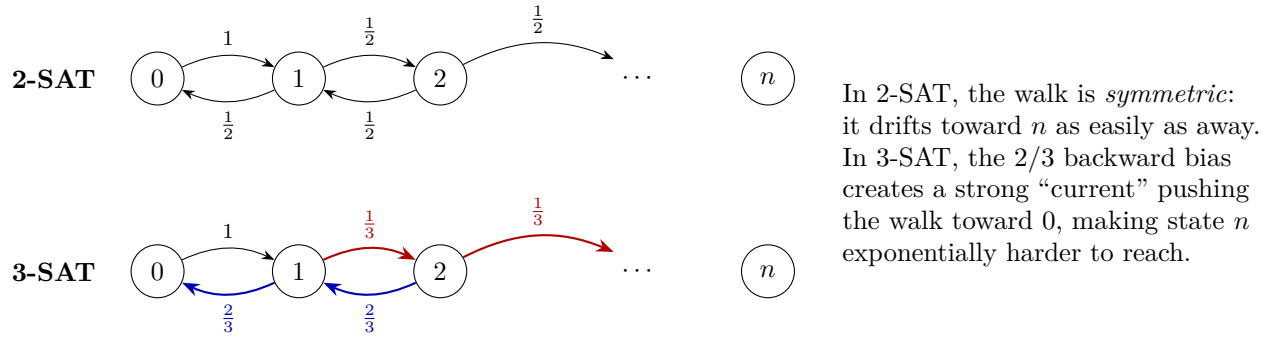


Figure 16: Comparison of the 2-SAT and 3-SAT random walks. The symmetric $1/2$ versus $1/2$ split gives polynomial hitting time (n^2), while the $1/3$ versus $2/3$ bias makes the hitting time exponential ($\Theta(2^n)$).

9.8.2 Hitting Time Analysis

The pessimistic Markov chain has:

$$h_n = 0, \quad h_0 = 1 + h_1, \quad h_j = \frac{2}{3} h_{j-1} + \frac{1}{3} h_{j+1} + 1 \quad (1 \leq j \leq n-1).$$

The solution is $h_j = 2^{n+2} - 2^{j+2} - 3(n-j)$, which can be verified using the relation $h_j = h_{j+1} + 2^{j+2} - 3$. In particular:

$$h_0 = 2^{n+2} - 3n - 4 = \Theta(2^n).$$

3-SAT Random Walk [MU: Alg. 7.2; hitting-time formula]

The naïve random walk for 3-SAT has expected hitting time $h_0 = 2^{n+2} - 3n - 4 = \Theta(2^n)$. This matches brute-force exhaustive search up to constants ($h_0 = 4 \cdot 2^n - O(n)$), so the random walk *by itself* buys us nothing over enumeration. The improvement, in the next section, comes from **where we start the walk**, not from the walk dynamics themselves.

Analyzing the 2-SAT random walk under a uniformly random initial assignment, rather than an adversarial one, is the subject of the following exercise.

Problem 14: Random Starting Point for 2-SAT

From [MU: Ex. 7.7].

Problem Statement: In the analysis of the 2-SAT random walk algorithm, we bounded the expected number of steps starting from the worst case $h_0 = n^2$. Suppose instead that the algorithm starts with a truth assignment chosen **uniformly at random**. The initial number of agreements X_0 with a satisfying assignment S then follows a Binomial($n, 1/2$) distribution, so the walk starts at a random state j rather than at 0.

How does this random starting point affect the expected time until a satisfying assignment is found? Derive the improved bound by computing $\mathbb{E}[h_{X_0}]$ where $X_0 \sim \text{Binomial}(n, 1/2)$.

A more careful look at the polynomial-versus-exponential boundary in the 3-SAT walk, identifying the precise bias at which the expected hitting time changes regime, is the focus of the following exercise.

Problem 15: The Polynomial vs. Exponential Threshold

Adapted from [MU: Ex. 7.9] (largest p for polynomial-time termination of the 3-SAT walk); the homework abstracts the same threshold question to a general up-probability p on $\{0, \dots, n\}$.

Problem Statement: Consider a random walk on $\{0, 1, \dots, n\}$ where, from state j ($1 \leq j \leq n-1$), the walk moves up with probability p and down with probability $1-p$. We saw that:

- When $p = 1/2$ (2-SAT), the expected hitting time is $h_0 = n^2$ (polynomial).
- When $p = 1/3$ (3-SAT), the expected hitting time is $h_0 = \Theta(2^n)$ (exponential).

For a general $p \in (0, 1)$, determine the expected hitting time h_0 as a function of n and p . What is the critical value of p that separates polynomial from exponential behavior? Prove your answer.

10 Improved 3-SAT and the Monte Carlo Method

Textbook Roadmap

For a parallel treatment in the course textbooks, see:

- *Improved 3-SAT (random initial assignment + $3n$ -step walk + restart, expected runtime $O(n^{3/2}(4/3)^n)$):* [MU: Ch. 7, §7.1.2, Alg. 7.3 & Lem. 7.3 (Stirling); runtime].
- *The Monte Carlo method (π -estimation example, (ε, δ) -approximation, FPRAS):* [MU: Ch. 11, §11.1, Defs. 11.1–11.2; Thm. 11.1 (sample complexity $m \geq 3 \ln(2/\delta)/(\varepsilon^2 \mu)$)].
- *Counting problems and $\#P$:* [MU: Ch. 11, §11.2 (intro)].
- *DNF counting — naïve sampling failure and the FPRAS via importance sampling:* [MU: Ch. 11, §11.2.1 (Alg. 11.1); §11.2.2 (Alg. 11.2 & Thm. 11.2)].

10.1 An Improved Algorithm for 3-SAT

The random walk algorithm we just analyzed for 3-SAT runs in $\Theta(2^n)$ expected time when started from an arbitrary assignment. This is better than the 2^n brute-force enumeration only by a constant factor in the exponent. The bottleneck is the starting point: from a worst-case $X_0 = 0$, the biased $1/3$ vs. $2/3$ walk must climb n levels, and each upward step is fought against twice the downward pressure. A natural fix is to start somewhere closer to a satisfying assignment.

10.1.1 Random Initialization and Restarts

The key observation is that a *random* starting point is much better than the worst case. If we initialize each variable independently and uniformly at random, the initial number of agreements

X_0 follows a Binomial($n, 1/2$) distribution, so $\mathbb{E}[X_0] = n/2$. On average, half the variables already agree with S , meaning we only need to “fix” approximately $n/2$ disagreements rather than n .

This suggests the following improved algorithm.

Algorithm 10 Improved Random Walk for 3-SAT (with Random Restarts)

Require: 3-SAT formula φ on n variables

Ensure: A satisfying assignment

```

1: while  $\varphi$  is not satisfied do
2:   Set each variable independently and uniformly at random to True or False
3:   for  $t = 1$  to  $3n$  do
4:     if current assignment satisfies  $\varphi$  then return current assignment
5:   end if
6:   Pick any unsatisfied clause  $C$  in  $\varphi$ 
7:   Pick one of the three literals of  $C$  uniformly at random
8:   Flip the chosen variable
9:   end for
10: end while

```

Why $3n$ steps? A random start gives roughly $n/2$ agreements, so we need about $n/2$ net “upward” steps. With $3n$ total steps, there is room for roughly n good steps and n bad steps (net gain $\approx n$). If the walk has not reached n agreements within this budget, it is more efficient to restart with a fresh random assignment than to continue from a position that may have drifted far from S .

10.1.2 Analysis of the Success Probability

Let j denote the number of *disagreements* in the initial random assignment, i.e., $j = n - X_0$. We have j disagreements with probability

$$\mathbb{P}(j \text{ disagreements}) = \binom{n}{j} \left(\frac{1}{2}\right)^n.$$

Let q_j denote the probability of reaching the satisfying assignment S (i.e., reducing to 0 disagreements) within $3n$ steps, starting from j disagreements. We analyze q_j using the pessimistic random walk where the probability of decreasing disagreements is exactly $1/3$ and the probability of increasing them is exactly $2/3$.

A walk from j to 0 that takes exactly $j + 2k$ steps must consist of $j + k$ “good” steps (decreasing disagreements) and k “bad” steps (increasing them). Starting at position j and ending at position 0 after $j + 2k$ steps, any such path must pass through 0 at some intermediate point. Since the walk is absorbed at 0 (a satisfying assignment has been found), the probability of absorption by time $j + 2k$ is at least the probability of the unrestricted walk being at position 0 at time $j + 2k$:

$$q_j \geq \binom{j+2k}{k} \left(\frac{1}{3}\right)^{j+k} \left(\frac{2}{3}\right)^k, \quad \text{for any } k \text{ with } j+2k \leq 3n.$$

Taking $k = j$ (so the walk has length $3j \leq 3n$):

$$q_j \geq \binom{3j}{j} \left(\frac{1}{3}\right)^{2j} \left(\frac{2}{3}\right)^j. \tag{87}$$

10.1.3 Applying Stirling's Approximation

To evaluate the bound (87), we use Stirling's approximation in the form:

$$\sqrt{2\pi m} \left(\frac{m}{e}\right)^m \leq m! \leq 2\sqrt{2\pi m} \left(\frac{m}{e}\right)^m.$$

We need a *lower bound* on $\binom{3j}{j} = \frac{(3j)!}{(2j)!j!}$, so we use the lower bound for the numerator $(3j)!$ and the upper bounds for the denominators $(2j)!$ and $j!$:

$$\begin{aligned} \binom{3j}{j} &\geq \frac{\sqrt{2\pi \cdot 3j} (3j/e)^{3j}}{2\sqrt{2\pi \cdot 2j} (2j/e)^{2j} \cdot 2\sqrt{2\pi j} (j/e)^j} \\ &= \frac{\sqrt{6\pi j} \cdot 3^{3j} j^{3j}}{4 \cdot 2\sqrt{2} \pi j \cdot 2^{2j} j^{2j} \cdot j^j} \\ &= \frac{\sqrt{6}}{8\sqrt{2}\sqrt{\pi j}} \cdot \frac{27^j}{4^j} = \frac{\sqrt{3}}{8\sqrt{\pi j}} \cdot \frac{27^j}{4^j}. \end{aligned}$$

Multiplying by $(1/3)^{2j}(2/3)^j = 2^j/27^j$:

$$q_j \geq \frac{\sqrt{3}}{8\sqrt{\pi j}} \cdot \frac{27^j}{4^j} \cdot \frac{2^j}{27^j} = \frac{\sqrt{3}}{8\sqrt{\pi j}} \cdot \left(\frac{1}{2}\right)^j.$$

Setting $C = \frac{\sqrt{3}}{8\sqrt{\pi}}$, we have:

$$q_j \geq \frac{C}{\sqrt{j}} \cdot \left(\frac{1}{2}\right)^j, \quad j \geq 1. \quad (88)$$

10.1.4 Overall Success Probability

The overall probability q that a single trial (random start + $3n$ -step walk) finds S is:

$$q = \sum_{j=0}^n \binom{n}{j} \left(\frac{1}{2}\right)^n \cdot q_j.$$

For $j = 0$ (the initial assignment already equals S): $q_0 = 1$, contributing $\left(\frac{1}{2}\right)^n$. For $j \geq 1$, we apply the bound (88):

$$q \geq \left(\frac{1}{2}\right)^n + \sum_{j=1}^n \binom{n}{j} \left(\frac{1}{2}\right)^n \cdot \frac{C}{\sqrt{j}} \cdot \left(\frac{1}{2}\right)^j.$$

Since $1/\sqrt{j} \geq 1/\sqrt{n}$ for all $1 \leq j \leq n$:

$$q \geq \left(\frac{1}{2}\right)^n + \frac{C}{\sqrt{n}} \sum_{j=1}^n \binom{n}{j} \left(\frac{1}{2}\right)^{n+j}.$$

Noting that $C < 1$ implies $C/\sqrt{n} \leq 1$ for all $n \geq 1$, we can fold the $j = 0$ term into the bound:

$$q \geq \frac{C}{\sqrt{n}} \left(\frac{1}{2}\right)^n \sum_{j=0}^n \binom{n}{j} \left(\frac{1}{2}\right)^j.$$

By the binomial theorem, $\sum_{j=0}^n \binom{n}{j} (1/2)^j = (1 + 1/2)^n = (3/2)^n$. Therefore:

$$q \geq \frac{C}{\sqrt{n}} \cdot \left(\frac{1}{2}\right)^n \cdot \left(\frac{3}{2}\right)^n = \frac{C}{\sqrt{n}} \cdot \left(\frac{3}{4}\right)^n =: \underline{q}.$$

10.1.5 Expected Runtime

Each trial consists of $3n$ steps. The expected number of trials until success is $1/q \leq 1/\underline{q}$. Therefore:

$$\mathbb{E}[\text{runtime}] \leq \frac{3n}{\underline{q}} = 3n \cdot \frac{\sqrt{n}}{C} \cdot \left(\frac{4}{3}\right)^n = \frac{3}{C} n^{3/2} \left(\frac{4}{3}\right)^n.$$

Improved 3-SAT Algorithm [MU: Alg. 7.3 & Lem. 7.3; runtime $O(n^{3/2}(4/3)^n)$]

The random-start, $3n$ -step walk with restarts finds a satisfying assignment (if one exists) in expected time $O(n^{3/2} \cdot (4/3)^n)$. This is a dramatic improvement over the naïve walk's $\Theta(2^n)$, since $(4/3)^n \approx 1.333^n \ll 2^n$.

Remark 39. An interesting question was raised in class: instead of discarding the walk after $3n$ steps, what if we *recorded the peak* number of agreements during the walk and restarted from that best-seen state? Intuitively, this “best-so-far” strategy could improve practical performance. However, this idea runs into two obstacles. First, the agreement count X_i with the satisfying assignment S is not directly observable: S is exactly what we are trying to find. A natural proxy — recording the state with the largest number of *satisfied clauses* — does not coincide with X_i and need not track the true distance to S . Second, even if we could observe X_i , the restarted walk no longer begins from an independent random state, so the analysis techniques used above (which rely on the random initial condition) break down. Formally bounding the improvement of such a strategy remains an open challenge.

Remark 40 (Where the story goes from here). The random-init-plus-restart idea is not specific to $k = 3$: the same template extends to general k -SAT and is known as Schöning’s algorithm, yielding an expected runtime of $O^*((2 - 2/k)^n)$. For $k = 3$ this recovers the $(4/3)^n$ bound above; for larger k the gain over brute-force enumeration shrinks but remains exponential. This bound, with various refinements, was the best-known worst-case 3-SAT algorithm for many years.

10.2 Amplifying the Success Probability

The algorithm above has success probability $q \geq \underline{q}$ per trial. We now show how to convert this into a bounded-time algorithm with controllable error probability.

Let Z denote the number of independent trials until the first success. Then $Z \sim \text{Geometric}(q)$, so $\mathbb{E}[Z] = 1/q \leq 1/\underline{q}$.

Set a budget of $m = \lceil 2/\underline{q} \rceil$ trials. By Markov’s inequality:

$$\mathbb{P}(Z > m) \leq \mathbb{P}(Z > 2\mathbb{E}[Z]) \leq \frac{\mathbb{E}[Z]}{2\mathbb{E}[Z]} = \frac{1}{2}.$$

So with a budget of m trials, the probability of *not* finding a satisfying assignment is at most $1/2$.

To reduce the failure probability further, we repeat this entire procedure r times independently. Each repetition uses m trials (each of $3n$ steps). The probability that all r repetitions fail is at most $(1/2)^r$. The total runtime is:

$$r \cdot \frac{2}{\underline{q}} \cdot 3n = O\left(r \cdot n^{3/2} \cdot \left(\frac{4}{3}\right)^n\right).$$

This is a **one-sided Monte Carlo algorithm**: if it outputs a truth assignment, that assignment is guaranteed to be correct (we can verify in polynomial time). The only possible error is failing to

find a satisfying assignment when one exists. By choosing $r = O(\log(1/\delta))$, we can make the failure probability at most δ .

Remark 41. Recall the distinction between the two families of randomized algorithms:

- **Las Vegas algorithms** always produce a correct answer; only the runtime is random. *Example:* Randomized Quicksort.
- **Monte Carlo algorithms** run in bounded time but may return an incorrect answer with small probability. *Example:* the bounded 3-SAT algorithm above.

The 3-SAT algorithm with a fixed budget is Monte Carlo (one-sided error). If instead we keep restarting without any time limit, the algorithm becomes Las Vegas: it always eventually finds the satisfying assignment, but the runtime is random.

10.3 The Monte Carlo Method

We now shift to an entirely different use of randomness: *estimating quantities through random sampling*. This family of techniques is broadly called the **Monte Carlo method**.

10.3.1 Counting Problems and #P

Many computational problems ask not just whether a solution *exists* (a decision problem), but *how many* solutions there are (a counting problem). For example:

- How many satisfying assignments does a Boolean formula have?
- How many independent sets does a graph contain?
- How many perfect matchings exist in a bipartite graph?

There is a complexity class associated with counting solutions to problems in NP, denoted **#P** (pronounced “sharp-P”). A problem is in #P if it asks for the number of accepting computations of a polynomial-time nondeterministic Turing machine. A problem is **#P-complete** if it is as hard as any other problem in #P.

It is unlikely that a polynomial-time algorithm exists for any #P-complete problem, since this would imply $P = NP$. This motivates the search for efficient *approximation* algorithms.

10.3.2 (ε, δ) -Approximation and FPRAS

Since exact counting is intractable, we settle for approximate answers with probabilistic guarantees.

Definition 42 ((ε, δ) -Approximation [MU: Def. 11.1]). A randomized algorithm M gives an (ε, δ) -**approximation** for a value $V(x)$ if the output $M(x)$ satisfies:

$$\mathbb{P}(|M(x) - V(x)| > \varepsilon V(x)) < \delta.$$

That is, the estimate $M(x)$ lies within a factor of $(1 \pm \varepsilon)$ of the true value $V(x)$ with probability at least $1 - \delta$.

Definition 43 (FPRAS [MU: Def. 11.2]). A **fully polynomial randomized approximation scheme (FPRAS)** for a problem is a randomized algorithm which, given an input x and any parameters ε, δ with $0 < \varepsilon, \delta < 1$, outputs an (ε, δ) -approximation to $V(x)$ in time polynomial in $1/\varepsilon$, $\ln(1/\delta)$, and the size of the input x .

What makes an approximation scheme “fully polynomial”? The runtime must be polynomial in *both* $1/\varepsilon$ (precision) and $\ln(1/\delta)$ (confidence), not just in the input size. This ensures that we can obtain arbitrarily precise estimates with arbitrarily high confidence in polynomial time.

The *median trick*, which boosts the success probability of an FPRAS from a constant such as $2/3$ to $1 - \delta$ at only logarithmic cost in δ , is developed in the following exercise.

Problem 16: The Median Trick for FPRAS

From [MU: Ex. 11.3].

Problem Statement: Show that the following alternative definition of FPRAS is equivalent to the standard one:

A fully polynomial randomized approximation scheme (FPRAS) for a problem is a randomized algorithm which, given input x and any parameter ε with $0 < \varepsilon < 1$, outputs an $(\varepsilon, 1/4)$ -approximation in time polynomial in $1/\varepsilon$ and the size of x .

That is, an algorithm M is an (ε, δ) -approximation for all δ if and only if it is an $(\varepsilon, 1/4)$ -approximation.

Hint: To boost the success probability from $3/4$ to $1 - \delta$, consider the **median** of several independent runs. Why is the median a better choice than the mean?

Deriving the sharp sample-complexity bound $m \geq 3 \ln(2/\delta)/(\varepsilon^2 \mu)$ for averages of i.i.d. indicator variables, which we will reuse repeatedly in the FPRAS analyses below, is the content of the following exercise.

Problem 17: Sample Complexity for Indicator Variables

From [MU: Ex. 11.1] (which asks the reader to formally prove [MU: Thm. 11.1]).

Problem Statement: Let $X_1, X_2, \dots, X_m$ be independent and identically distributed indicator random variables with mean $\mu = \mathbb{E}[X_i] = \mathbb{P}(X_i = 1)$. Prove that if

$$m \geq \frac{3 \ln(2/\delta)}{\varepsilon^2 \mu},$$

then

$$\mathbb{P}\left(\left|\frac{1}{m} \sum_{i=1}^m X_i - \mu\right| > \varepsilon \mu\right) < \delta.$$

That is, the sample mean of m indicator variables is an (ε, δ) -approximation of μ .

10.4 The DNF Counting Problem

We now apply the Monte Carlo framework to a concrete #P-complete problem: counting the number of satisfying assignments of a Boolean formula in **disjunctive normal form (DNF)**.

10.4.1 What is DNF?

A formula in **DNF** is a disjunction (OR) of clauses, where each clause is a conjunction (AND) of literals:

$$F = C_1 \vee C_2 \vee \cdots \vee C_t, \quad C_i = (\ell_{i,1} \wedge \ell_{i,2} \wedge \cdots \wedge \ell_{i,l_i}).$$

For example, $(x_1 \wedge \bar{x}_2 \wedge x_3) \vee (\bar{x}_1 \wedge x_3 \wedge x_4)$ is a DNF formula.

A key property of DNF is that *satisfying* a single clause is easy: to satisfy C_i , simply set each literal in C_i to True (the remaining variables can take any value). This gives 2^{n-l_i} satisfying assignments per clause, where l_i is the number of literals in C_i . The difficulty arises because these sets overlap, and counting $\left| \bigcup_{i=1}^t SC_i \right|$, where SC_i is the set of assignments satisfying clause C_i , is the hard part.

10.4.2 Why is DNF Counting #P-Complete?

By De Morgan's laws, the negation of a CNF formula H is a DNF formula $\bar{H}$ with the same number of variables and clauses. The formula H has a satisfying assignment if and only if the number of satisfying assignments of $\bar{H}$ is strictly less than 2^n . Therefore, counting DNF satisfying assignments is at least as hard as solving SAT, making it #P-complete.

10.4.3 The Naïve Sampling Approach

A natural first attempt mimics the Monte Carlo estimation of π : sample truth assignments uniformly at random from all 2^n possibilities and check whether each satisfies F . Let $c(F) = \left| \bigcup_{i=1}^t SC_i \right|$ denote the number of satisfying assignments. Define:

$$\mu = \frac{c(F)}{2^n}.$$

By Problem 17, we need $m \geq \frac{3 \ln(2/\delta)}{\varepsilon^2 \mu} = \frac{3 \cdot 2^n \ln(2/\delta)}{\varepsilon^2 c(F)}$ samples for an (ε, δ) -approximation.

The problem is clear: the required number of samples is proportional to $2^n/c(F)$. If $c(F)$ is polynomial in n (many satisfying assignments relative to the input size, but exponentially few relative to 2^n), then m is *exponential*. The satisfying assignments are simply not dense enough in the sample space $\{0, 1\}^n$ to be found efficiently by uniform sampling.

A direct construction showing that the naïve sampling estimator requires exponentially many samples in the worst case, ruling it out as an FPRAS, is given in the following exercise.

Problem 18: Naïve Sampling Fails

From [MU: Ex. 11.4].

Problem Statement: Consider a class of DNF instances where $c(F) = \text{poly}(n)$. Prove that if you take $2^{n/2}$ uniformly random samples from $\{0, 1\}^n$, the probability of finding even a single satisfying assignment is exponentially small in n .

10.5 An FPRAS for DNF Counting

Since uniform sampling over $\{0,1\}^n$ fails, we need a smarter sample space in which satisfying assignments are *dense*. The idea is to construct a *universe* U that contains all satisfying assignments (with possible duplicates) and in which each assignment appears with known multiplicity.

10.5.1 Constructing the Universe U

Let $F = C_1 \vee C_2 \vee \cdots \vee C_t$ be a DNF formula with t clauses. Clause C_i has l_i literals. Define SC_i as the set of truth assignments satisfying clause C_i . Since satisfying C_i requires fixing l_i variables (one per literal) and the remaining $n - l_i$ variables are free:

$$|SC_i| = 2^{n-l_i}.$$

Define the universe:

$$U = \{(i, a) \mid 1 \leq i \leq t, a \in SC_i\}.$$

Each element of U is a pair: a clause index i and an assignment a that satisfies clause C_i . The size of U is easily computable:

$$|U| = \sum_{i=1}^t |SC_i| = \sum_{i=1}^t 2^{n-l_i}.$$

Note that the same assignment a may appear in multiple pairs (i, a) and (j, a) if a satisfies both C_i and C_j . Therefore $|U| \geq c(F)$.

Example 44. Suppose $n = 3$ and consider subsets $SC_1 = \{1, 3\}$ and $SC_2 = \{1, 2\}$ (where the numbers represent assignments for simplicity). Then:

$$U = \{(1, 1), (1, 3), (2, 1), (2, 2)\}, \quad |U| = 4.$$

The union is $SC_1 \cup SC_2 = \{1, 2, 3\}$, so $c(F) = 3$, but $|U| = 4$ because assignment 1 appears in both SC_1 and SC_2 .

10.5.2 Defining the Canonical Set S

To relate $|U|$ to $c(F)$, we define a subset $S \subseteq U$ that contains exactly one representative for each satisfying assignment. For each assignment $a \in \bigcup SC_i$, we keep the pair (i, a) with the *smallest* clause index i :

$$S = \{(i, a) \in U \mid a \in SC_i \text{ and } a \notin SC_j \text{ for all } j < i\}.$$

By construction, $|S| = c(F) = \left| \bigcup_{i=1}^t SC_i \right|$: each satisfying assignment is counted exactly once.

Example 45 (Continued). From the previous example: assignment 1 belongs to both SC_1 and SC_2 , so its canonical representative is $(1, 1)$ (smallest index). We get $S = \{(1, 1), (1, 3), (2, 2)\}$, and indeed $|S| = 3 = c(F)$.

10.5.3 Estimating $c(F)$ via Sampling from U

We can now estimate $c(F)$ by estimating the ratio $|S|/|U|$:

$$c(F) = |S| = \frac{|S|}{|U|} \cdot |U|.$$

Since $|U|$ is known, it suffices to estimate the ratio $\mu = |S|/|U|$. Since each satisfying assignment belongs to at most t clauses, we have:

$$\mu = \frac{|S|}{|U|} \geq \frac{1}{t}.$$

This is the crucial point: μ is bounded below by $1/t$, which is polynomial in the input size. The satisfying assignments are *dense* in U , unlike in the full space $\{0,1\}^n$.

By Problem 17, the required sample size is:

$$m \geq \frac{3 \ln(2/\delta)}{\varepsilon^2 \mu} \leq \frac{3t \ln(2/\delta)}{\varepsilon^2},$$

which is polynomial in t , $1/\varepsilon$, and $\ln(1/\delta)$.

10.5.4 How to Sample Uniformly from U

To sample a pair (i, a) uniformly from U , we proceed in two steps:

1. **Choose a clause C_i** with probability proportional to $|SC_i|$:

$$\mathbb{P}(\text{choose } C_i) = \frac{|SC_i|}{|U|} = \frac{2^{n-l_i}}{\sum_{j=1}^t 2^{n-l_j}}.$$

2. **Sample an assignment $a \in SC_i$ uniformly at random.** This is easy: fix the l_i variables in clause C_i to their required values, and flip a fair coin for each of the remaining $n - l_i$ variables.

This gives $\mathbb{P}((i, a) \text{ is chosen}) = \frac{|SC_i|}{|U|} \cdot \frac{1}{|SC_i|} = \frac{1}{|U|}$, confirming uniform sampling.

After sampling (i, a) , we check whether $(i, a) \in S$ by verifying that $a \notin SC_j$ for all $j < i$. This can be done in polynomial time by checking each earlier clause.

FPRAS for DNF Counting [MU: Alg. 11.2 & Thm. 11.2]

The sampling procedure above, using $m = \lceil 3t \ln(2/\delta)/\varepsilon^2 \rceil$ samples, gives an (ε, δ) -approximation to $c(F)$. Concretely, for any desired precision ε and confidence $1 - \delta$, the total runtime is $O(t^2 n \varepsilon^{-2} \ln(1/\delta))$: each of $m = O(t \varepsilon^{-2} \ln(1/\delta))$ samples requires $O(tn)$ time to draw from U and verify membership in S . Since t and n are part of the input, this is polynomial in the formula size, $1/\varepsilon$, and $\ln(1/\delta)$, constituting a **fully polynomial randomized approximation scheme** for the #P-complete problem of DNF counting.

11 Approximate Counting, MCMC, and Coupling

Textbook Roadmap

For a parallel treatment in the course textbooks, see:

- *From approximate sampling to approximate counting (FPRAS $\Leftrightarrow$ FPAUS) — telescoping product, self-reducibility, $r_i \geq 1/2$:* [MU: Ch. 11, §11.3, Def. 11.3 (FPAUS); Lem. 11.3 (self-reducibility, $\tilde{r}_i(\varepsilon/2m, \delta/m)$ -approx); Lem. 11.4 & proof ($r_i \geq 1/2$ via injection $I \setminus \{v\}$); Thm. 11.5 (FPAUS $\Rightarrow$ FPRAS for independent sets)].
- *MCMC, detailed balance, Metropolis algorithm:* [MU: Ch. 11, §11.4, Lem. 11.7; §11.4.1].

- *Total variation distance and mixing time:* [MU: Ch. 12, §12.1, Def. 12.1; Lem. 12.1 (sup-form characterization); Def. 12.2 ($\Delta_x(t)$, $\tau(\varepsilon)$)].
- *Coupling of Markov chains, Coupling Lemma:* [MU: Ch. 12, §12.2, Def. 12.3; Lem. 12.2 (Coupling Lemma)].

The FPRAS for DNF counting we just constructed exploited a particular structural fact: the satisfying assignments were dense in the engineered universe U , with $|S|/|U| \geq 1/t$. This made uniform sampling from U a viable proxy for counting S . The DNF case is an instance of a much more general principle: *being able to sample uniformly from an appropriate space is closely tied to being able to count the objects in that space*. We now make this connection precise.

11.1 From Approximate Sampling to Approximate Counting

11.1.1 FPRAS and FPAUS

We have already seen the definition of an FPRAS. Let us now introduce its sampling counterpart.

Definition 46 (FPRAS [MU: Def. 11.2]). A **fully polynomial randomized approximation scheme (FPRAS)** for a counting problem is a randomized algorithm that, given input x and parameters $\varepsilon, \delta \in (0, 1)$, outputs an (ε, δ) -approximation to $|\Omega(x)|$ in time $\text{poly}(|x|, 1/\varepsilon, \ln(1/\delta))$.

Definition 47 (FPAUS [MU: Def. 11.3]). A **fully polynomial almost uniform sampler (FPAUS)** for a problem is a randomized algorithm that, given input x and a parameter $\varepsilon > 0$, generates an ε -uniform sample of the sample space $\Omega(x)$ in time $\text{poly}(|x|, \ln(1/\varepsilon))$.

Here, an ε -uniform sample means that the output distribution is close to uniform in total variation: for any subset $S \subseteq \Omega$,

$$\left| \mathbb{P}(w \in S) - \frac{|S|}{|\Omega|} \right| \leq \varepsilon.$$

Equivalently, writing $\|P - Q\|_{TV} = \max_{S \subseteq \Omega} |\mathbb{P}(S) - Q(S)|$, the requirement is

$$\|M(x) - U(\Omega(x))\|_{TV} \leq \varepsilon,$$

where $U(\Omega(x))$ denotes the uniform distribution on $\Omega(x)$, and the runtime is polynomial in $|x|$ and $\ln(1/\varepsilon)$.

The central result of this section is that, for *self-reducible* problems, these two notions are equivalent:

$$\mathbf{FPRAS} \longleftrightarrow \mathbf{FPAUS}.$$

We demonstrate the direction $\mathbf{FPAUS} \Rightarrow \mathbf{FPRAS}$ by showing how an almost uniform sampler can be used to construct an approximate counter.

The reverse direction ($\mathbf{FPRAS} \Rightarrow \mathbf{FPAUS}$). The converse also holds, though we will not prove it in detail. The high-level idea is as follows: given an FPRAS that can approximately count the number of objects satisfying any set of constraints, we can use *self-reducibility* to sample one element at a time. For example, to sample an independent set nearly uniformly, we decide for each vertex v (in some fixed order) whether v belongs to the set by using the FPRAS to estimate the fraction of independent sets that include v (conditioned on the decisions made so far), and then flipping a biased coin with that probability. Each step requires only an approximate count of a restricted subproblem, which the FPRAS provides. Repeating this for all n vertices produces a sample whose distribution is close to uniform.

11.1.2 Self-Reducible Problems: Independent Sets

We illustrate the reduction using the problem of counting independent sets in a graph $G = (V, E)$ with n nodes and m edges. Let $e_1, e_2, \dots, e_m$ be an arbitrary ordering of the edges, and define:

$$G_i = (V, \{e_1, \dots, e_i\}), \quad i = 0, 1, \dots, m.$$

So $G_m = G$ is the original graph, G_{i-1} is obtained from G_i by removing edge e_i , and G_0 has no edges at all.

Let $\Omega(G_i)$ denote the set of independent sets of G_i . Since G_0 has no edges, *every* subset of V is independent in G_0 , so

$$|\Omega(G_0)| = 2^n.$$

Moreover, removing an edge can only increase the set of independent sets, so

$$\Omega(G_m) \subseteq \Omega(G_{m-1}) \subseteq \dots \subseteq \Omega(G_1) \subseteq \Omega(G_0).$$

This nesting is the key structural property: any independent set of G_i is automatically an independent set of G_{i-1} , since G_{i-1} has fewer edge constraints.

11.1.3 The Telescoping Product

Define the ratios

$$r_i = \frac{|\Omega(G_i)|}{|\Omega(G_{i-1})|}, \quad i = 1, \dots, m.$$

Then we can write $|\Omega(G)|$ as a telescoping product:

$$|\Omega(G)| = |\Omega(G_m)| = \frac{|\Omega(G_m)|}{|\Omega(G_{m-1})|} \times \frac{|\Omega(G_{m-1})|}{|\Omega(G_{m-2})|} \times \dots \times \frac{|\Omega(G_1)|}{|\Omega(G_0)|} \times |\Omega(G_0)| = \left(\prod_{i=1}^m r_i \right) \cdot 2^n.$$

The intermediate terms cancel, leaving only $|\Omega(G_m)|$ in the numerator and $|\Omega(G_0)| = 2^n$ in the denominator. Since 2^n is known exactly, the problem reduces to estimating each ratio r_i .

Now, to estimate $r_i = |\Omega(G_i)|/|\Omega(G_{i-1})|$, we can sample (almost) uniformly from $\Omega(G_{i-1})$ using an **FPAUS** and check whether the sample also belongs to $\Omega(G_i)$. The fraction of samples that pass this test approximates r_i . This is precisely how an FPAUS yields an FPRAS.

11.1.4 The Self-Reducibility Lemma

To turn approximate estimates $\tilde{r}_i$ into an overall (ε, δ) -approximation for $|\Omega(G_m)|$, we need each individual estimate to be sufficiently accurate.

Lemma 48 (Self-Reducibility Lemma [MU: Lem. 11.3]). *If, for all $i = 1, \dots, m$, $\tilde{r}_i$ is an $(\varepsilon/2m, \delta/m)$ -approximation for r_i , then $2^n \prod_{i=1}^m \tilde{r}_i$ is an (ε, δ) -approximation for $|\Omega(G_m)|$.*

Proof. For each i , by definition of an $(\varepsilon/2m, \delta/m)$ -approximation:

$$\mathbb{P}\left(|\tilde{r}_i - r_i| > \frac{\varepsilon}{2m} r_i\right) < \frac{\delta}{m}.$$

By the union bound over $i = 1, \dots, m$:

$$\mathbb{P}\left(|\tilde{r}_i - r_i| > \frac{\varepsilon}{2m} r_i \text{ for some } i\right) < \delta.$$

Equivalently,

$$\mathbb{P}\left(\left|\frac{\tilde{r}_i}{r_i} - 1\right| \leq \frac{\varepsilon}{2m} \quad \forall i\right) \geq 1 - \delta.$$

When this event holds, we have $1 - \varepsilon/2m \leq \tilde{r}_i/r_i \leq 1 + \varepsilon/2m$ for all i , so

$$\left(1 - \frac{\varepsilon}{2m}\right)^m \leq \prod_{i=1}^m \frac{\tilde{r}_i}{r_i} \leq \left(1 + \frac{\varepsilon}{2m}\right)^m.$$

Since $(1 + \varepsilon/2m)^m \leq e^{\varepsilon/2} \leq 1 + \varepsilon$ for $\varepsilon \leq 1$ (and similarly $(1 - \varepsilon/2m)^m \geq 1 - \varepsilon$), we conclude:

$$\mathbb{P}\left(\left|2^n \prod_{i=1}^m \tilde{r}_i - |\Omega(G_m)|\right| \leq \varepsilon |\Omega(G_m)|\right) > 1 - \delta. \quad \square$$

11.1.5 Estimating r_i via Sampling

By Lemma 48, it suffices to obtain an $(\varepsilon/2m, \delta/m)$ -approximation for each r_i . We do this in two steps.

Step 1: From FPAUS to biased samples. Suppose we have an $(\varepsilon/6m)$ -FPAUS for $\Omega(G_{i-1})$. We draw M independent samples from this FPAUS and define

$$X_j = \begin{cases} 1 & \text{if the } j\text{-th sample from } \Omega(G_{i-1}) \text{ is also in } \Omega(G_i), \\ 0 & \text{otherwise,} \end{cases}$$

and set

$$\tilde{r}_i = \frac{1}{M} \sum_{j=1}^M X_j.$$

Since the samples are not exactly uniform but $(\varepsilon/6m)$ -close, we have a small bias:

$$|\mathbb{P}(X_j = 1) - r_i| < \frac{\varepsilon}{6m}.$$

By linearity of expectation,

$$|\mathbb{E}[\tilde{r}_i] - r_i| = \left| \mathbb{E}\left[\frac{1}{M} \sum_{j=1}^M X_j\right] - r_i \right| < \frac{\varepsilon}{6m}.$$

Step 2: Concentration around the mean. We now need $\tilde{r}_i$ to concentrate around $\mathbb{E}[\tilde{r}_i]$. Since the X_j 's are i.i.d. indicator variables, the sample-complexity result from Problem 17 applies. Specifically, the bound

$$m_{\text{samples}} \geq \frac{3 \ln(2/\delta')}{\varepsilon'^2 \mu}$$

guarantees $\mathbb{P}(|\tilde{r}_i - \mathbb{E}[\tilde{r}_i]| > \varepsilon' \mathbb{E}[\tilde{r}_i]) < \delta'$. Setting $\varepsilon' = \varepsilon/12m$ and $\delta' = \delta/m$, we need a lower bound on $\mathbb{E}[\tilde{r}_i]$.

Why $r_i \geq 1/2$. Since $\Omega(G_i) \subseteq \Omega(G_{i-1})$, we can write

$$r_i = \frac{|\Omega(G_i)|}{|\Omega(G_{i-1})|} = \frac{|\Omega(G_i)|}{|\Omega(G_i)| + |\Omega(G_{i-1}) \setminus \Omega(G_i)|}.$$

The graphs G_{i-1} and G_i differ by a single edge $\{u, v\}$. An independent set $I \in \Omega(G_{i-1}) \setminus \Omega(G_i)$ must contain both u and v . We can associate each such I with the independent set $I' = I \setminus \{v\} \in \Omega(G_i)$. Since at most one I maps to each I' under this association, we have $|\Omega(G_{i-1}) \setminus \Omega(G_i)| \leq |\Omega(G_i)|$, giving

$$r_i \geq \frac{1}{2}.$$

This $1/2$ lower bound is exactly what makes the whole construction polynomial. Recall that $|\Omega(G)| = 2^n \prod_{i=1}^m r_i$, so each multiplicative factor we have to estimate is a number in $[1/2, 1]$ — a constant range, independent of input size. If instead the r_i were allowed to shrink rapidly with i (say, exponentially), each $\tilde{r}_i$ would have to resolve a diminishing target with relative error $O(\varepsilon/m)$, and the sample size would blow up exponentially. We saw exactly this failure mode in the previous lecture's naive Monte Carlo attempt at DNF counting: the ratio $c(F)/2^n$ could be exponentially small, making uniform sampling useless. The self-reducible chain $G_0 \subset G_1 \subset \dots \subset G_m$ is engineered precisely so that consecutive terms differ by a single edge — a small enough perturbation that r_i cannot collapse below $1/2$.

Using $r_i \geq 1/2$ and the bias bound $|\mathbb{E}[\tilde{r}_i] - r_i| < \varepsilon/6m$:

$$\mathbb{E}[\tilde{r}_i] \geq r_i - \frac{\varepsilon}{6m} \geq \frac{1}{2} - \frac{1}{6} = \frac{1}{3}.$$

Combining bias and concentration. From the FPAUS bias bound and $r_i \geq 1/2$:

$$1 - \frac{\varepsilon}{3m} \leq 1 - \frac{\varepsilon}{6mr_i} < \frac{\mathbb{E}[\tilde{r}_i]}{r_i} < 1 + \frac{\varepsilon}{6mr_i} \leq 1 + \frac{\varepsilon}{3m}.$$

From concentration, with probability at least $1 - \delta/m$:

$$1 - \frac{\varepsilon}{12m} < \frac{\tilde{r}_i}{\mathbb{E}[\tilde{r}_i]} < 1 + \frac{\varepsilon}{12m}.$$

Multiplying these two sets of bounds:

$$\left(1 - \frac{\varepsilon}{3m}\right) \left(1 - \frac{\varepsilon}{12m}\right) \leq \frac{\tilde{r}_i}{r_i} \leq \left(1 + \frac{\varepsilon}{3m}\right) \left(1 + \frac{\varepsilon}{12m}\right).$$

A short calculation confirms that the upper bound satisfies

$$\left(1 + \frac{\varepsilon}{3m}\right) \left(1 + \frac{\varepsilon}{12m}\right) = 1 + \frac{\varepsilon}{3m} + \frac{\varepsilon}{12m} + \frac{\varepsilon^2}{36m^2} < 1 + \frac{\varepsilon}{2m},$$

and similarly the lower bound exceeds $1 - \varepsilon/2m$. This gives the required $(\varepsilon/2m, \delta/m)$ -approximation for r_i .

11.1.6 Sample Size

From the concentration step, the number of samples per ratio is

$$M = \left\lceil \frac{3 \ln(2m/\delta)}{(\varepsilon/12m)^2 \cdot \mathbb{E}[\tilde{r}_i]} \right\rceil.$$

Using $\mathbb{E}[\tilde{r}_i] \geq 1/3$:

$$M \leq \frac{3 \ln(2m/\delta)}{(\varepsilon/12m)^2 \cdot (1/3)} = \frac{9 \ln(2m/\delta) \cdot 144m^2}{\varepsilon^2} = \frac{1296 m^2 \ln(2m/\delta)}{\varepsilon^2},$$

which is polynomial in m , $1/\varepsilon$, and $\ln(1/\delta)$.

FPAUS $\Rightarrow$ FPRAS for Self-Reducible Problems [MU: Thm. 11.5]

Given a fully polynomial almost uniform sampler (FPAUS) for independent sets in any graph, we can construct a fully polynomial randomized approximation scheme (FPRAS) for counting independent sets. The reduction uses the telescoping product $|\Omega(G)| = 2^n \prod r_i$ and requires $O(m^3 \varepsilon^{-2} \ln(m/\delta))$ total samples.

Applying this self-reducibility framework to count the number of feasible knapsack solutions is developed in the following exercise.

Problem 19: Counting Knapsack Solutions

From [MU: Ex. 11.6].

Problem Statement: The problem of counting the number of solutions to a knapsack instance can be defined as follows: Given items with sizes $a_1, a_2, \dots, a_n > 0$ and an integer $b > 0$, find the number of vectors $(x_1, x_2, \dots, x_n) \in \{0, 1\}^n$ such that $\sum_{i=1}^n a_i x_i \leq b$.

- A naïve way of counting the number of solutions to this problem is to repeatedly choose $(x_1, x_2, \dots, x_n) \in \{0, 1\}^n$ uniformly at random, and return 2^n times the fraction of samples that yield valid solutions. Argue why this is not a good strategy in general; in particular, argue that it will work poorly when each a_i is 1 and $b = \sqrt{n}$.
- Consider a Markov chain $X_0, X_1, \dots$ on vectors $(x_1, x_2, \dots, x_n) \in \{0, 1\}^n$. Suppose X_j is $(x_1, x_2, \dots, x_n)$. At each step, the Markov chain chooses $i \in [1, n]$ uniformly at random. If $x_i = 1$, then X_{j+1} is obtained from X_j by setting x_i to 0. If $x_i = 0$, then X_{j+1} is obtained from X_j by setting x_i to 1 if doing so maintains the restriction $\sum_{i=1}^n a_i x_i \leq b$. Otherwise, $X_{j+1} = X_j$.

Argue that this Markov chain has a uniform stationary distribution whenever $\sum_{i=1}^n a_i > b$. Be sure to argue that the chain is irreducible and aperiodic.

- Argue that, if we have an FPAUS for the knapsack problem, then we can derive an FPRAS for the problem. To set the problem up properly, assume without loss of generality that $a_1 \leq a_2 \leq \dots \leq a_n$. Let $b_0 = 0$ and $b_i = \sum_{j=1}^i a_j$. Let $\Omega(b_i)$ be the set

of vectors $(x_1, x_2, \dots, x_n) \in \{0, 1\}^n$ that satisfy $\sum_{j=1}^n a_j x_j \leq b_i$. Let k be the smallest integer such that $b_k \geq b$. Consider the equation

$$|\Omega(b)| = \frac{|\Omega(b)|}{|\Omega(b_{k-1})|} \times \frac{|\Omega(b_{k-1})|}{|\Omega(b_{k-2})|} \times \dots \times \frac{|\Omega(b_1)|}{|\Omega(b_0)|} \times |\Omega(b_0)|.$$

You will need to argue that $|\Omega(b_{i-1})|/|\Omega(b_i)|$ is not too small. Specifically, argue that $|\Omega(b_i)| \leq (n+1)|\Omega(b_{i-1})|$.

11.2 Markov Chain Monte Carlo (MCMC)

The Monte Carlo method is based on sampling. It is often difficult to generate a random sample with the required probability distribution. For example, we saw in the previous section that we can count independent sets in a graph if we can generate an almost uniform sample from the set of independent sets. But how can we generate such a sample?

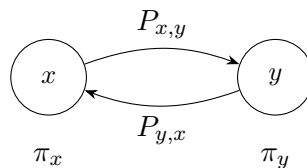
The **Markov chain Monte Carlo (MCMC)** method provides a general approach to sampling from a desired probability distribution. The basic idea is to define an ergodic Markov chain whose set of states is the sample space Ω and whose stationary distribution π is the required sampling distribution. After running the chain for sufficiently many steps, the distribution of the current state will be close to π , giving us an approximate sample.

11.2.1 Time Reversibility and Detailed Balance

A Markov chain with transition matrix P and stationary distribution π is said to satisfy **detailed balance** if

$$\pi_x P_{x,y} = \pi_y P_{y,x} \quad \forall x, y \in \Omega.$$

This says that, in stationarity, the “flow” from x to y equals the flow from y to x . Any distribution satisfying detailed balance is a stationary distribution, since summing over x gives $\pi^T P = \pi^T$.



For the MCMC approach to work, we need two properties of the chain:

- **Irreducible:** For every pair of states $x, y \in \Omega$, there exists a path of positive-probability transitions from x to y . This ensures the chain can reach any state from any starting point.
- **Aperiodic:** The greatest common divisor of the set of return times to any state is 1. This prevents the chain from oscillating between groups of states.

A Markov chain that is both irreducible and aperiodic is called *ergodic*. For an ergodic chain, the stationary distribution π is unique, and π satisfies

$$\pi^T = \pi^T P.$$

That is, π is a left eigenvector of P with eigenvalue 1. Starting from any initial state x , the distribution of the chain after t steps converges to π as $t \rightarrow \infty$.

11.3 Total Variation Distance and Mixing Time

The key question for MCMC is: *how many steps t are needed before the chain's distribution is close enough to π ?* To answer this, we need to formalize what “close” means.

11.3.1 Total Variation Distance

Let P_x^t denote the distribution over states after t steps, starting from state x .

Definition 49 (Total Variation Distance [MU: Def. 12.1 & Lem. 12.1]). The **total variation distance** between two distributions Π_1 and Π_2 on a finite set Ω is

$$\|\Pi_1 - \Pi_2\|_{TV} = \frac{1}{2} \sum_{z \in \Omega} |\Pi_1(z) - \Pi_2(z)| = \max_{S \subseteq \Omega} |\Pi_1(S) - \Pi_2(S)|.$$

The equality between these two expressions is not obvious. Establishing it, together with several other equivalent reformulations of total variation distance (including the supremum over events and the half- L^1 distance), is the content of the following exercise.

Problem 20: Equivalent Definitions of Total Variation

From [MU: Lem. 12.1] (in-chapter result; the homework asks the student to verify the chain of equalities).

Problem Statement: Let Π_1 and Π_2 be two probability distributions on a finite set Ω . Define

$$S^+ = \{x \in \Omega : \Pi_1(x) \geq \Pi_2(x)\}, \quad S^- = \{x \in \Omega : \Pi_1(x) < \Pi_2(x)\}.$$

Prove that all four quantities in the following chain of equalities are equal:

$$\frac{1}{2} \sum_{x \in \Omega} |\Pi_1(x) - \Pi_2(x)| = \max_{S \subseteq \Omega} |\Pi_1(S) - \Pi_2(S)| = \Pi_1(S^+) - \Pi_2(S^+) = \Pi_2(S^-) - \Pi_1(S^-).$$

11.3.2 Mixing Time

We now define the rate of convergence to stationarity.

Definition 50 (Mixing Time [MU: Def. 12.2]). For each starting state $x \in \Omega$, define

$$\Delta_x(t) = \|P_x^t - \pi\|_{TV}.$$

The worst-case distance at time t is

$$\Delta(t) = \max_{x \in \Omega} \Delta_x(t).$$

The **mixing time** of the Markov chain is the function

$$\tau(\varepsilon) = \max_{x \in \Omega} \tau_x(\varepsilon) = \max_{x \in \Omega} \min\{t : \Delta_x(t) \leq \varepsilon\}.$$

For MCMC to be efficient, we need $\tau(\varepsilon) = \text{poly}(\text{problem size}, \ln(1/\varepsilon))$. A chain with polynomial mixing time is said to be **rapidly mixing**.

An equivalent reformulation of the mixing-time definition, which is often more convenient in practice, is the subject of the following exercise.

Problem 21: An Equivalent Form of the Mixing Time

From [MU: Def. 12.2] (in-chapter; the homework asks the student to verify the equivalence of the two given forms of $\tau(\varepsilon)$).

Problem Statement: Show that the mixing time can equivalently be written as

$$\tau(\varepsilon) = \min\{t : \Delta(t) \leq \varepsilon\}.$$

That is, show that $\max_x \min\{t : \Delta_x(t) \leq \varepsilon\} = \min\{t : \max_x \Delta_x(t) \leq \varepsilon\}$.

Hint: Use the fact that $\Delta(t) = \max_x \Delta_x(t)$, and observe that the max and min can be interchanged because $\Delta_x(t)$ is non-increasing in t for each x .

11.4 Coupling of Markov Chains

To show that a chain mixes in polynomial time, we need a method for bounding $\tau(\varepsilon)$. A powerful technique for this is **coupling**.

11.4.1 The Coupling Idea

The idea is to run *two* copies of the Markov chain simultaneously, one starting from an arbitrary state x and the other from an arbitrary state y , and design their joint evolution so that they tend to “meet” (reach the same state) quickly.

Formally, a **coupling** of a Markov chain is a process $Z_t = (X_t, Y_t)$ on the product space $\Omega \times \Omega$ such that each component, viewed marginally, is a faithful copy of the original chain:

$$\mathbb{P}(X_{t+1} = x \mid Z_t = (x', y')) = \mathbb{P}(X_{t+1} = x \mid X_t = x').$$

That is, the marginal transitions of X_t (and similarly Y_t) obey the original chain’s transition probabilities, but X_t and Y_t may be correlated.

11.4.2 The Coupling Lemma

The following lemma shows that a good coupling directly implies a bound on the mixing time.

Lemma 51 (Coupling Lemma [MU: Def. 12.3 & Lem. 12.2]). *If there exists a coupling (X_t, Y_t) such that*

$$\mathbb{P}(X_T \neq Y_T \mid X_0 = x, Y_0 = y) \leq \varepsilon \quad \forall x, y \in \Omega,$$

then $\tau(\varepsilon) \leq T$.

The power of this lemma is that it converts a *probabilistic convergence* question (how fast does P_x^t approach π ?) into a *combinatorial meeting* question (how fast can two coupled copies collide?).

Designing clever couplings is the primary technique for proving rapid mixing of Markov chains, which we will explore in subsequent lectures.

Further Reading

This section introduced total variation distance, mixing times, and coupling as tools for analysing convergence of Markov chains. The reading below opens three doors out of these foundations: a comprehensive textbook on the area, an optimization view of the mixing problem, and a landmark application to card shuffling.

- Levin et al. [24], *Markov Chains and Mixing Times* (2nd ed.). The standard graduate-level textbook covering essentially every topic in this section in depth: total variation distance (Ch. 4.1), coupling and the coupling bound on total variation distance (Ch. 5), mixing time (Ch. 4.5), strong stationary times (Ch. 6), lower bounds on mixing (Ch. 7), spectral methods and the relaxation time (Ch. 12), and path coupling (Ch. 14). Worked examples include shuffling cards (Ch. 8), random walks on networks (Ch. 9), and hitting and cover times (Chs. 10–11).
- Boyd et al. [25], “Fastest Mixing Markov Chain on a Graph.” Inverts the question studied in this section: instead of analysing the mixing time of a *given* chain, the paper asks how to *design* the edge transition probabilities of a symmetric chain on a graph so as to *minimize* the second-largest eigenvalue modulus (SLEM) of the transition matrix, and hence its mixing rate. The optimization is formulated as a convex problem expressible as a semidefinite program (SDP); the authors show that the resulting Fastest Mixing Markov Chain often substantially outperforms the maximum-degree method and the Metropolis–Hastings algorithm as fast-mixing heuristics.
- Bayer and Diaconis [26], “Trailing the Dovetail Shuffle to its Lair.” A landmark application of total variation distance and mixing time to the riffle (“dovetail”) shuffle. Under the Gilbert–Shannon–Reeds model, the authors derive a closed-form expression for the post-shuffle distribution and prove the sharp threshold $m = \frac{3}{2} \log_2 n + \theta$: $\frac{3}{2} \log_2 n + \theta$ shuffles are necessary and sufficient to mix up n cards. For a standard deck of $n = 52$ cards, the total variation distance drops from 0.924 at $m = 5$ shuffles to 0.334 at $m = 7$ to 0.043 at $m = 10$, giving the famous “seven shuffles suffice” folklore.
- Wakin and Rozell [27], “A Markov Chain Analysis of Blackjack Strategy.” Applies the techniques of this section to a non-trivial decision-theoretic setting. The analysis couples two Markov chains: an absorbing chain that models a single hand (with states given by player and dealer totals, and absorbing states win, lose, push, and blackjack), and an ergodic chain whose stationary distribution captures the long-run composition of the shoe under a given card-counting system. The product of these two chains reproduces the per-hand edge reported by classical card-counting analyses, illustrating how transition matrices, stationary distributions, and absorbing states yield quantitative predictions outside the standard MCMC and mixing-time setting. The manuscript is a Rice University technical report, useful as a worked example rather than a primary citation.

References

- [1] N. Alon and J. Spencer. *The Probabilistic Method*. Wiley, 4th ed., 2016.
- [2] M. Mitzenmacher and E. Upfal. *Probability and Computing*. Cambridge University Press, 2017.
- [3] M. Harchol-Balter. *Introduction to Probability for Computing*. Cambridge University Press, 2023.
- [4] C. Dwork. Differential privacy. In *Proc. 33rd International Colloquium on Automata, Languages and Programming (ICALP)*, Part II, LNCS 4052, pp. 1–12. Springer, 2006.
- [5] C. Dwork, F. McSherry, K. Nissim, and A. Smith. Calibrating noise to sensitivity in private data analysis. In *Proc. 3rd Theory of Cryptography Conference (TCC)*, LNCS 3876, pp. 265–284. Springer, 2006.
- [6] F. Koufogiannis, S. Han, and G. J. Pappas. Optimality of the Laplace mechanism in differential privacy. *arXiv preprint arXiv:1504.00065*, 2015.
- [7] M. Papachristou and M. A. Rahimian. Differentially private distributed estimation and learning. *arXiv preprint arXiv:2306.15865*, 2023.
- [8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*. MIT Press, 4th ed., 2022.
- [9] U. Feige. A threshold of $\ln n$ for approximating set cover. *Journal of the ACM*, 45(4):634–652, 1998.
- [10] J. B. Shearer. On a problem of Spencer. *Combinatorica*, 5(3):241–245, 1985.
- [11] R. A. Moser and G. Tardos. A constructive proof of the general Lovász local lemma. *Journal of the ACM*, 57(2), Article 11, pp. 1–15, 2010.
- [12] F. McSherry and K. Talwar. Mechanism design via differential privacy. In *Proc. 48th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pp. 94–103, 2007. DOI: 10.1109/FOCS.2007.66.
- [13] J. Awan, A. Kenney, M. Reimherr, and A. Slavković. Benefits and pitfalls of the exponential mechanism with applications to Hilbert spaces and functional PCA. *arXiv preprint arXiv:1901.10864*, 2019.
- [14] P. Kairouz, S. Oh, and P. Viswanath. The composition theorem for differential privacy. *IEEE Transactions on Information Theory*, 63(6):4037–4049, 2017. DOI: 10.1109/TIT.2017.2685505. Conference version in *Proc. 32nd International Conference on Machine Learning (ICML)*, pp. 1376–1385, 2015.
- [15] A. Gupta, K. Ligett, F. McSherry, A. Roth, and K. Talwar. Differentially private combinatorial optimization. In *Proc. 21st Annual ACM–SIAM Symposium on Discrete Algorithms (SODA)*, pp. 1106–1125, 2010.
- [16] A. Gupta, G. Guruganesh, B. Peng, and D. Wajc. Stochastic online metric matching. In *Proc. 46th International Colloquium on Automata, Languages, and Programming (ICALP)*, LIPIcs vol. 132, pp. 67:1–67:14, 2019. Also available as *arXiv preprint arXiv:1904.09284*.

- [17] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang. Deep learning with differential privacy. In *Proc. 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 308–318, 2016. DOI: 10.1145/2976749.2978318.
- [18] A. Georgakopoulos, J. Haslegrave, T. Sauerwald, and J. Sylvester. The power of two choices for random walks. *arXiv preprint arXiv:1911.05170*, 2021. Preliminary version in *Proc. 11th Innovations in Theoretical Computer Science Conference (ITCS 2020)*, vol. 151 of LIPIcs, pp. 76:1–76:19.
- [19] M. Papachristou and J. Kleinberg. The power of choice in random sampling. SSRN preprint, abstract id 5748962, 2025.
- [20] I. Ashlagi, S. Kerimov, O. Tamuz, and G. Zhao. The power of two in token systems. *Management Science*, Articles in Advance, pp. 1–21, 2026. DOI: 10.1287/mnsc.2024.06176.
- [21] B. Mirzasoleiman, A. Badanidiyuru, A. Karbasi, J. Vondrák, and A. Krause. Lazier than lazy greedy. In *Proc. AAAI Conference on Artificial Intelligence*, 2015. Also available as *arXiv preprint arXiv:1409.7938*.
- [22] N. Buchbinder, M. Feldman, J. Naor, and R. Schwartz. A tight linear time $(1/2)$ -approximation for unconstrained submodular maximization. In *Proc. 53rd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pp. 649–658, 2012. DOI: 10.1109/FOCS.2012.73. Journal version in *SIAM Journal on Computing*, 44(5):1384–1402, 2015.
- [23] R. Kumar, B. Moseley, S. Vassilvitskii, and A. Vattani. Fast greedy algorithms in MapReduce and streaming. In *Proc. 25th Annual ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, pp. 1–10, 2013. DOI: 10.1145/2486159.2486168.
- [24] D. A. Levin, Y. Peres, and E. L. Wilmer. *Markov Chains and Mixing Times*. 2nd ed., 2009.
- [25] S. Boyd, P. Diaconis, and L. Xiao. Fastest mixing Markov chain on a graph. *SIAM Review*, 46(4):667–689, 2004. DOI: 10.1137/S0036144503423264.
- [26] D. Bayer and P. Diaconis. Trailing the dovetail shuffle to its lair. *The Annals of Applied Probability*, 2(2):294–313, 1992.
- [27] M. B. Wakin and C. J. Rozell. A Markov chain analysis of blackjack strategy. Technical report, Department of Electrical and Computer Engineering, Rice University, 2004. Available via the Rice University Digital Scholarship Archive.
- [28] R. Motwani and P. Raghavan. *Randomized Algorithms*. Cambridge University Press, 1995.